



UNIVERSITÄT
BAYREUTH

**Eine direkte Methode für
Optimalsteuerungsprobleme mit parabolischen
Differentialgleichungen**

Diplomarbeit

von

Florian Loos

FAKULTÄT FÜR MATHEMATIK, PHYSIK UND INFORMATIK
MATHEMATISCHES INSTITUT

Aufgabenstellung / Betreuung:

Prof. Dr. Hans Josef Pesch
Prof. Dr. Kurt Chudej

Datum:

23. Dezember 2008

Danksagung

An dieser Stelle möchte ich mich bei Herrn Prof. Dr. Hans Josef Pesch sowie seinem Assistenten Herrn Armin Rund für die hervorragende Betreuung und Unterstützung während der Erstellung meiner Diplomarbeit bedanken.

Den Mitarbeitern der Firma inuTech GmbH, insbesondere Herrn Frank Vogel, Herrn Sebastian Götschel, Herrn Frantz Tomety und Frau Cathleen Seidel, danke ich für die Hilfestellungen und die angenehme Arbeitsatmosphäre während des letzten Jahres.

Mein ganz besonderer Dank gilt meinen Eltern, die mich während meiner gesamten Ausbildung unterstützt und damit mein Mathematikstudium ermöglicht haben.

Inhaltsverzeichnis

Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
1 Einleitung	1
2 Partielle Differentialgleichungen	4
2.1 Motivation	4
2.2 Klassifikation partieller Differentialgleichungen	6
2.3 Anfangs- und Randbedingungen	8
2.3.1 Anfangsbedingungen	8
2.3.2 Randbedingungen	8
2.4 Steuerungsarten	9
2.4.1 Verteilte Steuerungen	9
2.4.2 Randsteuerungen	10
3 Problemstellung und funktionalanalytische Grundlagen	11
3.1 Problemstellung	11
3.2 Sobolev-Räume	12
3.2.1 Lösungsräume für elliptische Differentialgleichungen	13
3.2.2 Lösungsräume für parabolische Differentialgleichungen	16
3.3 Schwache Formulierung	18
4 Diskretisierungsmethoden	20
4.1 Zeitdiskretisierung mittels Finiten Differenzen	21
4.1.1 Allgemeine Vorgehensweise	21
4.1.2 Zeitdiskretisierung des Modellproblems	22
4.2 Ortsdiskretisierung mittels Finiten Elemente	22
4.2.1 Allgemeine Vorgehensweise und Definition eines Finiten Elements	22
4.2.2 Weitere Details zur Methode der Finiten Elemente	25
4.2.3 Anwendung der FEM auf das Modellproblem	32
5 Nichtlineare Optimierungsprobleme	34
5.1 Allgemeines nichtlineares Problem, Optimalitätsbedingungen und konvexe Optimierung	34
5.1.1 Optimalitätsbedingungen erster Ordnung	36
5.1.2 Optimalitätsbedingungen zweiter Ordnung	37
5.1.3 Konvexe Optimierung	37
5.2 Methoden zur Lösung nichtlinearer Optimierungsprobleme	39
5.2.1 Innere-Punkte-Methoden	39
5.2.2 Details zum in IPOPT implementierten primal-dualen Innere-Punkte-Algorithmus mit Filter-Line-Search	40
5.2.3 Methode der beweglichen Asymptoten	48
5.2.4 Sequentielle konvexe Optimierung	51

6	Implementierung	54
6.1	Bei der Implementierung eingesetzte mathematische Verfahren	54
6.1.1	Gradientenbestimmung mittels Finiten Differenzen	54
6.1.2	Behandlung von Nichtlinearitäten mit dem Newton-Raphson-Verfahren	56
6.2	DIFFPACK	57
6.2.1	Prinzipieller Aufbau eines DIFFPACK-Simulators	58
6.2.2	Konkrete Umsetzung des Löser zur Optimalsteuerung in DIFFPACK .	62
6.3	Aufruf der Optimierer aus der DIFFPACK-Umgebung	71
6.3.1	Aufruf von IPOPT aus DIFFPACK	71
6.3.2	Aufruf von SCIP aus DIFFPACK	74
7	Numerische Untersuchungen und Resultate	77
7.1	Testbeispiel 1	78
7.2	Testbeispiel 2	82
7.3	Testbeispiel 3	87
7.4	Testbeispiel 4	90
7.5	Testbeispiel 5	92
8	Zusammenfassung und Ausblick	98
A	Anhang	100
A.1	Abkürzungen und Notationen	100
A.1.1	Abkürzungsverzeichnis	100
A.1.2	Notationen	100
A.2	DIFFPACK-spezifische Datentypen, Klassen und Funktionen	102
A.3	Inhalt der Daten-CD-ROM	105
A.4	Erklärung der Menüeinträge eines Datenfiles	105
	Literaturverzeichnis	108
	Erklärung	110

Abbildungsverzeichnis

2.1	Verteilte Steuerung	9
2.2	Randsteuerung	10
4.1	Vertikale und horizontale Linienmethode	20
4.2	Triangulierung eines kreisförmigen Gebiets mittels Dreieckselementen	23
4.3	Stückweise lineare 1D-Ansatzfunktionen	26
4.4	Quadratische 1D-Basisfunktionen im Referenzelement	27
4.5	Beispiel einer diskretisierten, nichttrivialen Geometrie mit verschiedenförmigen 2D-Finiten Elementen	27
4.6	Lineares und quadratisches Lagrange-Dreiecks-Element	28
4.7	Bilineares und biquadratisches Lagrange-Rechtecks-Element	28
4.8	Lineare 2D-Ansatzfunktionen bei Dreieckselementen	28
4.9	Transformation eines zweidimensionalen Gebiets auf das zugehörige Vierecks- Referenzelement	29
4.10	Transformation eines zweidimensionalen Gebiets auf das zugehörige Dreiecks- Referenzelement	30
6.1	DIFFPACK-Programmablauf bei linearen Problemen	59
6.2	Zeitschleife zum Lösen einer zeitabhängigen linearen PDE	60
6.3	DIFFPACK-Programmablauf bei nichtlinearen Problemen	61
6.4	Zeitschleife zum Lösen einer zeitabhängigen nichtlinearen PDE	62
6.5	Klassendiagramm der implementierten Löser	62
6.6	Gradientenberechnung des Zielfunktional	65
6.7	Berechnung des Zielfunktionswerts durch die Funktion <code>evaluateF()</code>	66
6.8	Ablauf der Optimierungsroutine	70
7.1	Bang-Bang-Steuerung für Regularisierungsfaktor $\lambda = 0$	87

Tabellenverzeichnis

4.1	Beispiele für Finite Elemente und ihre Abbildung auf lokale Referenzelemente .	31
7.1	Problemdaten für die Auswertung von Beispiel 1 mit grober Diskretisierung . .	78
7.2	Zusammenfassung der Ergebnisse für Beispiel 1 bei grober Diskretisierung . . .	78
7.3	Vergleich der numerisch und analytisch berechneten Zustandswerte bei grober Diskretisierung für Beispiel 1	80
7.4	Problemdaten für die Auswertung von Beispiel 1 mit feiner Diskretisierung . .	80
7.5	Zusammenfassung der Ergebnisse für Beispiel 1 bei feiner Diskretisierung . . .	81
7.6	Vergleich der numerisch und analytisch berechneten Zustandswerte bei feiner Diskretisierung für Beispiel 1	81
7.7	Problemdaten für die Auswertung von Beispiel 2 bei grober Diskretisierung . .	82
7.8	Zusammenfassung der Ergebnisse für Beispiel 2 bei grober Diskretisierung . . .	83
7.9	Problemdaten für die Auswertung von Beispiel 2 bei feiner Diskretisierung . . .	83
7.10	Zusammenfassung der Ergebnisse für Beispiel 2 bei feiner Diskretisierung . . .	83
7.11	Numerisch und analytisch berechnete Steuerungen für Testbeispiel 2 bei feiner Diskretisierung mit $\theta = 1.0$ bzw. $\theta = 0.5$	84
7.12	Vergleich der numerisch und analytisch berechneten Zustandswerte bei feiner Diskretisierung für Beispiel 2	86
7.13	Problemdaten für die Auswertung von Beispiel 3 bei grober Diskretisierung . .	88
7.14	Zusammenfassung der Ergebnisse für Beispiel 3 bei grober Diskretisierung . . .	88
7.15	Vergleich der numerisch berechneten Steuerungswerte und der Ergebnisse aus [10] für Beispiel 3 bei feiner Diskretisierung	88
7.16	Problemdaten für die Auswertung von Beispiel 3 bei feiner Diskretisierung . . .	89
7.17	Zusammenfassung der Ergebnisse für Beispiel 3 bei feiner Diskretisierung . . .	89
7.18	Problemdaten für die Auswertung von Beispiel 4	90
7.19	Vergleich der numerisch berechneten Steuerungswerte (links) und der Ergebnisse aus [10] (rechts) für Beispiel 4	90
7.20	Problemdaten für die Auswertung von Beispiel 5 bei grober Diskretisierung . .	93
7.21	Zusammenfassung der Ergebnisse für Testbeispiel 5 bei grober Diskretisierung .	93
7.22	Vergleich der numerisch und analytisch berechneten Zustandswerte bei grober Diskretisierung für Beispiel 5	96
7.23	Problemdaten für die Auswertung von Beispiel 5 bei feiner Diskretisierung . . .	96
7.24	Zusammenfassung der Ergebnisse für Testbeispiel 5 bei feiner Diskretisierung .	97

1 Einleitung

Differentialgleichungen finden Anwendung in sehr vielen physikalischen und technischen Gebieten, in biologischen und chemischen Prozessen, aber auch alltägliche Vorgänge wie das Garen einer Kartoffel lassen sich mit ihnen beschreiben. Insbesondere partielle Differentialgleichungen, d. h. Gleichungen, die eine Funktion und deren partielle Ableitungen enthalten, sind ein wichtiges Mittel, um Abläufe sehr genau mit mathematischen Modellen wiederzugeben. Sie verhelfen somit dazu, eine möglichst detailgetreue Abbildung der Wirklichkeit zu schaffen.

Parabolische Differentialgleichungen sind spezielle partielle Differentialgleichungen, mit denen instationäre Vorgänge, d. h. Vorgänge, die nicht nur vom Ort, sondern auch von der Zeit abhängen, dargestellt werden. Diese Zeitabhängigkeit erhöht den Aufwand zur Lösung des Systems eminent, da für jeden einzelnen Zeitpunkt ein stationäres System zu lösen ist.

Oftmals möchte man in die von partiellen Differentialgleichungen modellierten Prozesse eingreifen und von außen Einfluss darauf nehmen. Ein Ziel könnte beispielsweise sein, mit dem System in möglichst kurzer Zeit einen bestimmten Endzustand zu erreichen. Zu diesem Zweck verwendet man Kontrollen oder Steuerungen. Gesucht ist dann eine Steuerfunktion, die den Prozess unter bestimmten Zielvorgaben und Nebenbedingungen optimiert.

Optimalsteuerungsprobleme unter parabolischen Differentialgleichungen sind also mathematische Probleme, bei denen versucht wird, ein Zielfunktional, das von der Steuerung und vom Zustand des Systems abhängt, zu minimieren. Das System ist durch Anfangs- und Randbedingungen sowie Nebenbedingungen, die insbesondere in der vorliegenden Diplomarbeit durch parabolische Differentialgleichungen beschrieben sind, festgelegt. Durch die Steuerung wird auf das System eingewirkt und dieses entsprechend beeinflusst. Sie kann Restriktionen, beispielsweise sogenannten Steuerbeschränkungen, unterliegen.

Ein Beispiel aus der Chemie, bei dem parabolische Differentialgleichungen zum Einsatz kommen, sind chemotaktische Strukturbildungsprozesse (vgl. [14]). Unter Chemotaxis versteht man die Beeinflussung der Fortbewegungsrichtung von Lebewesen oder Zellen durch Stoffkonzentrationsgradienten. Gehen wir davon aus, dass eine bestimmte Substanz (A) mit einer gewissen Konzentration an Zellen vorgegeben ist. Durch eine Steuerung von außen, beispielsweise in Form einer weiteren chemischen Substanz (B), die zu Zellkonzentrationsveränderungen der gegebenen Substanz (A) führt, wird versucht eine gewisse Zielzellverteilung der Substanz (A) zu einem festgelegten Endzeitpunkt zu erreichen. Die „Kosten“ für die Steuerung sollten dabei natürlich möglichst gering sein und die Konzentration der Substanz (A) im Endzustand sollte möglichst wenig von der Zielzellverteilung abweichen. Eine optimale Steuerung ist eine Steuerung, die diese beiden Ziele am besten verbindet. Dabei werden die Zellkonzentrationsänderung und weitere chemische Prozesse durch sehr komplexe parabolische Differentialgleichungen beschrieben.

Auch das alltägliche Problem des Garens einer Kartoffel mit einem Stock über dem Lagerfeuer kann als Optimalsteuerungsproblem mit parabolischen Differentialgleichungen dargestellt werden. Es wird versucht, innerhalb von zehn Minuten eine Kartoffel durch Drehen und Wenden möglichst so im Inneren zu erwärmen, dass sie gleichmäßig gar ist. Der Aufheizprozess der Kartoffel wird durch eine parabolische Differentialgleichung, die Wärmeleitungsgleichung, beschrieben. Das Drehen und Wenden der Kartoffel ist die Steuerung, durch die der Zustand entsprechend verändert wird. Die Temperatur der Kartoffel im Inneren zu Beginn des

Erwärmungsprozesses kann als Anfangsbedingung angesehen werden.

Grundsätzlich gibt es zur Lösung von Optimalsteuerungsproblemen zwei unterschiedliche Herangehensweisen: *direkte* und *indirekte Verfahren*. Bei direkten Methoden („first discretize, then optimize“) wird zuerst das System durch numerische Verfahren diskretisiert und anschließend muss ein meist sehr großes, nichtlineares, aber endlichdimensionales Problem gelöst werden. Hingegen werden bei indirekten Methoden („first optimize, then discretize“) zunächst im Unendlichdimensionalen notwendige Optimalitätsbedingungen und insbesondere die Adjungiertengleichung hergeleitet. Danach werden sowohl System- als auch die Adjungiertengleichungen mittels numerischer Verfahren gelöst.

Während eine indirekte Methode wohl der Art entspricht, wie ein „Mathematiker“ ein Optimalsteuerungsproblem löst, ist eine direkte Variante eher der des „Ingenieurs“ naheliegend. Bei der Optimalsteuerungstheorie handelt es sich um eine „schöne“, geschlossene Theorie, mit der notwendige Optimalitätsbedingungen hergeleitet und anschließend die Probleme gelöst werden können. Obwohl die resultierenden nichtlinearen Probleme bei feiner Diskretisierung oftmals extrem groß werden – was zu langen Laufzeiten der Algorithmen führt – ist die Bedeutung direkter Methoden nicht zu verachten. Bei komplizierten Problemen (vor allem aus der Praxis) ist es extrem schwierig oder gar unmöglich, notwendige Optimalitätsbedingungen im Unendlichdimensionalen aufzustellen. Somit bleibt gar keine andere Möglichkeit, als auf direkte Verfahren zurückzugreifen. Direkte Methoden sind zumeist relativ einfach zu implementieren, und es sind keinerlei Kenntnisse der Optimalsteuerungstheorie von Nöten. Außerdem gibt es auch bei direkten Verfahren Tricks und Ansätze, mit denen die Dimension der nichtlinearen Probleme „relativ klein“ gehalten werden kann.

Im Rahmen dieser Arbeit geht es ausschließlich um ein direktes Verfahren, die Adjungierte oder notwendige Optimalitätsbedingungen im Unendlichdimensionalen werden hier nicht weiter erwähnt. Für genaue Ausführungen zu indirekten Verfahren sei auf das Buch „Optimale Steuerung mit partiellen Differentialgleichungen“ von Tröltzsch [23] verwiesen.

Folgende grundlegende Vorgehensweise wird zur Lösung der Optimalsteuerungsprobleme eingesetzt: Die gegebenen Optimalsteuerungsprobleme und insbesondere die partiellen Differentialgleichungsnebenbedingungen werden zunächst durch Finite Differenzen in der Zeit diskretisiert. Anschließend wird mit Hilfe der Methode der Finiten Elemente das zu jedem Zeitschritt entstandene System in den Ortsvariablen diskretisiert und gelöst. Dabei kommt die von der inuTech GmbH vertriebene C++-Klassenbibliothek DIFFPACK zum Einsatz. Um eine neue Steuerung zu berechnen, bei der die Kosten im Zielfunktional geringer sind als die der vorherigen Steuerung, werden zur Lösung des (nichtlinearen) Systems die beiden Optimierer IPOPT und SCIP verwendet.

Insgesamt gliedert sich die Arbeit in sechs Hauptkapitel, wobei die ersten vier (Kapitel 2-5) die theoretischen Grundlagen für die praktische Anwendung liefern. Das sechste Kapitel stellt die Verbindung zwischen Theorie und Praxis her, indem es beschreibt, wie die Erkenntnisse der ersten Kapitel umgesetzt und implementiert wurden. Im letzten Kapitel sind die numerischen Untersuchungen und Testergebnisse dokumentiert.

Partielle Differentialgleichungen sind das Thema des zweiten Kapitels. Nach einer kurzen Motivation, weshalb partielle Differentialgleichungen von Bedeutung sind, werden sie nach verschiedenen Gesichtspunkten klassifiziert. Außerdem werden Informationen zu Anfangs- und Randbedingungen gegeben und die verschiedenen Steuerungsarten eingeführt.

Eine allgemeine Problemstellung und funktionalanalytische Hintergründe zu den Problemen liefert das dritte Kapitel. Neben den Sobolev-Räumen wird dort auf schwache Formulierungen und schwache Lösungen eingegangen.

Um die unendlichdimensionalen Probleme in endlichdimensionale umzuwandeln, werden Dis-

kretisierungsmethoden eingesetzt. Sie sind Gegenstand des vierten Kapitels. Zur Zeitdiskretisierung werden Finite Differenzen durch die θ -Methode verwendet, die Diskretisierung des Ortes geschieht mittels Finiter Elemente.

Inhalt des fünften Kapitels sind nichtlineare Optimierungsprobleme mit einem Standard-Optimierungsproblem, notwendige und hinreichende Optimalitätsbedingungen sowie die Funktionsweise Innerer-Punkte-Verfahren. Ausführliche Hintergrundinformationen zu den Optimierern IPOPT und SCIPIP schließen den Theorieteil ab.

Das sechste Kapitel zeigt die Implementierung des Verfahrens, und es werden mathematische Methoden wie Finite Differenzen zur Bestimmung der Gradienten und das Newton-Raphson-Verfahren zur Lösung nichtlinearer Gleichungssysteme erläutert. Beide Methoden haben sich als wichtige Hilfsmittel für die Implementierung erwiesen. Außerdem wird das Software-Tool DIFFPACK vorgestellt sowie der grundsätzliche Aufbau eines PDE-Lösers dargelegt. Insbesondere wird auf die Umsetzung der erarbeiteten Methode mittels Programmierung in C++ eingegangen, es wird gezeigt, wie DIFFPACK eingesetzt wird, wie die Optimierer IPOPT bzw. SCIPIP aufgerufen werden und wie alle Komponenten zusammenhängen.

Numerische Untersuchungen und Resultate, die anhand von fünf Testbeispielen erzielt wurden, sind Inhalt des siebten und letzten Kapitels. Das Verfahren wurde an Beispielen untersucht, bei denen entweder eine analytische Lösung bekannt ist oder Vergleichsergebnisse von anderen Arbeiten vorliegen. Dabei wurden die beiden Optimierer SCIPIP und IPOPT bezüglich Konvergenzverhalten und Geschwindigkeit für diese spezielle Problemklasse unter Verwendung des dargestellten Verfahrens verglichen.

2 Partielle Differentialgleichungen

Das Hauptanliegen dieser Arbeit besteht darin, ein direktes Verfahren für Optimalsteuerungsprobleme mit *parabolischen* Differentialgleichungen zu beschreiben. Jedoch spielen bei der verwendeten Methode auch *elliptische* Differentialgleichungen eine wichtige Rolle. Um einen Überblick über die verschiedenen Arten von partiellen Differentialgleichungen, im Folgenden stets mit PDE („partial differential equation“) abgekürzt, zu geben – wie man sie voneinander abgrenzt, welche Arten von Anfangs- und Randbedingungen es gibt und welche Kontrollmöglichkeiten mit Hilfe von Steuerungen existieren – wurde dieses Kapitel der Arbeit vorangestellt. Es beinhaltet die Grundbegriffe im Zusammenhang mit PDEs, die in den nächsten Kapiteln bei Optimalsteuerungsproblemen in den Nebenbedingungen auftreten. Der Inhalt dieses Kapitels orientiert sich an [8, 19, 25].

2.1 Motivation

Zunächst stellt sich die Frage, was man unter einer partiellen Differentialgleichung versteht: Eine PDE ist eine Gleichung (oder ein Gleichungssystem) für eine Funktion $y : \Omega \rightarrow \mathbb{R}$, die (partielle) Ableitungen von y enthält, wobei das Gebiet $\Omega \subseteq \mathbb{R}^n$ offen ist (es kann ein Gebiet oder eine niederdimensionale Mannigfaltigkeit sein).

Definition 2.1.1 (Partielle Differentialgleichung)

Es sei L ein Differentialoperator, der einer Funktion $y : \mathbb{R}^n \rightarrow \mathbb{R}$ eine Funktion $L[y] : \mathbb{R}^n \rightarrow \mathbb{R}$ zuordnet gemäß

$$L[y] = F\left(x_1, \dots, x_n, y, y_{x_1}, \dots, y_{x_n}, \dots, \frac{\partial^m}{\partial x_1^{s_1} \dots \partial x_n^{s_n}} y\right),$$

wobei in F außer der Funktion y auch partielle Ableitungen von y auftreten. Dann heißt

$$L[y] = f(x), \quad x \in \mathbb{R}^n$$

partielle Differentialgleichung in n Variablen ($n \geq 2$).

Man sagt, die PDE hat die Ordnung m , wenn $s_1 + \dots + s_n = m$ die höchste auftretende Ableitungsordnung ist. In dieser Arbeit geht es ausschließlich um PDEs zweiter Ordnung.

Für die untersuchten parabolischen Differentialgleichungen spielt die Zeitkomponente eine wichtige Rolle. Lösungen partieller Differentialgleichungen sind im Gegensatz zu gewöhnlichen Differentialgleichungen, bei denen die gesuchte Lösung nur von einer Variablen abhängt, je nach Typ der PDE

- von mehreren Raumdimensionen,
- von einer Raumdimension und der Zeit oder
- von mehreren Raumdimensionen und der Zeit

abhängig.

Durch PDEs lassen sich deshalb sehr viel komplexere Vorgänge beschreiben, als es bei der Verwendung gewöhnlicher Differentialgleichungen der Fall ist. Das erklärt die Vielfalt an verschiedenen Bereichen, in denen PDEs heutzutage zum Einsatz kommen. Nicht nur in den Ingenieur-, Natur- und Wirtschaftswissenschaften, sondern auch in der Medizin und den Finanzwissenschaften stellen PDEs ein unverzichtbares Mittel dar. Hier seien exemplarisch Gebiete, in denen sie zur Anwendung kommen, aufgezählt:

- Aeronautik: Luft- und Raumfahrt
- Meteorologie: Wettervorhersage
- Wirtschaft: Portfoliomodelle
- Biowissenschaften: Chemotherapie

Um den Einstieg abzurunden, werden ein paar sehr wichtige Beispiele präsentiert und ihre Bedeutung aufgezeigt:

- **Laplace- und Poisson-Gleichung:**

Die Laplace- und die Poisson-Gleichung lauten

$$\Delta y = 0 \quad \text{bzw.} \quad \Delta y = f$$

mit einer gegebenen Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Die Poisson-Gleichung beschreibt physikalisch die Auslenkung einer eingespannten Membran unter der äußeren Kraft-Einwirkung f , die Laplace-Gleichung erfasst einen Gleichgewichtszustand.

- **Wärmeleitungsgleichung:**

Wärme- und Ausbreitungsprozesse werden oftmals durch die Wärmeleitungsgleichung

$$y_t = \Delta y$$

modelliert.

- **Wellengleichung:**

Die Wellengleichung lautet

$$y_{tt} = \Delta y$$

und stellt Wellen- und Schwingungsphänomene dar.

Der Operator Δ bezeichnet für alle vier Gleichungen den *Laplace-Operator*

$$\Delta y = \sum_{i=1}^n \frac{\partial^2 y}{\partial x_i^2}.$$

Er ist wesentlicher Bestandteil vieler PDEs und beschreibt physikalisch einen Diffusions-Vorgang. Zu beachten ist außerdem, dass die Laplace- und Poisson-Gleichung lediglich ortsabhängig sind, bei der Wärmeleitungs- und Wellengleichung kommt eine Zeitabhängigkeit hinzu.

2.2 Klassifikation partieller Differentialgleichungen

PDEs haben mitunter sehr verschiedene Eigenschaften. Eine universelle Theorie für alle PDEs gibt es nicht, da die unterschiedlichen Charakteristika von PDEs zu differierenden mathematischen Eigenschaften führen, die für die jeweiligen (numerischen) Lösungsverfahren ausgenutzt werden. Zur Klassifizierung von PDEs gibt es mehrere mögliche Kriterien. Es werden die folgenden Kategorien unterschieden:

1. Typ der Gleichung (d.h. deren algebraische Eigenschaften)
2. Ordnung (höchste auftretende Ableitung)
3. Typeneinteilung der PDE (für PDEs zweiter Ordnung)
4. Lösbarkeit

Algebraische Eigenschaften

Hinsichtlich ihrer algebraischen Eigenschaften grenzt man bei PDEs lineare und nichtlineare Gleichungen voneinander ab, wobei es bei nichtlinearen Gleichungen zwei Spezialfälle gibt, nämlich semilineare und quasilineare Gleichungen.

- **Lineare Gleichungen:**

Die unbekannte Funktion y sowie alle auftretenden Ableitungen kommen nur linear vor.

- **Semilineare Gleichungen:**

Alle Ableitungen höchster Ordnung treten linear auf, die Funktion selbst und Ableitungen niedrigerer Ordnung jedoch nicht.

- **Quasilineare Gleichungen:**

Die Koeffizientenfunktionen vor der höchsten Ableitung hängen von niedrigeren Ableitungen und der unbekannten Funktion ab.

- **Nichtlineare Gleichungen:**

In allen anderen Fällen heißt die Gleichung nichtlinear.

Der Ausdruck, eine Funktion oder auftretende Ableitungen kommen *linear* vor, bedeutet in diesem Zusammenhang, dass die Koeffizientenfunktionen vor der unbekannten Funktion bzw. den Ableitungen nur von den Veränderlichen abhängen (und nicht von der Funktion selbst).

Ordnung einer PDE

Die Ordnung einer PDE wurde bereits definiert. Entsprechend ihrer Ordnung lassen sich PDEs klassifizieren. Dabei kann es vorkommen, dass man zwischen Orts- und Zeitableitung unterscheidet. Die einführenden Beispiele der Laplace-, Poisson-, Wärmeleitungs- und Wellengleichung sind jeweils PDEs zweiter Ordnung.

Typeinteilung von PDEs zweiter Ordnung

Für PDEs zweiter Ordnung gibt es eine Typeinteilung, die wesentliche Aussagen über die mathematischen Eigenschaften der PDEs entsprechenden Typs erlaubt. Entscheidend für die Zuordnung einer PDE zu einer der verschiedenen Arten sind die Koeffizienten vor den zweiten Ableitungen. Wir betrachten eine PDE zweiter Ordnung mit Lösung y , die in den zwei Variablen x_1 und x_2 mindestens zweimal stetig differenzierbar ist:

$$Ay_{x_1x_1} + 2By_{x_1x_2} + Cy_{x_2x_2} = F, \quad (2.1)$$

wobei A , B , C und F Funktionen in Abhängigkeit von x_1 , x_2 , y , y_{x_1} und y_{x_2} darstellen.

Definition 2.2.1 (Diskriminante einer PDE)

Die Funktion

$$d := AC - B^2$$

bezeichnet man als die Diskriminante der PDE.

Eine PDE ist genau dann

- **elliptisch**, wenn $d > 0$,
- **parabolisch**, wenn $d = 0$,
- **hyperbolisch**, wenn $d < 0$

auf dem Gebiet Ω ist.

Die Poisson- und Laplace-Gleichung stellen Beispiele elliptischer PDEs dar, die Wärmeleitungsgleichung ist eine parabolische PDE, bei der Wellengleichung handelt es sich um eine hyperbolische PDE.

Lösbarkeit

Eine weitere Möglichkeit PDEs zu gliedern ist es, sie hinsichtlich ihrer Lösbarkeit zu trennen. Dabei interessieren vor allem Fragen nach

- Existenz von Lösungen,
- Eindeutigkeit der Lösung,
- Stabilität, d.h. stetige Abhängigkeit von den Daten und
- welche Rand- und Anfangsbedingungen gestellt werden können.

In diesem Zusammenhang ist die Frage wichtig, ob ein Problem *sachgemäß gestellt* ist. In der Regel werden Differentialgleichungen auf Gebieten $\Omega \subset \mathbb{R}^n$ betrachtet. Zusätzlich treten häufig Bedingungen am Rand $\partial\Omega$ auf. Jedoch kann nicht jede beliebige Randbedingung gewählt werden, sie muss mit den Eigenschaften des Differentialoperators „zusammenpassen“. Bei Anfangs- und Randwertaufgaben gewöhnlicher Differentialgleichungen ist die Regularität der Lösung kein besonderes Thema, da sich die Regularität der Daten direkt auf die der Lösung überträgt. Bei PDEs ist das jedoch nicht immer der Fall, und es kann vorkommen, dass sie aufgrund ungeeigneter Nebenbedingungen (beispielsweise unpassender Randbedingungen) nicht nachfolgender Definition entsprechen:

Definition 2.2.2 (Sachgemäß gestellte PDE)

Eine PDE

$$L[y] = f$$

mit Nebenbedingungen heißt sachgemäß gestellt, wenn

- es eine Lösung y gibt,
- die Lösung eindeutig ist und
- die Lösung stetig von den Nebenbedingungen abhängt.

Ist eine der drei Bedingungen nicht erfüllt, so spricht man von einem schlecht gestellten Problem.

2.3 Anfangs- und Randbedingungen

Anfangs- und Randbedingungen beeinflussen nicht nur, ob ein Problem sachgemäß gestellt ist, sie legen auch die gesuchte Lösung einer PDE fest. Nicht für jede PDE kann jede beliebige Bedingung gewählt werden, sondern die Bedingung ist abhängig vom Typ der PDE. Im Folgenden werden Anfangsbedingungen und die wichtigsten Randbedingungen kurz aufgeführt.

2.3.1 Anfangsbedingungen

Viele PDEs besitzen bei Verzicht auf jegliche Nebenbedingungen unendlich viele Lösungen. Um die Auswahl an Lösungen einzugrenzen oder möglicherweise eine leere Lösungsmenge zu finden, führt man Anfangsbedingungen ein. Eine Anfangsbedingung sagt aus, welchen Funktionswert die gesuchte Lösung sowie gegebenenfalls ihre Ableitung(en) an bestimmten Stellen haben sollen. Wird durch die Differentialgleichung eine zeitliche Entwicklung beschrieben, beispielsweise die Bewegung eines Gegenstands im Raum, so determiniert die Anfangsbedingung in welchem Zustand die Bewegung beginnt. Wenn eine Bedingung den Zustand eines Systems am Ende oder zu einem anderen Zeitpunkt festlegt, so spricht man im mathematischen Sinne auch von einer Anfangsbedingung, obwohl der Begriff „Endbedingung“ oder „Zwischenzustand“ möglicherweise treffender wäre.

2.3.2 Randbedingungen

Randbedingungen sind Vorgaben auf dem Rand des Gebiets $\partial\Omega$, die die Berechnung der Lösung einer PDE wesentlich beeinflussen. Die bekanntesten Randbedingungen sind

- Dirichlet-Randbedingungen,
- Neumann-Randbedingungen und
- Robin-Randbedingungen (bzw. Randbedingungen dritter Art).

Die Funktion g wird im Folgenden als glatt angenommen:

Dirichlet-Randbedingungen

Dirichletsche Randbedingungen werden dargestellt durch

$$y = g \quad \text{auf} \quad \partial\Omega.$$

Sie setzen auf dem Rand des Gebiets Werte zur Berechnung der Lösungsfunktion fest. Dirichlet-Randbedingungen werden häufig auch *wesentliche Randbedingungen* genannt. Treten bei einer Problemstellung ausschließlich Dirichlet-Randbedingungen auf, so spricht man von einer *Dirichlet-Randwertaufgabe*.

Neumann-Randbedingungen

Eine Randbedingung der Form

$$\partial_\nu y = g \quad \text{auf} \quad \partial\Omega$$

heißt *Neumann-Randbedingung*. ∂_ν ist die Ableitung von y in Richtung der äußeren Normalen ν auf den Rand. Durch eine Neumann-Randbedingung werden folglich Werte auf dem Rand des Gebiets für die Normalenableitung der Lösung festgelegt. Auch hier gilt wieder, dass eine Problemstellung, bei der nur Neumann-Randbedingungen vorkommen, *Neumann-Randwertaufgabe* genannt wird.

Robin-Randbedingungen

Eine *Robin-Randbedingung* ist durch

$$\partial_\nu y + \alpha y = g \quad \text{auf} \quad \partial\Omega$$

gegeben. Der Koeffizient α wird als positiv angenommen.¹ Robin-Randbedingungen werden mitunter auch als Cauchy-Randbedingungen bezeichnet und stellen eine Kombination von Dirichlet- und Neumann-Randbedingung dar.

Neumann- und Robin-Randbedingungen fasst man unter dem Begriff *natürliche Randbedingungen* zusammen. Neben wesentlichen und natürlichen Randbedingungen werden in dieser Arbeit bei bestimmten Testbeispielen nichtlineare Randbedingungen vorkommen, sogenannte *Stefan-Boltzmann-Randbedingungen*.

2.4 Steuerungsarten

Zum Abschluss dieses Kapitels wird kurz aufgezeigt, welche Möglichkeiten bestehen, um auf gegebene Systeme und die im Anschluss behandelten Optimalsteuerungsprobleme Einfluss zu nehmen.

Im Normalfall wird der Zustand des Systems mit Hilfe einer PDE auf dem gesamten Gebiet Ω beschrieben. Zusätzlich sind spezielle Bedingungen am Rand vorgegeben. Durch die Steuerung kann auf das System eingewirkt werden. Man unterscheidet dabei zwei Arten von Steuerungen, nämlich *verteilte Steuerungen* und *Randsteuerungen*.

2.4.1 Verteilte Steuerungen

Ein Beispiel einer verteilten Steuerung ist das Aufheizen eines Körpers Ω durch elektromagnetische Induktion oder Mikrowellen. Die Steuerung wirkt bei verteilten Steuerungen auf das gesamte Gebiet Ω . Dabei wird das Ziel, eine vorgegebene stationäre Temperaturverteilung y_Ω zu erreichen, verfolgt.

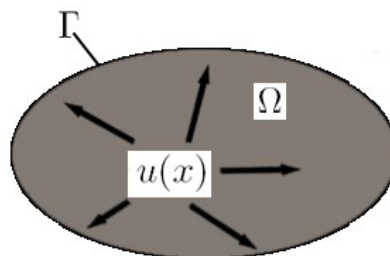


Abbildung 2.1: Verteilte Steuerung

¹Falls $\alpha = 0$ gilt, handelt es sich um eine Neumann-Randbedingung.

2.4.2 Randsteuerungen

Den Gegenpart zu verteilten Steuerungen bilden die sogenannten Randsteuerungen, bei denen nicht über das gesamte Gebiet Ω , sondern lediglich über den Rand $\partial\Omega$ Einfluss auf das System genommen wird. Im Fall von Wärmeleitprozessen geschieht das beispielsweise, indem an den Rand eine zeitlich konstante Temperatur — unsere Steuerung — gelegt wird. Das Ziel ist es ebenfalls, das System so über den Rand zu regulieren, dass die vorgegebene Temperaturverteilung y_Ω erreicht wird.

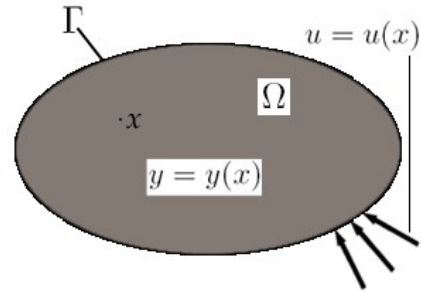


Abbildung 2.2: Randsteuerung

3 Problemstellung und funktionalanalytische Grundlagen

Das nachfolgende Kapitel zeigt eine allgemeine Problemstellung und beschreibt die funktionalanalytischen Grundlagen zur Optimalsteuerung mit partiellen Differentialgleichungen. Es orientiert sich weitestgehend an Tröltzschs Buch „Optimale Steuerung mit partiellen Differentialgleichungen“ [23], der Dissertation von Theißen [24] und der Diplomarbeit von Tomety [22]. Zwei zentrale Begriffe im Zusammenhang mit partiellen Differentialgleichungen sind der sogenannte *Sobolev-Raum* und die *schwache Formulierung*, die in diesem Kapitel definiert werden.

3.1 Problemstellung

Da in dieser Arbeit Optimalsteuerungsprobleme mit parabolischen PDEs untersucht werden, hängen der *Zustand* y und die *Steuerung* u , falls nichts anderes erwähnt wird, stets von Ort und Zeit ab. Bei den Steuerungen handelt es sich ausschließlich um Randsteuerungen. Außerdem sind Anfangs- und Randbedingungen gegeben. Welche Art von Randbedingung vorliegt, wird für das jeweilige Beispiel spezifiziert.

Um bei den Testbeispielen in Kapitel 7 nicht in jedem einzelnen Fall das Vorgehen zur Berechnung der Lösung, das prinzipiell immer analog verläuft, schildern zu müssen, wird ein vereinfachtes exemplarisches Modellproblem¹, eine *Anfangs-Randwert-Aufgabe*, eingeführt. Es lautet:

Minimiere

$$J(y, u) := \frac{1}{2} \|y(x, T) - y_\Omega(x)\|_{L^2(\Omega)}^2 + \frac{\lambda}{2} \|u(x, t)\|_{L^2(\Sigma)}^2$$

unter

$$\begin{aligned} y_t(x, t) - \Delta_x y(x, t) &= f(x, t) && \text{in } Q, \\ y(x, 0) &= y_0(x) && \text{in } \Omega, \\ \partial_\nu y(x, t) + \vartheta(y(x, t)) + g(x, t) &= u(x, t) && \text{auf } \Sigma, \\ u_a &\leq u(x, t) \leq u_b && \text{auf } \Sigma. \end{aligned} \tag{3.1}$$

Dabei ist $\Omega \subset \mathbb{R}^n$ ein beschränktes Lipschitzgebiet (Definition siehe (3.2.1)) mit Rand $\Gamma = \partial\Omega$, $Q = \Omega \times (0, T)$ ein Raum-Zeit-Zylinder mit festem Endzeitpunkt $T > 0$ und Rand $\Sigma = \Gamma \times (0, t)$. y_Ω steht für die zum Endzeitpunkt T auf Ω anzusteuernde Wunschfunktion, y_0 für den vorgegebenen Anfangszustand auf Ω , $\partial_\nu y$ für die Normalenableitung von y auf dem Rand von Ω , λ für einen sogenannten Regularisierungsparameter und u_a, u_b sind untere und obere Steuerbeschränkungen. f, g und ϑ stellen (zum Teil nichtlineare) Funktionen in Abhängigkeit der angegebenen Parameter dar, der Koeffizient α ist eine reelle Konstante.

Die Testbeispiele in Kapitel 7 unterscheiden sich teilweise im Zielfunktional von unserem Modellproblem, oder es existieren mitunter zwei verschiedene Randbedingungen. Um jedoch all-

¹Das Modellproblem wurde bereits so umgeformt, dass es am passendsten für die späteren Testbeispiele ist. Fasst man $\vartheta(y(x, t)) + g(x, t)$ zu $b(x, t, y)$ zusammen, so gehört es exakt dem Typ von Aufgabe an, der in [23, S. 205ff] mit den Bedingungen für Existenz und Eindeutigkeit beschrieben ist.

gemeine Aussagen über die Probleme bezüglich der Herleitung der schwachen Formulierung sowie der anschließenden Implementierung treffen zu können und um dabei die Darstellung übersichtlich zu halten, wird diese vereinfachte Problemstellung als Grundlage genommen.

3.2 Sobolev-Räume

Obwohl aufgrund der verwendeten Diskretisierungsmethode (siehe Kapitel 4) Sobolev-Räume und schwache Formulierungen nur für elliptische Differentialgleichungen benötigt werden, wird in diesem Kapitel die Sobolev-Theorie für elliptische und parabolische PDEs in Auszügen dargestellt. Da parabolische PDEs wesentlicher Bestandteil des Themas der Diplomarbeit sind, wird auf die Darstellung der Lösungsräume dieser Differentialgleichungen nicht verzichtet. Zudem tragen die funktionalanalytischen Hintergründe zu parabolischen PDEs zu einem besseren Verständnis der Thematik bei.

Die Motivation zur Behandlung von Sobolev-Räumen und der schwachen Lösung ist folgende: *Klassische Lösungen*, also glatte Lösungen, die die PDE punktweise erfüllen, erweisen sich hinsichtlich partieller Differentialgleichungen oftmals als zu einschränkend. Durch eine neue Definition der Lösung der PDE mit Hilfe der schwachen Formulierung wird der Raum der zulässigen Lösungen vergrößert. Im Zusammenhang mit der Erweiterung des Lösungsbegriffs werden als neue Funktionenräume die Sobolev-Räume eingeführt.

Zunächst wird das Rüstzeug in Form von Sätzen und Definitionen bereitgestellt, bevor die konkreten Lösungsräume für elliptische und parabolische PDEs behandelt werden:

Generell bezeichnet $\Gamma := \partial\Omega$ den Rand von Ω und $\overline{M}$ den Abschluss einer Menge M . Letzterer wird zur Definition eines *Lipschitzgebiets* benötigt.

Bei der Theorie von PDEs wird von Ortsgebieten $\Omega \subset \mathbb{R}^n$ mit hinreichend glattem Rand ausgegangen.

Definition 3.2.1 (Lipschitzgebiet)

Sei $\Omega \subset \mathbb{R}^n$, $n \geq 2$, ein beschränktes Gebiet mit Rand Γ . Ω bzw. sein Rand Γ gehören zur Klasse $C^{k,1}$, $k \in \mathbb{N}_0$, wenn endlich viele Koordinatensysteme $S_1, \dots, S_M$, Funktionen $h_1, \dots, h_M$ sowie Konstanten $a > 0$, $b > 0$ mit folgenden Eigenschaften existieren:

1. Alle h_i sind auf dem $(n-1)$ -dimensionalen, abgeschlossenen Würfel

$$\overline{Q}_{n-1} = \{x = (x_1, \dots, x_{n-1}) : |x_i| \leq a, i = 1, \dots, n-1\}$$

k -mal stetig differenzierbar mit Lipschitz-stetigen Ableitungen der Ordnung k .

2. Zu jedem $P \in \Gamma$ gibt es ein $i \in \{1, \dots, M\}$, so dass P im Koordinatensystem S_i die Darstellung $P = (x, h_i(x))$, $x \in Q_{n-1}$, besitzt.

3. Im lokalen Koordinatensystem S_i gilt

$$\begin{aligned} (x, x_n) \in \Omega &\Leftrightarrow x \in \overline{Q}_{n-1}, h_i(x) < x_n < h_i(x) + b, \\ (x, x_n) \notin \Omega &\Leftrightarrow x \in \overline{Q}_{n-1}, h_i(x) - b < x_n < h_i(x). \end{aligned}$$

Gebiete bzw. Ränder der Klasse $C^{0,1}$ heißen **Lipschitzgebiete** (auch reguläre Gebiete) bzw. **Lipschitzränder**.

Es werden für $n \geq 2$ Gebiete mit Lipschitzrand betrachtet. Da der Rand eines eindimensionalen Ortsgebiets lediglich aus zwei Punkten besteht, ist für diesen Fall keine weitere Klassifizierung nötig.

Der Vektor natürlicher Zahlen $\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{N}^n$ ist ein sogenannter *Multiindex*. Die Komponente α_i gibt an, wie oft die Funktion $y : \Omega \rightarrow \mathbb{R}$ nach der Ortskomponente x_i von Ω differenziert wird. Dabei wird die abkürzende Schreibweise $D^\alpha y(x)$ verwendet:

$$D^\alpha y(x) := D_1^{\alpha_1} \dots D_n^{\alpha_n} y(x) := \frac{\partial^{|\alpha|} y}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}}.$$

$|\alpha| = \alpha_1 + \dots + \alpha_n$ definiert die Ordnung von α .

Definition 3.2.2 (Träger)

Die Menge $\text{supp } v = \{x \in \Omega : v(x) \neq 0\}$ heißt **Träger (support)** von v . Es handelt sich dabei um die kleinste abgeschlossene Menge außerhalb der v identisch verschwindet.

3.2.1 Lösungsräume für elliptische Differentialgleichungen

Einer der grundlegendsten Räume bezüglich der Optimalsteuerungstheorie elliptischer Differentialgleichungen ist der Raum der messbaren Funktionen:

Definition 3.2.3 ($\mathbb{L}^p, \mathbb{L}^\infty$)

Unter $\mathbb{L}^p(\Omega)$, $1 \leq p < \infty$, versteht man den Raum der messbaren Funktionen $y : \Omega \rightarrow \mathbb{R}$ mit der Eigenschaft

$$\int_{\Omega} |y(x)|^p dx < \infty.$$

Funktionen, die sich nur auf einer Menge vom Maß Null unterscheiden, werden als gleich angesehen und gehören einer gemeinsamen Äquivalenzklasse an.

Mit $\mathbb{L}^\infty(\Omega)$ wird der Raum aller fast überall gleichmäßig beschränkten und messbaren Funktionen $y : \Omega \rightarrow \mathbb{R}$ bezeichnet.

Versehen mit folgenden Normen

$$\|y\|_{\mathbb{L}^p(\Omega)} = \left(\int_{\Omega} |y(x)|^p dx \right)^{\frac{1}{p}} \quad \text{für } 1 \leq p < \infty,$$

$$\|y\|_{\mathbb{L}^\infty(\Omega)} = \text{ess sup}_{x \in \Omega} |y(x)|,$$

stellen die Räume $\mathbb{L}^p(\Omega)$, $1 \leq p \leq \infty$, Banachräume dar. Dabei bezeichnet ess sup das sogenannte *essentielle* oder *wirkliche* Maximum bzw. Supremum, d. h. das Supremum der Funktion ohne Berücksichtigung von Mengen mit Maß Null.

Definition 3.2.4 ($C(\overline{\Omega})$)

Mit $C(\overline{\Omega})$ wird der Raum aller auf dem Abschluss $\overline{\Omega}$ stetigen reellwertigen Funktionen bezeichnet, mit $C^k(\overline{\Omega})$ der Raum aller auf $\overline{\Omega}$ k -mal stetig partiell differenzierbaren reellwertigen Funktionen.

Versehen mit den Normen

$$\|y\|_{C(\overline{\Omega})} = \max_{x \in \overline{\Omega}} |y(x)|, \quad \|y\|_{C^k(\overline{\Omega})} = \max_{x \in \overline{\Omega}} \sum_{|\alpha| \leq k} |D^\alpha y(x)|$$

sind sie Banachräume.

Um schwache Ableitungen korrekt definieren zu können, benötigt man sogenannte *Testfunktionen*. Diese Funktionen liefern bei der partiellen Integration verschwindende Randintegrale, da sie auf dem Rand von Ω den Wert Null besitzen. Sie stammen aus dem Raum der beliebig oft stetig differenzierbaren Funktionen mit kompaktem Träger in Ω :

Definition 3.2.5 ($C_0^k(\Omega)$, $C_0^\infty(\Omega)$)

Mit $C_0^k(\Omega)$, $0 \leq k \leq \infty$, wird die Menge aller k -mal stetig differenzierbaren Funktionen $v : \Omega \rightarrow \mathbb{R}$ mit kompaktem Träger in Ω bezeichnet:

$$C_0^k(\Omega) := \left\{ v \in C^k(\Omega) \mid \text{supp}(v) \subset \Omega \right\}.$$

Die Menge aller beliebig oft stetig differenzierbarer Funktionen $v : \Omega \rightarrow \mathbb{R}$ mit kompaktem Träger in Ω heißt C_0^∞ :

$$C_0^\infty(\Omega) := \{ v \in C^\infty(\Omega) \mid \text{supp}(v) \subset \Omega \}.$$

Vor der Verallgemeinerung des klassischen Ableitungsbegriffs wird die Menge der in Ω lokal integrierbaren Funktionen eingeführt:

Definition 3.2.6 ($\mathbb{L}_{loc}^1(\Omega)$)

Die Menge aller in Ω lokal integrierbaren Funktionen bezeichnet man mit $\mathbb{L}_{loc}^1(\Omega)$. Eine Funktion heißt lokal integrierbar in Ω , wenn sie auf jeder kompakten Teilmenge von Ω im Lebesgueschen Sinne integrierbar ist.

Somit ist die Vorarbeit geleistet, und der Begriff der schwachen Ableitung kann nun eingeführt werden:

Definition 3.2.7 (Schwache Ableitung)

Es seien $y \in \mathbb{L}_{loc}^1(\Omega)$ und ein Multiindex α gegeben. Existiert eine Funktion $w \in \mathbb{L}_{loc}^1(\Omega)$, so dass

$$\int_{\Omega} y(x) D^\alpha v(x) dx = (-1)^{|\alpha|} \int_{\Omega} w(x) v(x) dx$$

für alle $v \in C_0^\infty(\Omega)$ erfüllt ist, so heißt w die **schwache Ableitung der Ordnung α von y** , die ebenfalls mit $w = D^\alpha y$ bezeichnet wird.

Erfüllt w also an Stelle der (starken) Ableitung $D^\alpha y$ die Formel der partiellen Integration, so ist w schwache Ableitung der Ordnung α von y .

Der Raum $\mathbb{L}_{loc}^1$ ist aus mathematischer Sicht etwas unhandlich. Deshalb interessiert man sich besonders für schwache Ableitungen, die „schöneren“ Räumen, beispielsweise den $\mathbb{L}^p$ -Räumen, angehören.

Definition 3.2.8 (Sobolev-Räume)

Es sei $1 \leq p < \infty$, $k \in \mathbb{N}$. Unter $W^{k,p}(\Omega)$ versteht man den Raum aller $y \in \mathbb{L}^p(\Omega)$, für welche die schwachen Ableitungen $D^\alpha y$ für alle Multiindizes α mit $|\alpha| \leq k$ existieren und ebenfalls zu $\mathbb{L}^p(\Omega)$ gehören. Versehen mit der Norm

$$\|y\|_{W^{k,p}(\Omega)} = \left(\sum_{|\alpha| \leq k} \int_{\Omega} |D^\alpha y|^p dx \right)^{\frac{1}{p}}$$

sind die Räume $\mathbb{W}^{k,p}(\Omega)$ Banachräume und werden **Sobolev-Räume** genannt. Analog wird $\mathbb{W}^{k,\infty}(\Omega)$ für $p = \infty$ mit der Norm

$$\|y\|_{\mathbb{W}^{k,\infty}(\Omega)} = \max_{|\alpha| \leq k} \|D^\alpha y\|_{\mathbb{L}^\infty(\Omega)}$$

eingeführt.

Der Fall $p = 2$ ist von besonderem Interesse, weshalb man den zugehörigen Sobolev-Raum mit

$$\mathbb{H}^k(\Omega) := \mathbb{W}^{k,2}(\Omega)$$

abkürzt. Da der Raum $\mathbb{H}^1(\Omega)$ in der Literatur besonders häufig vorkommt, wird seine Definition hier noch einmal explizit aufgeschrieben:

$$\mathbb{H}^1(\Omega) = \{y \in \mathbb{L}^2(\Omega) : D_i y \in \mathbb{L}^2(\Omega), i = 1, \dots, n\}.$$

Versehen mit der Norm

$$\|y\|_{\mathbb{H}^1(\Omega)} = \left(\int_{\Omega} (y^2 + |\nabla y|^2) dx \right)^{\frac{1}{2}}$$

mit $|\nabla y|^2 := (D_1 y)^2 + \dots + (D_n y)^2$ und durch Einführung des Skalarprodukts

$$(u, v)_{\mathbb{H}^1(\Omega)} = \int_{\Omega} u v dx + \int_{\Omega} \nabla u \cdot \nabla v dx$$

wird $\mathbb{H}^1(\Omega)$ zu einem Hilbertraum.

Der Rand eines Gebiets ist üblicherweise eine Nullmenge und Funktionen, die bis auf Nullmengen übereinstimmen, werden in den obigen Funktionenräumen als gleich angesehen. Um Randwerte für Funktionen aus Sobolev-Räumen definieren zu können, benötigt man Funktionenräume auf Ω , bei denen Funktionen mit unterschiedlichen Werten auf dem Rand Γ nicht als gleich angesehen werden (wie es in $L^p(\Omega)$ der Fall ist). Deshalb definiert man eine Vervollständigung der Sobolev-Räume:

Definition 3.2.9 (Vervollständigung der Sobolev-Räume)

Die Vervollständigung von $C_0^\infty(\Omega)$ in $\mathbb{W}^{k,p}(\Omega)$ bezeichnet man mit $\mathbb{W}_0^{k,p}(\Omega)$. Dieser Raum wird mit der gleichen Norm wie $\mathbb{W}^{k,p}(\Omega)$ versehen und ist ein abgeschlossener Teilraum von $\mathbb{W}^{k,p}(\Omega)$. Insbesondere setzt man $\mathbb{H}_0^k(\Omega) := \mathbb{W}_0^{k,2}(\Omega)$.

Der Raum $\mathbb{W}_0^{k,p}(\Omega)$ umfasst folglich Funktionen, bei denen die Randwerte aller Ableitungen bis zur Ordnung $k-1$ verschwinden. Inhomogene Randwerte definiert man durch die *Spur* von Funktionen aus $\mathbb{W}^{k,p}(\Omega)$ auf dem Rand:

Satz 3.2.1 (Spursatz)

Ist Ω ein beschränktes Lipschitzgebiet, so existiert eine lineare und stetige Abbildung $\tau : \mathbb{W}^{1,p}(\Omega) \rightarrow L^p(\Gamma)$, so dass für alle $y \in C(\overline{\Omega})$ gilt:

$$(\tau y)(x) = y(x) \quad \text{f.ü. auf } \Gamma.$$

Definition 3.2.10 (Spur, Spuroperator)

Das Element τy heißt *Spur* von y auf Γ , die Abbildung τ *Spuroperator*.

3.2.2 Lösungsräume für parabolische Differentialgleichungen

Die bislang betrachteten Funktionen „leben“ nur auf dem Ortsgebiet $\Omega \subset \mathbb{R}^n$. Bei den in dieser Arbeit untersuchten Problemen handelt es sich aber um instationäre Vorgänge, d.h. neben der Ortsabhängigkeit kommt eine Zeitkomponente hinzu. Die zu behandelnden Funktionen sind also auf dem Raum-Zeit-Zylinder $Q := \Omega \times (0, T)$ definiert. Die Dynamik der Systeme wird auf dem Ortsgebiet Ω vom Anfangszeitpunkt $t_0 = 0$ bis zur Endzeit $T > 0$ analysiert.

In den nachfolgenden Abschnitten besitze das beschränkte Gebiet Ω stets einen $C^{1,1}$ -Rand Γ . Der Raum der auf Q quadratisch Lebesgue-integrierbaren Funktionen

$$\mathbb{L}^2(Q) := \left\{ y : Q \rightarrow \mathbb{R} \mid \|y\|_{\mathbb{L}^2(Q)}^2 < \infty \right\}$$

ist mit der zugehörigen Norm

$$\|y\|_{\mathbb{L}^2(Q)} := \left(\int_0^T \int_{\Omega} y^2(x, t) dx dt \right)^{\frac{1}{2}}$$

ein Banachraum.

Für Optimalsteuerungsprobleme mit parabolischen Differentialgleichungen, bei denen die gegebenen Steuerungen aus $\mathbb{L}^2$ -Räumen gewählt werden, kann die Forderung nach einer klassischen Lösung, bei der alle auftretenden Ableitungen existieren und im Inneren des Raum-Zeit-Zylinders Q stetig sein müssen (d. h. $y \in C^{2,1}(Q)$), zu stark sein. Deshalb definiert man analog zu den schwachen Ableitungen in Ω den Begriff der schwachen Ableitung für Funktionen aus Q :

Definition 3.2.11 (Schwache Ortsableitungen in Q)

Es seien $y \in \mathbb{L}^2(Q)$ und ein Multiindex $\alpha \in \mathbb{N}_0^n$ gegeben. $w \in \mathbb{L}_{loc}^1(Q)$ heißt **schwache Ableitung der Ordnung $|\alpha|$ von y** , wenn für alle $v \in C_0^\infty(Q)$ gilt:

$$\iint_Q y(x, t) D^\alpha v(x, t) dx dt = (-1)^{|\alpha|} \iint_Q w(x, t) v(x, t) dx dt.$$

Die schwache Ableitung der Ordnung $|\alpha|$ von y wird mit $w := D^\alpha y$ bezeichnet.

Somit erhält man für Q ein Analogon zum Raum $\mathbb{H}^1(\Omega)$:

Definition 3.2.12 ($\mathbb{W}_2^{1,0}(Q)$)

Der Raum $\mathbb{W}_2^{1,0}(Q)$ ist der Raum aller (Äquivalenzklassen von) Funktionen $y \in \mathbb{L}^2(Q)$, die alle schwachen Ableitungen erster Ordnung nach $x_1, \dots, x_n$ im Raum $\mathbb{L}^2(Q)$ besitzen.

Versehen mit der Norm

$$\|y\|_{\mathbb{W}_2^{1,0}(Q)} = \left(\|y\|_{\mathbb{L}^2(Q)}^2 + \|\nabla_x y\|_{\mathbb{L}^2(Q)}^2 \right)^{\frac{1}{2}}$$

ist $\mathbb{W}_2^{1,0}(Q)$ ein Hilbertraum. Als abkürzende Schreibweise benutzt man

$$\mathbb{H}^{1,0}(Q) := \mathbb{W}_2^{1,0}(Q) = \{ y \in \mathbb{L}^2(Q) \mid D_i y \in \mathbb{L}^2(Q) \quad \forall i = 1, \dots, n \}.$$

Soll für Funktionen $y \in \mathbb{H}^{1,0}(Q)$ die einfache zeitliche Ableitung auch in $\mathbb{L}^2(Q)$ liegen, so verwendet man den Raum

$$\mathbb{H}^{1,1}(Q) := \mathbb{W}_2^{1,1}(Q) := \{ y \in \mathbb{L}^2(Q) \mid y_t, D_i y \in \mathbb{L}^2(Q) \quad \forall i = 1, \dots, n \},$$

welcher versehen mit der Norm

$$\|y\|_{\mathbb{H}^{1,1}(Q)} := \left(\|y\|_{\mathbb{L}^2(Q)}^2 + \|y_t\|_{\mathbb{L}^2(Q)}^2 + \|\nabla_x y\|_{\mathbb{L}^2(Q)}^2 \right)^{\frac{1}{2}}$$

ein Hilbertraum ist.

Da prinzipiell nicht davon ausgegangen werden kann, dass die zeitliche Ableitung des Zustands im Raum $\mathbb{L}^2(Q)$ liegt, ist der Raum $\mathbb{W}_2^{1,1}(Q)$ nicht der passende Lösungsraum für einen semilinearen parabolischen Steuerprozess. Aus diesem Grund besteht die Notwendigkeit „geeigneter“ Funktionenräume zu finden. Die Herleitung des Funktionenraums $\mathbb{W}(0, T)$ wird hier nur sehr stichpunktartig skizziert. Eine umfassende Behandlung ist nur im Rahmen der Distributionentheorie möglich, was den vorhandenen Rahmen sprengen würde. Für genauere Ausführungen sei auf [23, 29, 11] verwiesen.

Zur Beschreibung instationärer Gleichungen hat sich das Konzept *abstrakter Funktionen* bewährt. Unter einer abstrakten Funktion versteht man Folgendes:

Definition 3.2.13 (Abstrakte Funktion)

Eine Abbildung von einem kompakten Intervall $[a, b] \subset \mathbb{R}$ in einen Banachraum heißt *abstrakte Funktion*.

Funktionen aus dem Raum $\mathbb{W}_2^{1,0}(Q)$ können auch als abstrakte Funktionen interpretiert werden, die jeden Zeitpunkt $t \in [0, T]$ auf eine Funktion $y(\cdot, t) \in \mathbb{H}^1(\Omega)$ abbilden.

Im Weiteren werden einige wichtige Räume abstrakter Funktionen bereitgestellt.

Definition 3.2.14 ($C([a, b]; X)$)

$C([a, b]; X)$ ist der Raum aller stetigen abstrakten Funktionen $y : [a, b] \rightarrow X$. Eine abstrakte Funktion heißt *stetig im Punkt t* , wenn

$$\lim_{\tau \rightarrow t} \|y(\tau) - y(t)\| = 0$$

gilt.

Definition 3.2.15 (messbare abstrakte Funktion)

Eine abstrakte Funktion $y : [a, b] \rightarrow X$ heißt *messbar*, wenn eine Folge $\{y_k\}_{k=1}^\infty$ von Treppenfunktionen $y_k : [a, b] \rightarrow X$ existiert, so dass $y(t) = \lim_{k \rightarrow \infty} y_k(t)$ fast überall auf $[a, b]$ gilt.

Die Definition messbarer abstrakter Funktionen erlaubt es nun, $\mathbb{L}^p$ -Räume abstrakter Funktionen einzuführen.

Definition 3.2.16 ($\mathbb{L}^p(a, b; X), \mathbb{L}^\infty(a, b; X)$)

Unter $\mathbb{L}^p(a, b; X)$, $1 \leq p < \infty$, versteht man den Raum aller (Äquivalenzklassen von) messbaren abstrakten Funktionen $y : [a, b] \rightarrow X$ mit der Eigenschaft

$$\int_a^b \|y(t)\|_X^p dt < \infty.$$

$\mathbb{L}^\infty(a, b; X)$ besteht aus allen (Äquivalenzklassen von) fast überall beschränkten und messbaren abstrakten Funktionen.

Mit den Normen

$$\|y\|_{\mathbb{L}^p(a,b;X)} = \left(\int_a^b \|y(t)\|_X^p dt \right)^{\frac{1}{p}}, \quad \|y\|_{\mathbb{L}^\infty(a,b;X)} = \operatorname{ess\,sup}_{[a,b]} \|y(t)\|_X.$$

sind die $\mathbb{L}^p$ -Räume, $1 \leq p \leq \infty$, Banachräume.

Betrachtet man den Raum $\mathbb{L}^2(0, T; \mathbb{H}^1(\Omega))$, so lässt sich feststellen, dass die Normen $\mathbb{H}^{1,0}(Q)$ und $\mathbb{L}^2(0, T; \mathbb{H}^1(\Omega))$ identisch und die Räume sogar isomorph zueinander sind:

$$\mathbb{H}^{1,0}(Q) \cong \mathbb{L}^2(0, T; \mathbb{H}^1(\Omega)).$$

Es lässt sich zeigen, dass jede Funktion $y \in \mathbb{W}_2^{1,0}(\Omega)$ durch Abänderung auf einer Menge vom Maß Null zu einer Funktion aus $\mathbb{L}^2(0, T; \mathbb{H}^1(\Omega))$ wird und umgekehrt.

Nun kann der bereits erwähnte Funktionenraum $\mathbb{W}(0, T)$ eingeführt werden. Auf die Definition und Hintergründe zu *Distributionen* wird aus oben genanntem Grund nicht eingegangen.

Definition 3.2.17 ($\mathbb{W}(0, T)$)

$\mathbb{W}(0, T)$ ist der Raum aller $y \in \mathbb{L}^2(0, T; \mathbb{H}^1(\Omega))$ mit (distributioneller) Ableitung y' in $L^2(0, T; \mathbb{H}^1(\Omega)^*)$,

$$\mathbb{W}(0, T) := \{y \in \mathbb{L}^2(0, T; \mathbb{H}^1(\Omega)) \mid y' \in L^2(0, T; \mathbb{H}^1(\Omega)^*)\}$$

versehen mit der Norm

$$\|y\|_{\mathbb{W}(0,T)} = \left(\int_0^T \left(\|y(t)\|_{\mathbb{H}^1(\Omega)}^2 + \|y'(t)\|_{\mathbb{H}^1(\Omega)^*}^2 \right) dt \right)^{\frac{1}{2}}.$$

Wie bereits erwähnt, spielen in dieser Arbeit aufgrund der speziellen Vorgehensweise Sobolev-Räume für parabolische PDEs eine untergeordnete Rolle. Im folgenden Abschnitt werden deshalb elliptische Probleme betrachtet.

3.3 Schwache Formulierung

Die Motivation für das Konzept der schwachen Formulierung ist es, die Suche nach einer Lösung für eine PDE auf eine größere Menge von möglichen Funktionen als bei der klassischen Betrachtung zu erweitern. Man spricht im Allgemeinen nicht von **der** schwachen Formulierung einer PDE, da verschiedene Darstellungsweisen existieren, die sich im Wesentlichen in den zugrundeliegenden Funktionenräumen unterscheiden.

Anhand eines relativ einfachen Beispiels, übernommen aus [23], soll das grundsätzliche Vorgehen bei der schwachen Formulierung gezeigt werden. Ausgegangen wird von einer Randwertaufgabe mit Robin-Randbedingung:

$$\begin{aligned} -\Delta y + c_0 y &= f \quad \text{in } \Omega, \\ \partial_\nu y + \alpha y &= g \quad \text{auf } \Gamma, \end{aligned} \tag{3.2}$$

wobei Funktionen $f \in \mathbb{L}^2(\Omega)$ und $g \in \mathbb{L}^2(\Gamma)$ sowie die nichtnegativen Koeffizientenfunktionen $c_0 \in \mathbb{L}^\infty(\Omega)$, $\alpha \in \mathbb{L}^\infty(\Gamma)$ vorgegeben sind.

Die Funktion f kann sehr irregulär sein, weshalb eine klassische Lösung der PDE mit $y \in C^2(\Omega) \cap C^1(\overline{\Omega})$ meist nicht zu finden ist. Stattdessen wird eine schwache Lösung y im Raum $\mathbb{H}^1(\Omega)$ gesucht.

Zur Herleitung der schwachen Formulierung wird angenommen, dass f hinreichend glatt ist und $y \in C^2(\Omega) \cap C^1(\overline{\Omega})$ eine klassische Lösung von (3.2) darstellt. Da das Gebiet Ω beschränkt ist, existieren alle auftretenden Integrale. Durch Multiplikation der oberen Gleichung von (3.2) mit einer beliebigen, aber festen *Testfunktion* $v \in C^1(\overline{\Omega})$ und Integration über Ω erhält man

$$-\int_{\Omega} v \Delta y \, dx + \int_{\Omega} c_0 y v \, dx = \int_{\Omega} f v \, dx.$$

Durch partielle Integration ergibt sich

$$-\int_{\Gamma} v \partial_{\nu} y \, ds + \int_{\Omega} \nabla y \nabla v \, dx + \int_{\Omega} c_0 y v \, dx = \int_{\Omega} f v \, dx$$

und durch Einsetzen der Randbedingung dritter Art aus (3.2) folgt

$$\int_{\Omega} \nabla y \nabla v \, dx + \int_{\Omega} c_0 y v \, dx + \int_{\Gamma} \alpha y v \, ds = \int_{\Omega} f v \, dx + \int_{\Gamma} g v \, ds \quad (3.3)$$

für alle $v \in C^1(\overline{\Omega})$. Da $C^1(\overline{\Omega})$ in $\mathbb{H}^1(\Omega)$ dicht liegt, gilt diese letzte Gleichung auch für alle $v \in \mathbb{H}^1(\Omega)$. Umgekehrt kann aufgrund der oben genannten Voraussetzungen an y , insbesondere $y \in C^2(\Omega) \cap C^1(\overline{\Omega})$, und der Beliebigkeit von $v \in C^1(\overline{\Omega})$ aus der eben hergeleiteten Beziehung auf die Gleichung (3.2) geschlossen werden. Somit gelangt man zu folgender Definition:

Definition 3.3.1 (Schwache Lösung)

Ein $y \in \mathbb{H}^1(\Omega)$ heißt schwache Lösung der Randwertaufgabe (3.2), wenn die Variationsformulierung (3.3) für alle $v \in \mathbb{H}^1(\Omega)$ erfüllt ist.

Zu bemerken ist, dass für die schwache Formulierung eine schwache Differenzierbarkeit von y reicht. Existenz und Eindeutigkeit schwacher Lösungen sind für die Optimalsteuerung ein sehr bedeutsames Thema, auf das in dieser Arbeit nicht tiefer eingegangen wird. Stattdessen wird diesbezüglich auf [23, 22] verwiesen, wo Optimalsteuerungsprobleme mit semilinearen elliptischen Differentialgleichungen untersucht werden.

Nichtsdestotrotz spielt die schwache Formulierung einer PDE in dieser Arbeit eine wichtige Rolle, da sie die Grundlage für das numerische Lösen einer PDE mit DIFFPACK darstellt. Die Vorgehensweise zur Herleitung der schwachen Formulierung verläuft immer analog wie oben dargestellt (Multiplikation mit Testfunktionen und Integration über das Gebiet Ω), jedoch sind die entsprechenden Funktionenräume zu beachten, insbesondere bei nichtlinearen PDEs.

4 Diskretisierungsmethoden

Um die gegebenen Optimalsteuerungsprobleme mit Hilfe verschiedener Optimierer lösen zu können, müssen sowohl das Zielfunktional, als auch die Nebenbedingungen, insbesondere die partiellen Differentialgleichungen, diskretisiert werden. Zur Diskretisierung zeitabhängiger Differentialgleichungen bieten sich sogenannte *Linienmethoden* an. Dabei unterscheidet man, neben weiteren, weniger gebräuchlichen Linienmethoden, zwischen der *vertikalen* und der *horizontalen Linienmethode* (vgl. [8]). Es handelt sich jeweils um *Semidiskretisierungen*, denen ein weiterer Diskretisierungsschritt nachgeschaltet werden muss, um eine komplette Diskretisierung (*Volldiskretisierung*) zu erhalten. Der Grundgedanke der Semidiskretisierungen besteht darin, dass als (Zwischen-)Resultat Aufgaben bekannter Struktur entstehen.

Die Bezeichnung „vertikal“ oder „horizontal“ der Semidiskretisierungen erschließt sich aus der graphischen Darstellung des Definitionsbereichs der gesuchten Funktion im räumlich ein-dimensionalen Fall. Wird nämlich die erste (horizontale) Achse des Koordinatensystems für die Ortsvariable und die zweite (vertikale) Achse für die Zeitvariable reserviert, so erhält man durch die räumliche Diskretisierung Probleme, die entlang vertikaler Linien „leben“. Durch die zeitliche Diskretisierung entstehen entsprechend Probleme, die entlang horizontaler Linien „leben“ (vgl. [16]).

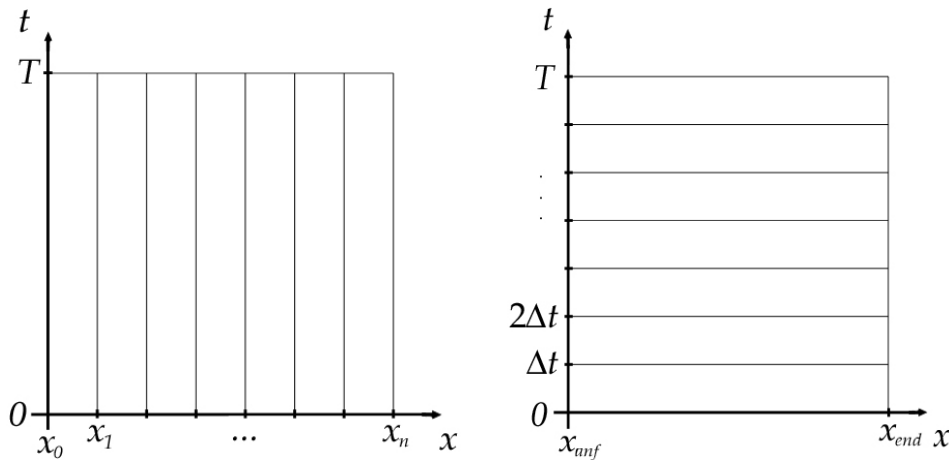


Abbildung 4.1: Vertikale und horizontale Linienmethode

Bei der gängigeren vertikalen Linienmethode führt man zunächst eine Ortsdiskretisierung durch, so dass ein System gewöhnlicher Differentialgleichungen entsteht, für dessen Behandlung eventuell ein passender problemspezifischer Löser existiert. Problematisch bei diesem Ansatz ist jedoch, dass diese Systeme für mehrere Ortsdimensionen sehr groß werden können (vgl. [11]).

In dieser Arbeit wird die horizontale Linienmethode, auch *Rothe-Methode* genannt, verwendet, bei der erst in der Zeit, dann im Ort diskretisiert wird. Die Zeitdiskretisierung der auftretenden Differentialgleichung mit Hilfe Finiter Differenzen liefert eine Folge elliptischer Randwertprobleme. Zur Ortsdiskretisierung der entstandenen elliptischen Randwertprobleme bedient man sich der Methode der Finiten Elemente.

Ein Grund für diese Herangehensweise ist die einfache Implementierbarkeit adaptiver Verfahren, was zu einer schnellen und genauen Lösung der Differentialgleichungen beiträgt. Des Weiteren lassen sich mit dieser Methode nicht nur Optimalsteuerungsprobleme mit linearen Nebenbedingungen behandeln, sondern es können auf diese Weise auch schwierigere Probleme mit Nichtlinearitäten in den Zustandsgleichungen und den Randbedingungen gelöst werden. Der Inhalt dieses Kapitels ist weitestgehend an dem Buch „Computation Partial Differential Equations“ von Langtangen angelehnt [17, insbesondere Kapitel 2] sowie an den Büchern [16, 12, 7] und den Diplomarbeiten [21, 11, 22].

4.1 Zeitdiskretisierung mittels Finiter Differenzen

4.1.1 Allgemeine Vorgehensweise

Generell werden bei Differenzenverfahren kontinuierliche Gebiete durch diskrete Mengen – in diesem Fall das Zeitintervall $[0, T]$ durch eine endliche Menge von Zeitpunkten mit Schrittweite Δt – ersetzt. Die Ableitungen nach der Zeit werden durch Differenzenapproximationen substituiert (siehe [12]). Da die in dieser Arbeit betrachteten Differentialgleichungen nur erste Ableitungen nach t enthalten, kann man mit Zweipunktapproximation in t -Richtung (als einfachste Möglichkeit) auskommen; man spricht im Ergebnis von *Zweischichtschemas* [12]. Ein weit verbreitetes Finite-Differenzen-Schema stellt die θ -Regel dar. Bei ihr wird die Gleichung $y_t = G$ durch

$$\frac{y^l - y^{l-1}}{\Delta t} = \theta G^l + (1 - \theta)G^{l-1}, \quad 0 \leq \theta \leq 1$$

approximiert mit Δt als Zeitschrittweite und y^l als abkürzende Schreibweise für $y(x, t^l) = y(x, l\Delta t)$.

Für spezielle Werte von θ erhält man unterschiedliche Verfahren:

- $\theta = 1$: *implizites Euler-Verfahren* (Euler-Rückwärts-Verfahren),
- $\theta = 0.5$: *Crank-Nicolson-Schema* (Mittelpunktsregel),
- $\theta = 0$: *explizites Euler-Verfahren* (Euler-Vorwärts-Verfahren).

Zu beachten ist, dass es beim expliziten Euler-Verfahren zu Stabilitätsproblemen kommen kann, weshalb von der Wahl $\theta = 0$ eher abgeraten wird. Im Gegensatz dazu zeichnen sich die Verfahren für $\theta \geq 0.5$ durch bedingungslose Stabilität für alle Δt aus. Das Crank-Nicolson-Schema besitzt den weiteren Vorteil, dass der Diskretisierungsfehler in der Zeit von der Ordnung Δt^2 ist, für alle anderen θ -Werte ist er eine Ordnung schlechter (vgl. [17, 11]).

Im Allgemeinen müssen bei expliziten Schemata strenge Stabilitätskriterien an Δt erfüllt sein, während implizite Verfahren für alle Werte von Δt stabil sind. Beispielsweise muss für die Diskretisierung der eindimensionalen Wärmeleitungsgleichung¹ die Formel $\Delta t \leq h^2/2$ mit h als Schrittweite der Ortsdiskretisierung gelten (vgl. [17, S. 677]). Auf diese Problematik wird in dieser Arbeit im Abschnitt Numerische Untersuchungen und Resultate, Testbeispiel 1 (7.1) genauer eingegangen.

¹ $y_t(x, t) = \frac{\partial^2 y(x, t)}{\partial x^2}$ mit $x \in \Omega$

4.1.2 Zeitdiskretisierung des Modellproblems

Die θ -Regel wird nun auf die parabolische Differentialgleichung des Modellproblems (3.1) angewendet:

$$\frac{y^l(x) - y^{l-1}(x)}{\Delta t} = \theta(\Delta_x y^l(x) + f^l(x)) + (1 - \theta)(\Delta_x y^{l-1}(x) + f^{l-1}(x))$$

$$y^l(x) - \theta \Delta t (\Delta_x y^l(x) + f^l(x)) = y^{l-1}(x) + (1 - \theta) \Delta t (\Delta_x y^{l-1}(x) + f^{l-1}(x))$$

mit $x \in \Omega$, $l = 1, \dots, N_T$, $0 \leq \theta \leq 1$, $y^l = y(x, t^l) = y(x, l\Delta t)$, $u^l = u(x, t^l)$ und $f^l = f(x, t^l)$. N_T gibt die Anzahl der Zeitdiskretisierungspunkte an, wobei $l = 0$ für den Anfangszeitpunkt steht. Für jeden diskreten Zeitschritt l ist nun also eine elliptische Differentialgleichung zu lösen.

Das Optimalsteuerungsproblem (3.1) wird somit zu folgendem semidiskretisierten Optimalsteuerungsproblem:

Minimiere

$$J(y^l, u^l) := \frac{1}{2} \|y^l(x) - y_\Omega(x)\|_{L^2(\Omega)}^2 + \frac{\lambda}{2} \|u^l(x)\|_{L^2(\Gamma)}^2$$

unter

$$\begin{aligned} y^l(x) - \theta \Delta t (\Delta_x y^l(x) + f^l(x)) &= y^{l-1}(x) + (1 - \theta) \Delta t (\Delta_x y^{l-1}(x) + f^{l-1}(x)) && \text{in } \Omega, \\ y^0(x) &= y_0(x) && \text{in } \Omega, \\ \partial_\nu y^l(x) + \vartheta(y^l(x)) + g^l(x) &= u^l(x) && \text{auf } \Gamma, \\ u_a &\leq u^l(x) \leq u_b && \text{auf } \Gamma, \end{aligned} \quad (4.1)$$

mit

$$l = 1, \dots, N_T.$$

4.2 Ortsdiskretisierung mittels Finiten Elemente

Die Methode der Finiten Elemente (kurz: FEM) bietet ein sehr flexibles, ausgereiftes und effektives numerisches Verfahren zur näherungsweise Lösung (elliptischer) partieller Differentialgleichungen. Sie gehört der Klasse der Variationsmethoden an und gilt gerade bei geometrisch komplizierten Gebieten als besonders anpassungsfähig (vgl. [7]). Da das Thema äußerst umfangreich ist, würde eine ausführliche Einführung den Rahmen dieser Arbeit sprengen. Aus diesem Grund werden hier nur die allgemeine Vorgehensweise und die wichtigsten Definitionen erläutert. Die im vorhergehenden Kapitel eingeführten Funktionenräume und die schwache Formulierung kommen nun bei den elliptischen Differentialgleichungen zum Einsatz. Die dargestellten Funktionen sind in diesen Ausführungen nicht mehr zeit-, sondern nur ortsabhängig.

4.2.1 Allgemeine Vorgehensweise und Definition eines Finiten Elements

Die Hauptidee der FEM ist es, eine Approximation der unbekannten Funktion $y(x)$ zu finden, die die partielle Differentialgleichung auf dem Gebiet Ω löst:

$$y \approx \hat{y} = \sum_{j=1}^N y_j N_j(x).$$

Die Funktionen $N_j(x)$ bezeichnen dabei sogenannte *Basis-, Ansatz- oder Formfunktionen*, auf deren Bedeutung später noch genauer eingegangen wird. Die Vektoren y_j , deren Dimension von der Raumdimension des Gebiets Ω abhängt, werden *Knotenvariable* genannt.

Zunächst wird das Berechnungsgebiet Ω in eine beliebig große Anzahl an Teilen – vorerst etwas ungenau als die Elemente bezeichnet – zerlegt.² Entsprechend der Dimension von Ω handelt es sich dabei um Gebiete gleicher Dimension, im eindimensionalen Fall beispielsweise um Intervalle, im zweidimensionalen zumeist um Drei- oder Vierecke und im dreidimensionalen um Tetraeder oder Quader.

Die bei der Zerlegung des Ortsgebiets entstehende *endliche* Anzahl an Elementen, deren Größe endlich („finit“) klein ist, macht deutlich, dass der Name „Methode der Finiten Elemente“ für die geschilderte Vorgehensweise sehr zutreffend ist. Außerdem lassen sich die Elemente durch eine endliche Zahl von Parametern beschreiben (vgl. [2]).

Die Unterteilung des Gebiets Ω in mehrere Teilgebiete erfüllt bestimmte Eigenschaften und wird als *Triangulierung* bezeichnet (siehe [12]):

Definition 4.2.1 (Triangulierung)

Eine Zerlegung $T_h = \{T_1, \dots, T_N\}$ des Gebiets $\Omega \subset \mathbb{R}^n$ derart, dass gilt

- jedes $T_i \in T_h$ ist abgeschlossen,
- jedes $T_i \in T_h$ hat ein nichtleeres Inneres $\text{int}(T_i)$, das ein Lipschitz-Gebiet ist,
- $\bar{\Omega} = \bigcup_{T \in T_h} T$,
- $\text{int}(T_i) \cap \text{int}(T_j) = \emptyset$, $i \neq j$
- jede Seite von $T_i \in T_h$ ist entweder Teilmenge des Randes von Ω oder identisch mit einer Seite eines anderen $T_j \in T_h$, $i \neq j$,

heißt **Triangulierung** von Ω .

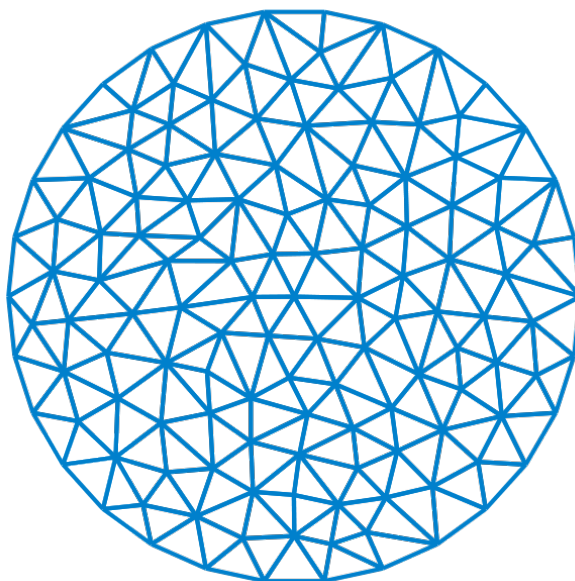


Abbildung 4.2: Triangulierung eines kreisförmigen Gebiets mittels Dreieckselementen

²Die exakte Definition eines Finiten Elements erfolgt am Ende dieses Abschnitts.

Anschließend wird die gesuchte Funktion y durch Kombination der oben bereits erwähnten Ansatzfunktionen N_j approximiert. Das Ziel besteht darin, die Knotenvariablen y_j so zu berechnen, dass der Fehler $y - \hat{y}$ oder die Norm des Fehlers $\|y - \hat{y}\|$ minimiert wird. Ein wesentlicher Punkt dabei ist, dass auf jedem einzelnen Teilgebiet e passende Koeffizienten y_j der Ansatzfunktionen N_j gesucht werden:

$$y^{(e)} \approx \hat{y}^{(e)} = \sum_{j=1}^N y_j^{(e)} N_j^{(e)}(x).$$

Bei den Ansatzfunktionen handelt es sich um gebietsweise vorgegebene Funktionen, die in der Regel nur auf wenigen Teilgebieten Werte ungleich null annehmen. Die Dimension des Raums der Ansatzfunktionen ist endlich, d. h. das ursprüngliche Problem wird nicht auf einem unendlich-dimensionalen Sobolev-Raum gelöst, sondern „nur“ auf einem endlich-dimensionalen Teilraum dieses Sobolev-Raums. Wichtig dabei ist, dass der Raum der Ansatzfunktionen dicht im eigentlichen Lösungsraum liegt. Die Ansatzfunktionen, zumeist Polynome beziehungsweise stückweise zusammengesetzte Polynome bestimmten Grades, bilden eine Basis dieses Teilraums.

Im Folgenden bezeichnet $P_k(K)$ die Menge der Polynome vom Grad k , α steht für einen im vorherigen Kapitel definierten Multiindex:

$$P_k(K) = \left\{ p : K \rightarrow \mathbb{R}, p(x) = \sum_{|\alpha| \leq k} y_\alpha x^\alpha \right\}.$$

$P_k(K)$ erfüllt die Eigenschaften eines Vektorraums, und jedes $p \in P_k(K)$ lässt sich beliebig oft differenzieren.

Bei Durchführung der beiden genannten Schritte (Unterteilung des Berechnungsgebiets Ω in endlich viele Teilgebiete und Darstellung der gesuchten Funktion y mittels Ansatzfunktionen als $\hat{y}$) erhält man nach Einsetzen der approximierten Lösung $\hat{y}$ in die schwache Formulierung der Differentialgleichung ein Gleichungssystem mit den Knotenvariablen $y_j^{(e)}$ als Unbekannten, das numerisch gelöst werden muss. Bei linearen PDEs ist dieses Gleichungssystem linear, ansonsten nichtlinear. Zur Lösung der oftmals dünnbesetzten Gleichungssysteme benutzt man effiziente iterative Algorithmen, im linearen Fall beispielsweise konjugierte Gradienten-Verfahren, im nichtlinearen Fall das Newton-Raphson-Verfahren.

Nachdem die prinzipielle Vorgehensweise bei der FEM kurz geschildert wurde, folgt nun die formale Definition eines *Finiten Elements*:

Definition 4.2.2 (Finites Element)

Ein *Finites Element* ist ein Tripel (Ω_e, Φ, K) mit folgenden Eigenschaften:

1. Ω_e ist ein konvexes polyedrisches Gebiet mit nichtleerem Inneren.
2. Φ ist der Raum der Ansatzfunktionen auf diesem Gebiet, Φ ist endlichdimensional.
3. K ist die Menge der Freiheitsgrade.

Verantwortlich für das Aussehen der Elemente sind die sogenannten *Geometriefunktionen*. In vielen Fällen stimmen sie mit den Ansatzfunktionen N_j überein; solche Elemente heißen *isoparametrische Elemente*, ansonsten *nicht-isoparametrische Elemente*.

Auf den dritten Bestandteil der Definition eines Finiten Elements, der Menge der *Freiheitsgrade*, wurde bislang noch nicht eingegangen. Unter den Freiheitsgraden versteht man unabhängige Informationen in Form von Anzahl, Lage und Art der Vorgaben für die Ansatzfunktionen.

Die Ansatzfunktionen sind durch „Daten“ wie Funktionswerte und Ableitungen an den Knoten der Diskretisierung von Ω eindeutig bestimmt.

Handelt es sich bei den Freiheitsgraden ausschließlich um Funktionswerte an den Knoten, so spricht man von *Lagrange-Elementen*. *Hermite-Elemente* werden durch weitere Freiheitsgrade, nämlich den Werten der Ableitungen an den Knoten, festgelegt (vgl. [12]).

4.2.2 Weitere Details zur Methode der Finiten Elemente

Es wurde bereits erwähnt, dass man zumeist Ansatzfunktionen verwendet, die nur auf wenigen Teilgebieten von Ω Werte ungleich null annehmen und überall sonst verschwinden. In Abbildung 4.3 ist das beispielsweise für stückweise lineare Ansatzfunktionen im eindimensionalen Fall visualisiert. Die Wahl von Polynomen als Ansatzfunktionen führt zu einem vollbesetzten Gleichungssystem, wodurch ein schnelles und effizientes Lösen erschwert wird. Um das zu verhindern, werden stattdessen stückweise Polynome herangezogen, die die Eigenschaft haben, dass sie jeweils an einem Knoten den Wert eins annehmen und ansonsten null sind. Also besitzen sie nur einen lokalen Träger. Im Allgemeinen werden die Ansatzfunktionen so gewählt, dass jedes N_i die beiden Eigenschaften erfüllt (siehe [17]):

1. N_i ist über jedem Element polynomial und durch seine Werte an den Knoten der Elemente eindeutig bestimmt.
2. $N_i(x^{[j]}) = \delta_{ij}$.

δ_{ij} stellt das Kronecker-Delta dar, und es gilt $\delta_{ij} = \begin{cases} 1, & \text{falls } i = j, \\ 0, & \text{sonst.} \end{cases}$

Punkt 2 impliziert die Eigenschaft, dass die Knotenvariable y_i dem Wert von $\hat{y}$ am i -ten Knoten entspricht, denn es gilt:

$$\hat{y}(x^{[i]}) = \sum_{j=1}^N y_j N_j(x^{[i]}) = \sum_{j=1}^N y_j \delta_{ij} = y_i.$$

Finite Elemente im eindimensionalen Raum

Im konkreten Fall *linearer Elemente* im Eindimensionalen ist das Gebiet Ω in Intervalle Ω_e zerlegt und man hat den Raum der Ansatzfunktionen Φ durch $P_1(\Omega_e)$ gegeben. Da die Werte der Polynome an den Rändern der einzelnen Teilintervalle Ω_e durch die Eigenschaft 2 festgelegt sind, sind die stückweisen Polynome N_i eindeutig bestimmt.

Es sei noch erwähnt, dass für den Raum $P_1(\Omega_e)$ die beiden folgenden lokalen Funktionen eine Basis darstellen:

$$\begin{aligned} \tilde{N}_1(x) &= x, \\ \tilde{N}_2(x) &= 1 - x. \end{aligned}$$

Diese Basis wird auch als „Hütchenbasis“ bezeichnet und ist in Abbildung 4.3, bei der $\Omega = [0, 1]$ in m nicht überlappende Elemente $\Omega_1, \dots, \Omega_m$ mit Knoten $x^{[j]}$, $j = 1, \dots, m+1$ unterteilt ist, zu sehen. Dort sind die Ansatzfunktionen $N_2(x)$, $N_3(x)$ und $N_4(x)$, die sich stückweise aus $\tilde{N}_1(x)$ und $\tilde{N}_2(x)$ zusammensetzen, dargestellt.

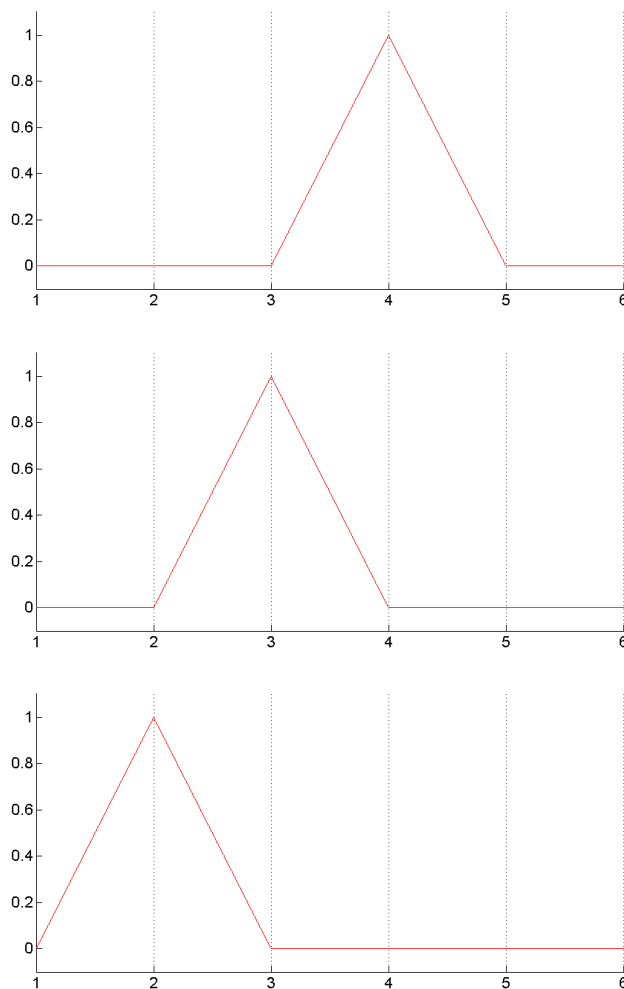


Abbildung 4.3: Stückweise lineare 1D-Ansatzfunktionen

Quadratische Basisfunktionen im Eindimensionalen unterscheiden sich von den linearen durch einen weiteren Punkt im Inneren der Intervalle. Außerdem lautet der Raum der Ansatzfunktionen $P_2(\Omega_e)$. Bekanntlich sind Polynome zweiten Grades durch drei Punkte (in diesem Fall die beiden Intervallrandpunkte und den Intervallmittelpunkt) determiniert, also lassen sich auch die Ansatzfunktionen N_i eindeutig bestimmen.

Eine Basis für den Raum $P_2(\Omega_e)$ bilden beispielsweise die Funktionen

$$\begin{aligned}\tilde{N}_1(x) &= \frac{1}{2}x(x-1), \\ \tilde{N}_2(x) &= (1+x)(1-x), \\ \tilde{N}_3(x) &= \frac{1}{2}x(x+1).\end{aligned}$$

Abbildung 4.4 zeigt die drei Basisfunktionen $\tilde{N}_1$, $\tilde{N}_2$ und $\tilde{N}_3$ im Referenzelement $[-1, 1]$, auf dessen Bedeutung später eingegangen wird.

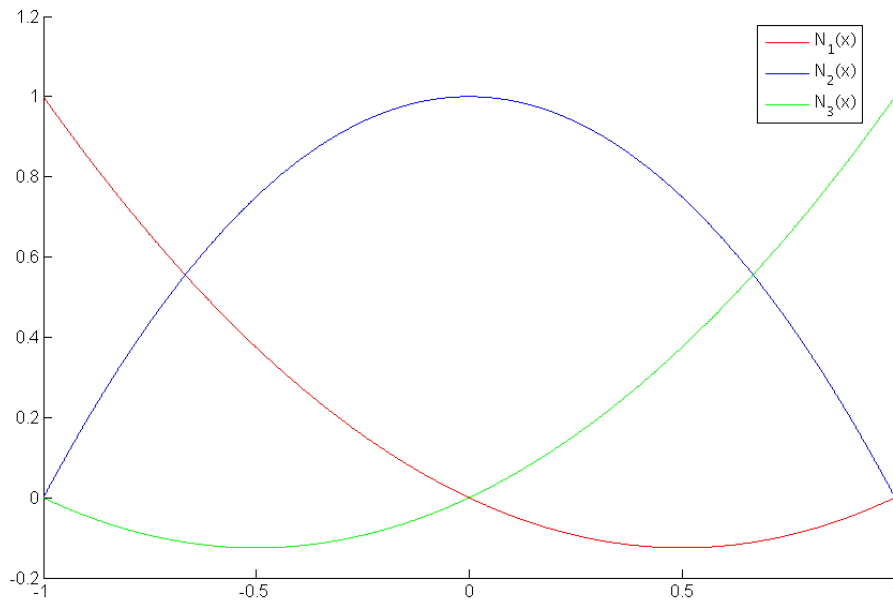


Abbildung 4.4: Quadratische 1D-Basisfunktionen im Referenzelement

Für den Raum der Ansatzfunktionen kann man natürlich auch stückweise Polynome höheren Grades verwenden. Dadurch erzielt man mitunter genauere Approximationen der kontinuierlichen Lösung an den Knoten, einher geht allerdings auch der aus der Polynominterpolation bekannte gravierende Nachteil der großen Schwingungen (siehe [11]).

Finite Elemente im zweidimensionalen Raum

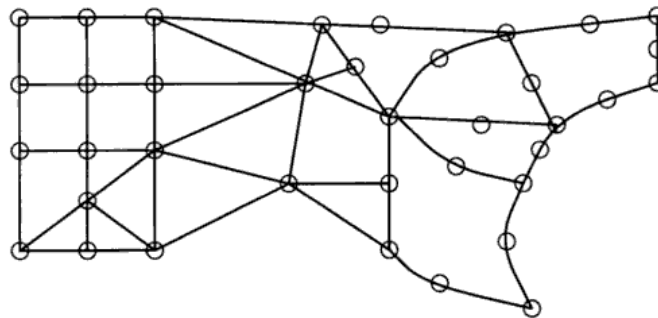


Abbildung 4.5: Beispiel einer diskretisierten, nichttrivialen Geometrie mit verschiedenförmigen 2D-Finiten Elementen

In Abbildung 4.5 ist dargestellt, wie Finite Elemente typischerweise im Zweidimensionalen aussehen können. Die Elemente haben die Form eines Drei- oder Vierecks, oftmals auch mit gekrümmten Seiten. Dadurch gewinnt man eine große Flexibilität beim Diskretisieren von Gebieten komplizierter Gestalt. Eine wichtige Forderung bei der Unterteilung ist, dass ein Knoten am Rand zwischen zwei Elementen ein in *beiden* Elementen „zulässiger“ Knoten sein muss.

Die Wahl der Ansatzfunktionen erfolgt analog der Vorgehensweise im eindimensionalen Raum, sie sind ebenfalls durch die Werte an den Knoten eindeutig bestimmt. Ob für die Elemente Drei- und/oder Vierecke gewählt werden, hängt von der Form des zu diskretisierenden Gebiets

ab, das grundsätzliche Verfahren ist aber in beiden Fällen das gleiche.

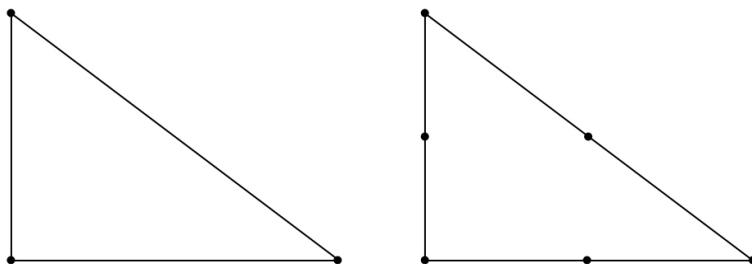


Abbildung 4.6: Lineares und quadratisches Lagrange-Dreiecks-Element

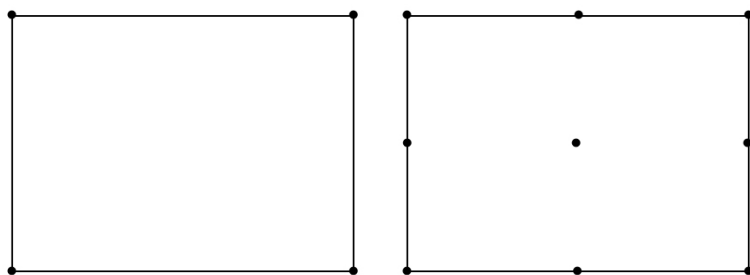


Abbildung 4.7: Bilineares und biquadratisches Lagrange-Rechtecks-Element

Die Abbildungen 4.6 und 4.7 zeigen das lineare bzw. quadratische Lagrange-Dreiecks-Element und das bilineare bzw. biquadratische Rechtecks-Element. Die Freiheitsgrade dieser Elemente bestehen, wie im vorherigen Abschnitt erwähnt, ausschließlich aus Funktionswerten an den Knotenpunkten.

Beim Lagrange-Dreieck mit linearen Ansatzfunktionen aus $P_1(\Omega_e)$ ist die Menge der Freiheitsgrade K durch die Funktionswerte der Polynome p an den drei Eckpunkten z_i gegeben:

$$K = \{p(z_i) : i = 1, 2, 3\}.$$

In Abbildung 4.8 sind lineare Basisfunktionen im Zweidimensionalen bei Dreieckselementen dargestellt:

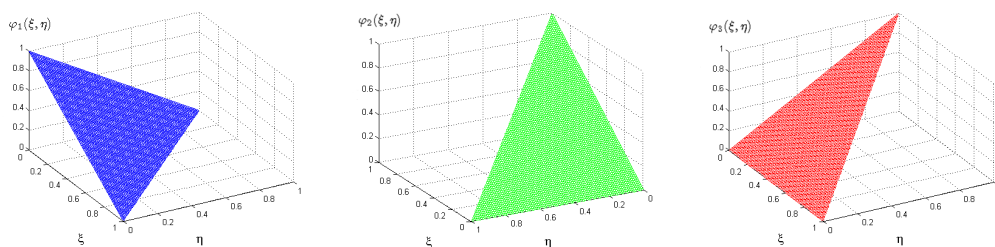


Abbildung 4.8: Lineare 2D-Ansatzfunktionen bei Dreieckselementen

Die Bestimmung quadratischer Ansatzfunktionen oder (stückweiser) Polynome höherer Ordnung ist durch Angabe der Koordinaten an den Ecken nicht mehr eindeutig. Deshalb benötigt man weitere Freiheitsgrade in Form von Funktionswerten an zusätzlichen Knoten. Für das

Dreieckselement mit quadratischen Ansatzfunktionen sind beispielsweise sechs Knoten, die drei Ecken und die Mittelpunkte der Seiten, zur eindeutigen Festlegung erforderlich. Mitunter gestaltet sich die Bestimmung der Basisfunktionen im zwei- oder höherdimensionalen Fall schwierig. Deshalb versucht man einfachere Ansätze wie beispielsweise die sogenannte *Tensorprodukt-Verallgemeinerung der 1D-Elemente* für bilineare 2D-Elemente zu verwenden. Weitere Details diesbezüglich sind zu finden unter [17, Kapitel 2.7.1 und 2.7.2].

Natürlich lässt sich die FEM auch auf Probleme mit Gebieten höherer Raumdimension anwenden. Diese Diplomarbeit beschränkt sich jedoch auf ein- und zweidimensionale Berechnungen, deshalb wird auf weitere Darstellungen in dieser kurzen Einführung verzichtet; für Ansätze höherer Ordnung vergleiche z. B. [17, Kapitel 2.7]).

Transformationen

In der Praxis ist das Berechnungsgebiet im Normalfall unregelmäßig und die Elemente sind verschiedenförmig, mitunter deformiert. In Abbildung 4.5 ist ein typisches FE-Gitter mit gemischten, zum Teil deformierten Finiten Elementen zu sehen. Zur Bestimmung der schwachen Lösung einer PDE müssen Integrationen über die verschiedenen Elemente durchgeführt werden. Die Unregelmäßigkeiten der Elemente erschweren die Berechnungen enorm.

Um auch bei „unregelmäßigen Gebieten“ in einheitlicher Art und Weise integrieren zu können, werden sämtliche Teilgebiete gleicher Form auf ein sogenanntes *Referenzelement* transformiert. Ein entscheidender Vorteil besteht darin, dass so bei allen Teilgebieten für die Integration das identische Integrationsgebiet zu Grunde liegt. Das Referenzelement hat eine einfache, regelmäßige Form und seine Punkte werden durch *lokale Koordinaten* angegeben. Außerdem lassen sich dort die *Formfunktionen* N_i explizit darstellen.

Nach der Diskretisierung des Gebiets Ω führt man eine Elementnummerierung ein, die jedem Teilelement des diskretisierten Gebiets eine feste Nummer zuweist. Mit Hilfe der lokalen Koordinaten eines Punkts im Referenzelement und der Nummer des Elements lassen sich die *globalen Koordinaten* eines Punkts reproduzieren; umgekehrt werden bei fester Nummerierung der Elemente jedem Punkt lokale Koordinaten im Referenzelement und eine Elementnummer zugewiesen. Diese Transformation stellt ein wichtiges Hilfsmittel dar und ist für alle Elementtypen durchführbar.

Beispielsweise dient im eindimensionalen Fall das Einheitsintervall $[-1, 1]$ als Referenzelement, im Zweidimensionalen werden Rechtecke auf das Einheitsquadrat $[-1, 1] \times [-1, 1]$ abgebildet, Dreiecke auf $[0, 1] \times [0, 1]$, wobei die Summe der lokalen Koordinaten kleiner oder gleich eins ist.

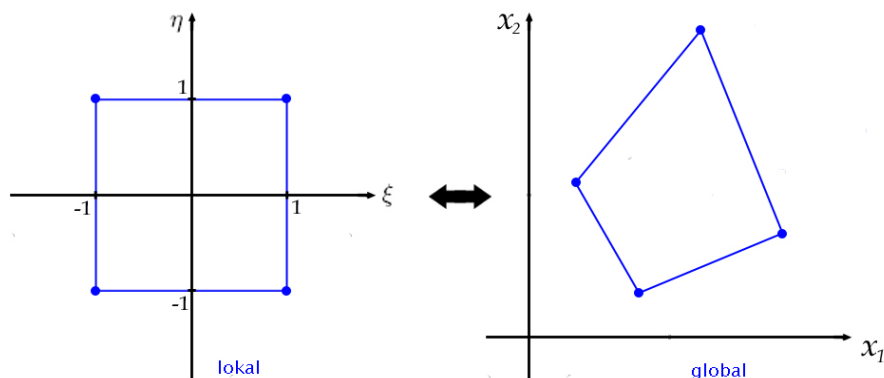


Abbildung 4.9: Transformation eines zweidimensionalen Gebiets auf das zugehörige Vierecks-Referenzelement

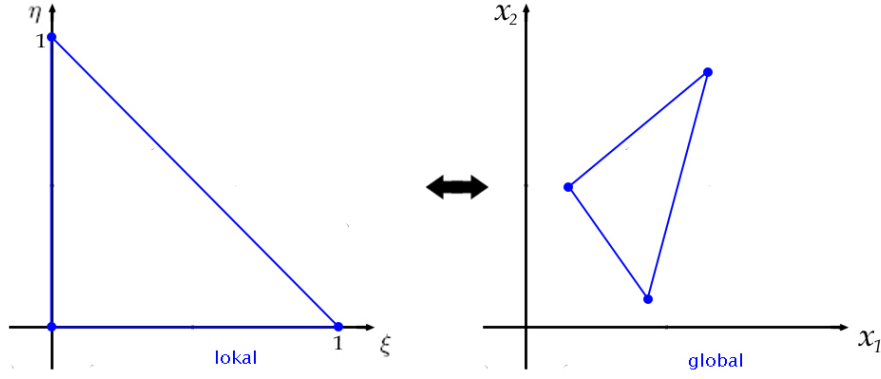


Abbildung 4.10: Transformation eines zweidimensionalen Gebiets auf das zugehörige Dreiecks-Referenzelement

Die Transformation von lokalen in globale Koordinaten und umgekehrt ist in den obigen Abbildungen 4.9 und 4.10 für Vierecks- und Dreieckselemente visualisiert.

Nachfolgende Tabelle aus [21] beinhaltet eine Zusammenstellung wichtiger Referenzelemente im 1D- und 2D-Fall. Neben dem Gebiet des jeweiligen Referenzelements ist die Formel zur Abbildung der lokalen Koordinaten in die globalen aufgeführt und die lokalen Basisfunktionen sind explizit angegeben.

1-D, N_i linear	Eigenschaften: Koordinatenabbildung: Ansatzfunktionen:	$\tilde{\Omega} = [-1, 1], \xi \in [-1, 1], n_e = 2$ $x^e(\xi) = \frac{1}{2}(x^e + x^{e+1}) + \xi \frac{1}{2}(x^e - x^{e+1})$ $\tilde{N}_i(\xi) = a_i \xi + b_i$ $\tilde{N}_1(\xi) = -\frac{1}{2}\xi + \frac{1}{2}$ $\tilde{N}_2(\xi) = \frac{1}{2}\xi + \frac{1}{2}$
1-D, N_i quadratisch	Eigenschaften: Koordinatenabbildung: Ansatzfunktionen:	$\tilde{\Omega} = [-1, 1], \xi \in [-1, 1], n_e = 3$ $x^e(\xi) = x^{2e} + \xi \frac{1}{2}(x^{2e+1} - x^{2e-1})$ $\tilde{N}_i(\xi) = a_i \xi^2 + b_i \xi + c_i$ $\tilde{N}_1(\xi) = \frac{1}{2}\xi^2 - \frac{1}{2}\xi$ $\tilde{N}_2(\xi) = 1 - \xi^2$ $\tilde{N}_3(\xi) = \frac{1}{2}\xi^2 + \frac{1}{2}\xi$
2-D, N_i linear (Dreieckselement)	Eigenschaften: Koordinatenabbildung: Ansatzfunktionen:	$\tilde{\Omega} = [0, 1] \times [0, 1],$ $\xi, \eta \in [0, 1], n_e = 3$ $x^e(\xi, \eta) = \sum_{i=1}^{n_e} \tilde{N}_i(\xi, \eta) x^{q(e,i)}$ $\tilde{N}_i(\xi, \eta) = a_i + b_i \xi + c_i \eta$ $\tilde{N}_1(\xi, \eta) = 1 - \xi - \eta$ $\tilde{N}_2(\xi, \eta) = \xi$ $\tilde{N}_3(\xi, \eta) = \eta$

2-D, N_i linear (Viereckselement)	Eigenschaften: Koordinatenabbildung: Ansatzfunktionen:	$\tilde{\Omega} = [-1, 1] \times [-1, 1]$, $\xi, \eta \in [-1, 1], n_e = 4$ $x^e(\xi, \eta) = \sum_{i=1}^{n_e} \tilde{N}_i(\xi, \eta) x^{q(e,i)}$ $\tilde{N}_i(\xi, \eta) = a_i + b_i \xi + c_i \eta + d_i \eta \xi$ $\tilde{N}_1(\xi, \eta) = \frac{1}{4}(1 - \xi - \eta + \xi \eta)$ $\tilde{N}_2(\xi, \eta) = \frac{1}{4}(1 + \xi - \eta - \xi \eta)$ $\tilde{N}_3(\xi, \eta) = \frac{1}{4}(1 - \xi + \eta - \xi \eta)$ $\tilde{N}_4(\xi, \eta) = \frac{1}{4}(1 + \xi + \eta + \xi \eta)$
--	--	--

Tabelle 4.1: Beispiele für Finite Elemente und ihre Abbildung auf lokale Referenzelemente

Variationsformulierung und Galerkin-Ansatz

Für die in jedem Zeitschritt l zu lösende elliptische Gleichung der Nebenbedingungen des semidiskretisierten Modellproblems (4.1) lässt sich die schwache Formulierung in Form einer sogenannten *Variationsgleichung* aufstellen (vgl. [16, S. 39] und [12, S. 91]).

Sie lautet bei gegebener, zur elliptischen Gleichung gehörender Bilinearform $a(\cdot, \cdot) : V \times V \rightarrow \mathbb{R}$ und einer Linearform $b \in V^*$ (dem Dualraum von V , also $b : V \rightarrow \mathbb{R}$) mit $V = \mathbb{H}^1(\Omega)$:

$$\text{Finde } y^l \in V \quad \text{mit} \quad a(y^l, v) = b(v) \quad \text{für alle } v \in V.$$

Obiger Ansatz wird als *Galerkin-Ansatz* bezeichnet.

Alternativ zum Galerkin-Ansatz existiert der sogenannte *Ritz-Ansatz*. Bei ihm ist ein $y^l \in V$ gesucht, das das Energiefunktional $E(v) = \frac{1}{2}a(v, v) - b(v)$ minimiert; es soll also gelten $E(y^l) = \min \{E(v) \mid v \in V\}$.

Beide Verfahren resultieren in unterschiedlichen zu lösenden Gleichungen, sind aber mathematisch äquivalent. In dieser Arbeit wird jedoch ausschließlich der Galerkin-Ansatz verwendet.

Hat man einen endlich-dimensionalen Unterraum $\hat{V} \subset V$, der in V dicht liegt und V approximiert (meist ein entsprechender $P_k(K)$ -Raum), so bleibt die folgende Aufgabe zu lösen:

$$\text{Finde } \hat{y}^l \in \hat{V} \quad \text{mit} \quad a(\hat{y}^l, \hat{v}) = b(\hat{v}) \quad \text{für alle } \hat{v} \in \hat{V}.$$

$\hat{y}^l$ approximiert y^l , $\hat{v}$ ist die Näherung von v :

$$\hat{y}^l = \sum_{i=1}^N y_i^l N_i, \quad \hat{v} = \sum_{i=1}^N v_i N_i.$$

Es gilt:

$$\begin{aligned} a(\hat{y}^l, \hat{v}) = b(\hat{v}) &\Leftrightarrow a(\hat{y}^l, N_i) = b(N_i) \quad \forall \quad i = 1, \dots, N, \\ &\Leftrightarrow a\left(\sum_{j=1}^N y_j^l N_j, N_i\right) = b(N_i) \quad \forall \quad i = 1, \dots, N, \\ &\Leftrightarrow \sum_{j=1}^N a(N_j, N_i) y_j^l = b(N_i) \quad \forall \quad i = 1, \dots, N, \\ &\Leftrightarrow \tilde{A} \tilde{y} = \tilde{b} \end{aligned}$$

mit $\tilde{A} = (a(N_j, N_i))_{ji} \in \mathbb{R}^{N \times N}$, $\tilde{y} = (y_j^l)_{j=1, \dots, N}^T$, $\tilde{b} = (b(N_i))_i \in \mathbb{R}^N$.

Die Galerkin-Methode liefert also für jeden Zeitschritt ein Gleichungssystem mit Matrix $\tilde{A}$ als *Steifigkeitsmatrix*. Eigenschaften dieses Gleichungssystems hinsichtlich Linearität und die Konsequenzen daraus werden im kommenden Abschnitt bei der Anwendung der FEM auf das Modellproblem aufgezeigt.

4.2.3 Anwendung der FEM auf das Modellproblem

Die FEM wird nun auf das semidiskretisierte Modellproblem (4.1), bei dem in jedem diskreten Zeitschritt l ein elliptisches Optimalsteuerungsproblem zu lösen ist, angewendet. Genauer gesagt wird die FEM zur Ortsdiskretisierung der PDE-Nebenbedingungen eingesetzt, die Diskretisierung des Zielfunktional und der Anfangsbedingung wird nicht aufgezeigt. Die Lösung der PDE zum l -ten Zeitpunkt y^l bei gegebener Steuerung u^l wird wie folgt mit Hilfe von Basisfunktionen N_j und Knotenvariablen y_j^l approximiert:

$$y^l \approx \hat{y}^l = \sum_{j=1}^N y_j^l N_j(x).$$

Diese Approximation setzt man anschließend in die im vorhergehenden Kapitel eingeführte schwache Formulierung ein und erhält in Abhängigkeit von den Eigenschaften der Funktion $\vartheta(y^l(x))$ ein lineares oder nichtlineares Gleichungssystem.

Anwendung der FEM auf das Modellproblem im linearen Fall

Für den Fall, dass die Funktion $\vartheta(y^l(x))$ linear in y^l ist, ergeben sich für die PDE-Nebenbedingung nach partieller Integration und Einsetzen der Randbedingungen (Greensche Formel) sowie Ausnutzen der Linearität von ϑ und Sortierung nach Integrationsgebieten die Gleichungen

$$\begin{aligned} & \sum_{j=1}^N \left[(M_{ij} + K_{ij}) - \theta \Delta t \int_{\Gamma} \vartheta(N_j) N_i d\Gamma \right] y_j^l \\ &= \int_{\Omega} \left(\theta \Delta t f^l + y^{l-1} + (1 - \theta) \Delta t f^{l-1} \right) N_i - \left((1 - \theta) \Delta t \nabla y^{l-1} \right) \nabla N_i d\Omega \\ &+ \int_{\Gamma} \left(\theta \Delta t (u^l - g^l) - (1 - \theta) \Delta t (\vartheta(y^{l-1}) - u^{l-1} + g^{l-1}) \right) N_i d\Gamma \end{aligned} \quad (4.2)$$

mit $M_{ij} = \int_{\Omega} N_i N_j d\Omega$, $K_{ij} = \theta \Delta t \int_{\Omega} \nabla N_i \nabla N_j d\Omega$

für $i = 1, \dots, N$.

Zu lösen bleibt also für jeden Zeitschritt l ($l = 1, \dots, N_T$) ein lineares Gleichungssystem in den Unbekannten y_j^l ($j = 1, \dots, N$) mit Koeffizientenmatrix A

$$a_{ij} = \left[(M_{ij} + K_{ij}) - \theta \Delta t \int_{\Gamma} \vartheta(N_j) N_i d\Gamma \right]$$

und rechter Seite b

$$\begin{aligned} b_i &= \int_{\Omega} \left(\theta \Delta t f^l + y^{l-1} + (1 - \theta) \Delta t f^{l-1} \right) N_i - \left((1 - \theta) \Delta t \nabla y^{l-1} \right) \nabla N_i d\Omega \\ &+ \int_{\Gamma} \left(\theta \Delta t (u^l - g^l) - (1 - \theta) \Delta t (\vartheta(y^{l-1}) - u^{l-1} + g^{l-1}) \right) N_i d\Gamma. \end{aligned}$$

Anwendung der FEM auf das Modellproblem im nichtlinearen Fall

Falls es sich bei $\vartheta(y^l(x))$ um eine nichtlineare Funktion handelt, so muss mit Hilfe eines iterativen Verfahrens ein nichtlineares Gleichungssystem gelöst werden. Das Standardverfahren zur Lösung nichtlinearer Gleichungssysteme ist das Newton-Raphson-Verfahren, auf das ausführlich in (6.1.2) eingegangen wird. Es sei vorerst nur soviel gesagt, dass bei ihm statt des Gleichungssystems $Ay = b$ ($y = (y_1^l, \dots, y_N^l)^T$) die äquivalente Aufgabe $F(y) = 0$ mit $F(y) = Ay - b$ gelöst wird. Dabei wird eine Iterationsschleife mit Iterationszähler k verwendet und $y_j^{l,k}$ bezeichnet die Approximation y_j^l in der k -ten Iteration. Die beiden für das Newton-Raphson-Verfahren wichtigen Größen F_i und J_{ij} werden für das Modellproblem (3.1) hier bereits angegeben, ihre Bedeutung und die Art, wie man sie berechnet, wird in Kapitel 6 erläutert:

$$\begin{aligned}
 F_i &= \int_{\Omega} \left(\hat{y}^l - \hat{y}^{l-1} - \theta \Delta t f^l - (1 - \theta) \Delta t f^{l-1} \right) N_i d\Omega \\
 &\quad + \int_{\Omega} \left(\theta \Delta t \nabla \hat{y}^l + (1 - \theta) \Delta t \nabla \hat{y}^{l-1} \right) \nabla N_i d\Omega \\
 &\quad + \int_{\Gamma} \left(\theta \Delta t \left(\vartheta(\hat{y}^l) - u^l + g^l \right) - (1 - \theta) \Delta t \left(\vartheta(\hat{y}^{l-1}) - u^{l-1} + g^{l-1} \right) \right) N_i d\Gamma \\
 J_{ij} &= \frac{\partial F_i}{\partial y_j^l} = M_{ij} + K_{ij} + \int_{\Gamma} \theta \Delta t \frac{d\vartheta(\hat{y}^{l,k})}{d\hat{y}} N_i N_j d\Gamma.
 \end{aligned}$$

5 Nichtlineare Optimierungsprobleme

Durch die im vorherigen Kapitel beschriebenen Diskretisierungsmethoden werden Optimalsteuerungsprobleme mit parabolischen Differentialgleichungsnebenbedingungen in nichtlineare Optimierungsprobleme umgewandelt, die man nun numerisch lösen möchte. Im Gegensatz zu Optimalsteuerungsproblemen mit PDEs, die auf unendlichdimensionalen Funktionenräumen „leben“, sind diese nichtlinearen Optimierungsprobleme *finit* - es können also Konzepte der endlichdimensionalen Optimierung angewendet werden, was eine wesentliche Vereinfachung darstellt. Da die Dimension der entstehenden Optimierungsprobleme meist sehr groß ist, spricht man von „large scale optimization“.

Die Entwicklung von Programmen zur Lösung dieser Probleme stellt ein eigenes Forschungsgebiet dar, weshalb auf bereits implementierte Routinen zurückgegriffen wurde.

Neben den bekannten SQP¹-Verfahren bieten sich zu diesem Zweck *Innere-Punkte-Methoden* an. Die in dieser Arbeit eingesetzten Optimierer, IPOPT² und SCIP³, verwenden beide Innere-Punkte-Verfahren, allerdings sind sie mit weiteren, unterschiedlichen Vorgehensweisen gekoppelt.

IPOPT und SCIP eignen sich besonders gut zur Lösung nichtlinearer Optimierungsprobleme mit einer großen Anzahl an Optimierungsvariablen und Nebenbedingungen. Auf den primal-dualen Innere-Punkte-Algorithmus mit Filter-Line-Search, der im IPOPT-Code programmiert wurde, wird ausführlich eingegangen. Die meisten Informationen zu IPOPT stammen aus [27]. Daneben wurde SCIP, ein von Zillober in Fortran77 implementierter Optimierer, der ein Inneres-Punkte-Verfahren mit der *Methode der beweglichen Asymptoten* (Method of Moving Asymptotes, MMA) und *Sequentieller konvexer Optimierung* (Sequential Convex Programming, SCP) kombiniert, verwendet. Die Hintergründe zu den Routinen dieser Algorithmen werden ebenfalls aufgeführt und sind hauptsächlich entnommen aus [30, 21]. Schöne Einführungen in die Nichtlineare Optimierung, an denen sich weitestgehend orientiert wurde, sind in [4, 9] zu finden. In dieser Arbeit sind ein paar Resultate der Nichtlinearen Optimierung zusammengefasst, die zur Erklärung der Informationen der verwendeten Optimierer benötigt werden.

5.1 Allgemeines nichtlineares Problem, Optimalitätsbedingungen und konvexe Optimierung

Bei einem allgemeinen *Optimierungsproblem* wird eine gegebene Funktion $f : S \rightarrow \mathbb{R}$, die *Zielfunktion*, innerhalb des *zulässigen Bereichs* $S \subset \mathbb{R}^n$ maximiert oder minimiert. Im Folgenden wird stets von einem Minimierungsproblem ausgegangen; Maximierungsprobleme lassen sich durch Multiplikation der Zielfunktion mit -1 leicht in Minimierungsprobleme transformieren. Ein allgemeines Optimierungsproblem in der n -dimensionalen Optimierungsvariablen x lautet

$$\begin{array}{ll} \text{Minimiere} & f(x) \\ \text{unter} & x \in S. \end{array} \quad (5.1)$$

¹Sequentiel Quadratic Programming

²Interior Point Optimizer

³Sequential Convex Programming Interior Points

Ein Optimalpunkt muss dabei innerhalb des zulässigen Bereichs S liegen, der häufig durch Gleichungs- und/oder Ungleichungsnebenbedingungen angegeben wird. Gilt $S = \mathbb{R}^n$, so spricht man von einem *unrestringierten Optimierungsproblem*, ansonsten von einem *restringierten*.

Definition 5.1.1 (Globales Minimum)

Ein Punkt $x^* \in S$ heißt **globales Minimum** von (5.1), falls gilt

$$f(x^*) \leq f(x) \quad \forall x \in S.$$

x^* heißt **striktes globales Minimum**, falls gilt

$$f(x^*) < f(x) \quad \forall x \in S, x \neq x^*.$$

Definition 5.1.2 (Lokales Minimum)

Ein Punkt $x^* \in S$ heißt **lokales Minimum** von (5.1), falls ein $\epsilon > 0$ existiert, so dass gilt

$$f(x^*) \leq f(x) \quad \forall x \in S \cap U_\epsilon(x^*);$$

dabei bezeichnet $U_\epsilon(x^*) := \{x \in \mathbb{R}^n \mid \|x - x^*\| < \epsilon\}$ die ϵ -Kugel um den Punkt x^* .

x^* heißt **striktes lokales Minimum**, falls ein $\epsilon > 0$ existiert mit

$$f(x^*) < f(x) \quad \forall x \in S \cap U_\epsilon(x^*), x \neq x^*.$$

Im Fall der restringierten Optimierung lässt sich die Menge S oftmals folgendermaßen darstellen:

$$S = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0, i = 1, \dots, m_{ieq}, h_j(x) = 0, j = 1, \dots, m_{eq}\}.$$

Somit gelangt man zur

Definition 5.1.3 (Standard-Optimierungsproblem)

Seien die differenzierbare Funktion $f : D \rightarrow \mathbb{R}$ und die stetig differenzierbaren Abbildungen $g_i : D \rightarrow \mathbb{R}$, $i = 1, \dots, m_{ieq}$, und $h_j : D \rightarrow \mathbb{R}$, $j = 1, \dots, m_{eq}$, gegeben und sei $S \subset D$. Dann bezeichnet man das *Optimierungsproblem*

$$\begin{array}{ll} \text{Minimiere} & f(x) \\ \text{unter} & x \in S \\ \text{mit} & S = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0, i = 1, \dots, m_{ieq}, h_j(x) = 0, j = 1, \dots, m_{eq}\} \end{array}$$

als **Standard-Optimierungsproblem (SOP)**.

Eine für die Herleitung von Optimalitätsbedingungen und Innere-Punkte-Methoden sehr wichtige Abbildung ist die *Lagrange-Funktion*:

Definition 5.1.4 (Lagrange-Funktion)

Gegeben sei ein (SOP) mit einem zulässigen Punkt $x \in S$. Die durch

$$\mathcal{L}(x, \lambda, \mu) := f(x) + \sum_{i=1}^{m_{ieq}} \lambda_i g_i(x) + \sum_{j=1}^{m_{eq}} \mu_j h_j(x)$$

definierte Abbildung $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}^{m_{ieq}} \times \mathbb{R}^{m_{eq}} \rightarrow \mathbb{R}$ heißt **Lagrange-Funktion** des restringierten Optimierungsproblems (SOP).

5.1.1 Optimalitätsbedingungen erster Ordnung

Mit Hilfe der Lagrange-Funktion werden nun die *Karush-Kuhn-Tucker-Bedingungen* eingeführt:

Definition 5.1.5 (Karush-Kuhn-Tucker-Bedingungen)

Gegeben sei ein (SOP) mit stetig differenzierbaren Funktionen f, g und h .

(i) Die Bedingungen

$$\begin{aligned}\nabla_x \mathcal{L}(x, \lambda, \mu) &= 0, \\ h(x) &= 0, \\ \lambda \geq 0, \quad g(x) \leq 0, \quad \lambda^T g(x) &= 0\end{aligned}$$

heißen *Karush-Kuhn-Tucker-Bedingungen* (kurz: *KKT-Bedingungen*) des (SOP), wobei

$$\nabla_x \mathcal{L}(x, \lambda, \mu) = \nabla f(x) + \sum_{i=1}^{m_{ieq}} \lambda_i \nabla g_i(x) + \sum_{j=1}^{m_{eq}} \mu_j \nabla h_j(x)$$

den Gradienten der Lagrange-Funktion $\mathcal{L}$ bezüglich der x -Variablen bezeichnet.

(ii) Jeder Vektor $(x^*, \lambda^*, \mu^*) \in \mathbb{R}^n \times \mathbb{R}^{m_{ieq}} \times \mathbb{R}^{m_{eq}}$, der den KKT-Bedingungen genügt, heißt *Karush-Kuhn-Tucker-Punkt* (kurz: *KKT-Punkt*) von (SOP); die Komponenten von λ^* und μ^* werden auch *Lagrange-Multiplikatoren* genannt.

Die KKT-Bedingungen stellen unter gewissen Regularitätsannahmen an die zulässige Menge sogenannte *constraint qualifications* (CQ), notwendige (im konvexen Fall sogar hinreichende) Bedingungen erster Ordnung dar. Wenn keine Restriktionen vorliegen, reduzieren sie sich auf die Forderung $\nabla f(x) = 0$ — das ist die übliche notwendige Optimalitätsbedingung erster Ordnung für unrestringierte Minimierungsaufgaben.

Eine der oben erwähnten Regularitätsannahmen ist die *LICQ*, die *linear independance constraint qualification*. Die Bedeutung der LICQ und weitere Details zu Optimalitätsbedingungen erster Ordnung sind zu finden unter [9, Abschnitt 2.2].

Eng verwandt mit den KKT-Bedingungen sind die *Fritz John-Bedingungen*, die ebenfalls notwendige Optimalitätsbedingungen erster Ordnung darstellen. Sie benötigen keinerlei Regularitätsannahmen, jedoch ist die Aussage im Vergleich zu den KKT-Bedingungen etwas schwächer:

Definition 5.1.6 (Fritz John-Bedingungen)

Gegeben sei ein (SOP) mit stetig differenzierbaren Funktionen f, g und h .

(i) Die Bedingungen

$$\begin{aligned}r \nabla f(x) + \sum_{i=1}^{m_{ieq}} \lambda_i \nabla g_i(x) + \sum_{j=1}^{m_{eq}} \mu_j \nabla h_j &= 0, \\ h(x) &= 0, \\ \lambda \geq 0, \quad g(x) \leq 0, \quad \lambda^T g(x) &= 0, \\ r &= 0\end{aligned}$$

heißen *Fritz John-Bedingungen* (kurz: *FJ-Bedingungen*) des (SOP).

(ii) Jeder Vektor $(r^*, x^*, \lambda^*, \mu^*) \in \mathbb{R} \times \mathbb{R}^n \times \mathbb{R}^{m_{ieq}} \times \mathbb{R}^{m_{eq}}$, der den FJ-Bedingungen genügt, heißt *Fritz John-Punkt* (kurz: *FJ-Punkt*) von (SOP).

Satz 5.1.1 (Fritz John-Punkt)

Sei $x^* \in \mathbb{R}^n$ ein lokales Minimum von (SOP). Dann existiert ein vom Nullvektor verschiedener Vektor $(r^*, \lambda^*, \mu^*) \in \mathbb{R} \times \mathbb{R}^{m_{ieq}} \times \mathbb{R}^{m_{eq}}$, so dass $(r^*, x^*, \lambda^*, \mu^*)$ ein FJ-Punkt für (SOP) ist.

Für den Beweis dieses Satzes sei auf [9, S. 62f] verwiesen.

5.1.2 Optimalitätsbedingungen zweiter Ordnung

Es sei erneut das (SOP) gegeben, wobei in diesem Abschnitt die Abbildungen $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $g : \mathbb{R}^n \rightarrow \mathbb{R}^{m_{ieq}}$ und $h : \mathbb{R}^n \rightarrow \mathbb{R}^{m_{eq}}$ als zweimal stetig differenzierbar vorausgesetzt sind. (x^*, λ^*, μ^*) sei ein KKT-Punkt des (SOP).

Die Indexmenge der in x^* aktiven Ungleichungsrestriktionen wird mit

$$I(x^*) := \{i \mid g_i(x^*) = 0\}$$

bezeichnet. Diese Menge lässt sich wie folgt zerlegen:

$$I(x^*) = I_0(x^*) \cup I_{>}(x^*)$$

mit

$$I_0(x^*) := \{i \in I(x^*) \mid \lambda_i^* = 0\}, \quad I_{>}(x^*) := \{i \in I(x^*) \mid \lambda_i^* > 0\}.$$

Nun kann man die für die notwendigen und hinreichenden Optimalitätskriterien wichtige Menge

$$\begin{aligned} \mathcal{T}(x^*) := \{d \in \mathbb{R}^n \mid & \nabla g_i(x^*)^T d = 0 \ (i \in I_{>}(x^*)), \\ & \nabla g_i(x^*)^T d \leq 0 \ (i \in I_0(x^*)), \\ & \nabla h_j(x^*)^T d = 0 \ (j = 1, \dots, m_{eq})\} \end{aligned}$$

definieren.

Satz 5.1.2 (Notwendiges Optimalitätskriterium zweiter Ordnung)

Sei $x^* \in X$ ein lokales Minimum von (SOP), welches der LICQ-Bedingung genügt. Dann ist

$$d^T \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*, \mu^*) d \geq 0 \quad \forall d \in \mathcal{T}(x^*), \ d \neq 0,$$

wobei $\lambda^* \in \mathbb{R}^{m_{ieq}}$ und $\mu^* \in \mathbb{R}^{m_{eq}}$ die eindeutig bestimmten Lagrange-Multiplikatoren zu x^* sind.

Satz 5.1.3 (Hinreichendes Optimalitätskriterium zweiter Ordnung)

Sei (x^*, λ^*, μ^*) ein KKT-Punkt von (SOP) mit

$$d^T \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*, \mu^*) d > 0 \quad \forall d \in \mathcal{T}(x^*), \ d \neq 0.$$

Dann ist x^* ein striktes lokales Minimum von (SOP).

Die Beweise zu diesen beiden Sätzen sind in [9, S. 64ff] zu finden.

5.1.3 Konvexe Optimierung

Dieser Abschnitt beschäftigt sich mit Hintergründen zur konvexen Optimierung. Neben konvexen Mengen und Funktionen wird ein konvexes Optimierungsproblem eingeführt.

Definition 5.1.7 (Konvexe Menge)

Eine Menge $X \subseteq \mathbb{R}^n$ heißt konvex, falls

$$\lambda x + (1 - \lambda)y \in X$$

für alle $x, y \in X$ und alle $\lambda \in (0, 1)$ gilt.

Geometrisch gesehen besagt diese Definition, dass eine Menge $X \subseteq \mathbb{R}^n$ konvex ist, wenn mit je zwei Punkten dieser Menge auch die gesamte Verbindungsstrecke dieser zwei Punkte zur Menge X gehört.

Definition 5.1.8 (Konvexe Funktion)

Sei $X \subseteq \mathbb{R}^n$ eine nichtleere konvexe Menge. Eine Funktion $f : X \rightarrow \mathbb{R}$ heißt

- konvex auf X , falls

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

für alle $x, y \in X$ und für alle $\lambda \in (0, 1)$ gilt.

- strikt konvex auf X , falls

$$f(\lambda x + (1 - \lambda)y) < \lambda f(x) + (1 - \lambda)f(y)$$

für alle $x, y \in X$ mit $x \neq y$ und für alle $\lambda \in (0, 1)$ gilt.

- gleichmäßig konvex auf X , wenn es eine Konstante $k > 0$ gibt, so dass

$$f(\lambda x + (1 - \lambda)y) + k\lambda(1 - \lambda) \|x - y\|^2 \leq \lambda f(x) + (1 - \lambda)f(y)$$

für alle $x, y \in X$ und für alle $\lambda \in (0, 1)$ gilt.

Stetig differenzierbare konvexe Funktionen sind durch nachfolgenden Satz charakterisiert:

Satz 5.1.4 (Charakterisierung differenzierbarer konvexer Funktionen)

Seien $X \subseteq \mathbb{R}^n$ eine offene und konvexe Menge sowie $f : X \rightarrow \mathbb{R}$ stetig differenzierbar. Dann gilt:

- f ist genau dann konvex auf X , wenn für alle $x, y \in X$ gilt:

$$f(x) - f(y) \geq \nabla f(y)^T (x - y).$$

- f ist genau dann strikt konvex auf X , wenn für alle $x, y \in X$ mit $x \neq y$ gilt:

$$f(x) - f(y) > \nabla f(y)^T (x - y).$$

- f ist genau dann gleichmäßig konvex auf X , wenn es ein $k > 0$ gibt mit

$$f(x) - f(y) \geq \nabla f(y)^T (x - y) + k \|x - y\|^2$$

für alle $x, y \in X$.

Der Beweis zu diesem Satz ist nachzulesen in [9, S. 26f].

Sind bei einem Optimierungsproblem die Zielfunktion f und die Ungleichungsnebenbedingungen $g_i, i = 1, \dots, m_{ieq}$ konvex sowie die Gleichungsbedingungen $h_j, j = 1, \dots, m_{eq}$ linear, so ist der zulässige Bereich konvex und man spricht von einem *konvexen Optimierungsproblem*.

Konvexe Optimierungsprobleme haben folgende wichtige Eigenschaft:

Satz 5.1.5 (Extrema bei konvexen Optimierungsproblemen)

Sei $X \subseteq \mathbb{R}^n$ nichtleer und konvex sowie $f : X \rightarrow \mathbb{R}$ konvex, dann ist jedes lokale Minimum des zugehörigen konvexen Optimierungsproblems bereits ein globales Minimum.

Daneben gibt es für konvexe Mengen, Funktionen und Optimierungsprobleme eine Vielzahl an Charakteristika, die beispielsweise in [4, 9] zu finden sind.

5.2 Methoden zur Lösung nichtlinearer Optimierungsprobleme

Um ein nichtlineares Optimierungsproblem der Form (SOP) numerisch zu lösen, kann man, wie bereits erwähnt, Innere-Punkte-Methoden einsetzen. Der folgende Abschnitt beinhaltet eine kurze Beschreibung der prinzipiellen Funktionsweise dieser Optimierungsverfahren (vgl. [27, 6, 24]).

5.2.1 Innere-Punkte-Methoden

Es wird von folgendem nichtlinearen Optimierungsproblem ausgegangen:

$$\begin{array}{ll} \text{Minimiere} & f(x) \\ \text{unter} & g_i(x) \leq 0, \quad i = 1, \dots, m, \end{array} \quad (5.2)$$

wobei $f : \mathbb{R}^n \rightarrow \mathbb{R}$ und $g := (g_1, \dots, g_m)^T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ zweimal stetig partiell differenzierbare Funktionen und x ein n -dimensionaler Optimierungsvektor unbeschränkter Variablen sind. Die Gleichheitsrestriktionen, in (SOP) mit $h := (h_1, \dots, h_{m_{eq}})$ bezeichnet, lassen sich jeweils durch zwei Ungleichungsnebenbedingungen darstellen.

Durch Einführen von *Schlupfvariablen* $w^T = (w_1, \dots, w_m)$ wird aus (5.2) ein Optimierungsproblem mit einer Gleichungsrestriktion und einer Vorzeichenbedingung für die Schlupfvariablen (in vektorieller Schreibweise):

$$\begin{array}{ll} \text{Minimiere} & f(x) \\ \text{unter} & g(x) + w = 0, \\ & w \geq 0. \end{array} \quad (5.3)$$

Zur Elimination der Vorzeichenbedingung für die Schlupfvariablen erweitert man die Zielfunktion um einen *logarithmischen Barriereterm*. Es wird anschließend eine Folge von *Barriere-Problemen*

$$\begin{array}{ll} \text{Minimiere} & f(x) - \mu \sum_{i=1}^m \ln(w_i) \\ \text{unter} & g(x) + w = 0 \end{array} \quad (5.4)$$

mit $\mu \geq 0$ gelöst. Der Barriereparameter μ stellt dabei eine Nullfolge dar, und für $\mu \rightarrow 0$ erwartet man, dass die Lösung von (5.4) gegen die Lösung des Problems (5.3) konvergiert.

Zur Lösung des Barriere-Problems (5.4) bedient man sich der aus der Nichtlinearen Optimierung bekannten *Lagrange-Methode*. Mit dem Lagrange-Multiplikator $\lambda \in \mathbb{R}^m$ erhält man folgende Lagrange-Funktion:

$$\mathcal{L}(x, w, \lambda) = f(x) - \mu \sum_{i=1}^m \ln(w_i) + \sum_{i=1}^m \lambda_i (g_i(x) + w_i). \quad (5.5)$$

Die notwendigen Optimalitätsbedingungen erster Ordnung bekommt man durch Nullsetzen der Ableitungen der Lagrange-Funktion nach den angegebenen Variablen:

$$\begin{aligned} \mathcal{L}_x(x, w, \lambda) &= \nabla f(x) + \nabla g(x)^T \lambda = 0, \\ \mathcal{L}_w(x, w, \lambda) &= -\mu W^{-1} e + \lambda = 0, \\ \mathcal{L}_\lambda(x, w, \lambda) &= g(x) + w = 0, \end{aligned} \quad (5.6)$$

wobei $W := \text{diag}(w_1, \dots, w_m)$, $e = (1, \dots, 1)^T \in \mathbb{R}^m$ und $\nabla g(x) \in \mathbb{R}^n$ die Jacobi-Matrix von $g(x)$ darstellt. Durch Multiplikation der zweiten Gleichung mit W gelangt man schließlich zu

einem *primal-dualen System*:

$$\begin{aligned} \nabla f(x) + \nabla g(x)^T \lambda &= 0, \\ -\mu e + W \Lambda e &= 0, \quad \Lambda := \text{diag}(\lambda_1, \dots, \lambda_m), \\ g(x) + w &= 0. \end{aligned} \quad (5.7)$$

Der Begriff primal-duales System ist selbsterklärend: die erste Gleichung entspricht der dualen, die dritte Gleichung der primalen Zulässigkeit.

Um iterativ eine approximative Lösung des primal-dualen Systems (5.7) zu erhalten, wendet man das *Newton-Verfahren* bzw. Varianten davon an. Seien die Matrizen H , A , Λ^k und W^k als

$$\begin{aligned} H(x, \lambda) &:= \nabla^2 f(x) + \sum_{i=1}^m \lambda_i \nabla^2 g_i(x), \quad A(x) := \nabla g(x), \\ \Lambda^k &:= \text{diag}(\lambda_1^k, \dots, \lambda_m^k), \quad W^k := \text{diag}(w_1^k, \dots, w_m^k), \end{aligned}$$

definiert, so muss in der k -ten Iteration des Newton-Verfahrens das Gleichungssystem

$$\begin{pmatrix} H(x^k, \lambda^k) & 0 & A(x^k)^T \\ 0 & \Lambda^k & W^k \\ A(x^k) & I & 0 \end{pmatrix} \begin{pmatrix} x^{k+1} - x^k \\ w^{k+1} - w^k \\ \lambda^{k+1} - \lambda^k \end{pmatrix} = \begin{pmatrix} -\nabla f(x^k) - A(x^k)^T \lambda^k \\ \mu e - W^k \lambda^k \\ -g(x^k) - w^k \end{pmatrix} \quad (5.8)$$

gelöst werden.

Die iterative Lösung eines Gleichungssystems ähnlich wie in (5.8) stellt die Grundlage beinahe aller Inneren-Punkte-Methoden dar. Dabei variieren, abhängig vom speziellen Verfahren, beispielsweise die Barrierefunktion oder die Meritfunktion⁴ der Newton-Iteration.

5.2.2 Details zum in IPOPT implementierten primal-dualen Innere-Punkte-Algorithmus mit Filter-Line-Search

Ausgegangen wird von einem (SOP), bei dem Ungleichungsnebenbedingungen bereits durch Einführen von Schlupfvariablen eliminiert wurden und man somit auf folgende äquivalente Form kommt:

$$\begin{aligned} \text{Minimiere} \quad & f(x) \\ \text{unter} \quad & h(x) = 0, \\ & x_L \leq x \leq x_U, \\ & x \in \mathbb{R}^n, \end{aligned} \quad (5.9)$$

wobei $x_L \in [-\infty, \infty]^n$ und $x_U \in (-\infty, \infty]^n$, $x_L^{(i)} \leq x_U^{(i)}$, obere und untere Schranken der Variablen x darstellen. Es wird angenommen, dass die Zielfunktion $f: \mathbb{R}^n \rightarrow \mathbb{R}$ und die Gleichungsnebenbedingungen $h: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ($m \leq n$) zweimal stetig differenzierbar sind.

Vor der Darstellung des Algorithmus in formal zusammengefasster Form am Ende dieses Abschnitts ist etwas Vorarbeit nötig. Nach dem primal-dualen Barriereansatz und der Lösung des Barriere-Problems werden eine Line-Search-Filter-Methode und Korrekturen zweiter Ordnung (SOC) präsentiert (vgl. [27]).

⁴Eine Funktion, die anstelle der Zielfunktion zur Kontrolle des erreichten Abstiegs bei der Schrittweitenberechnung verwendet wird.

Der primal-duale Barriereansatz

Um die Notation zu vereinfachen, wird die in IPOPT implementierte Methode für Problemformulierungen der Art

$$\begin{aligned} &\text{Minimiere} && f(x) \\ &\text{unter} && h(x) = 0, \\ &&& x \geq 0 \end{aligned} \quad (5.10)$$

beschrieben. Die notwendigen Änderungen zur Behandlung des allgemeinen Falls (5.9) sind in [27, Abschnitt 3.4] aufgezeigt.

Da es sich um eine Barriermethode handelt, berechnet der Algorithmus, wie im vorherigen Abschnitt dargestellt, Lösungen für eine Folge von Barriere-Problemen:

$$\begin{aligned} &\text{Minimiere} && \varphi_\mu(x) := f(x) - \mu \sum_{i=1}^n \ln(x_i) \\ &\text{unter} && h(x) = 0 \end{aligned} \quad (5.11)$$

mit einer fallenden Nullfolge von Barriereparametern μ . Die primal-dualen Gleichungen lauten in diesem Fall

$$\begin{aligned} \nabla f(x) + \nabla h(x)^T \lambda - z &= 0, \\ h(x) &= 0, \\ XZe - \mu e &= 0, \end{aligned} \quad (5.12)$$

wobei gilt $X := \text{diag}(x)$, $Z := \text{diag}(z)$. $\lambda \in \mathbb{R}^m$ und $z \in \mathbb{R}^n$ sind die Lagrange-Multiplikatoren der Gleichungsbedingungen beziehungsweise der (unteren) Schranken. Für $\mu = 0$ entsprechen die primal-dualen Gleichungen (5.12) zusammen mit $x, z \geq 0$ den KKT-Bedingungen des Originalproblems (5.10). Wenn zusätzlich constraint qualifications erfüllt sind, stellen sie die Optimalitätsbedingungen erster Ordnung für (5.10) dar.

Das in IPOPT verwendete Verfahren approximiert die Lösung des Barriere-Problems (5.11) für einen festen Wert von μ , verkleinert den Barriereparameter und berechnet, ausgehend von der vorherigen Approximation, die Lösung des neuen Barriere-Problems.

Die Abweichung eines Barriere-Problems von der Optimalität wird durch den *Optimalitätsfehler*

$$E_\mu := \max \left\{ \frac{\|\nabla f(x) + \nabla h(x)^T \lambda - z\|_\infty}{s_d}, \|h(x)\|_\infty, \frac{\|XZe - \mu e\|_\infty}{s_c} \right\} \quad (5.13)$$

mit nachfolgend definierten Skalierungsparametern $s_d, s_c \geq 1$ gemessen. Der Fehler $E_0(x, \lambda, z)$ gibt den Optimalitätsfehler für das Originalproblem (5.10) an. Der Algorithmus terminiert, sobald eine approximierende Lösung und entsprechende Multiplikatoren $(\tilde{x}_*, \tilde{\lambda}_*, \tilde{z}_*)$ gefunden wurden, die das Abbruchkriterium

$$E_0(\tilde{x}_*, \tilde{\lambda}_*, \tilde{z}_*) \leq \epsilon_{tol}, \quad (5.14)$$

mit einer vom Benutzer vorgegebenen Fehlertoleranz ϵ_{tol} , erfüllen.

Sogar bei einem gut skalierten Originalproblem kann es passieren, dass die Multiplikatoren λ und z sehr groß werden, beispielsweise wenn die Gradienten der aktiven Bedingungen (fast) linear von der Lösung von (5.10) abhängen. In diesem Fall kann es bei den unskalierten primal-dualen Gleichungen zu numerischen Schwierigkeiten kommen. Um das Abbruchkriterium für solche Gegebenheiten anzupassen, werden die Skalierungsfaktoren wie folgt gewählt:

$$s_d = \frac{\max \left\{ s_{max}, \frac{\|\lambda\|_1 + \|z\|_1}{(m+n)} \right\}}{s_{max}}, \quad s_c = \frac{\max \left\{ s_{max}, \frac{\|z\|_1}{n} \right\}}{s_{max}}.$$

Somit wird eine Komponente des Optimalitätsfehlers skaliert, wenn der Durchschnittswert der Multiplikatoren größer als eine feste Zahl $s_{max} \geq 1$ ist. Für den Fall, dass die Multiplikatoren divergieren, kann $E_0(x, \lambda, z)$ nur klein werden, wenn ein FJ-Punkt für (5.10) angenähert ist oder die primalen Variablen ebenfalls divergieren.

Um schnelle lokale Konvergenz gegen eine lokale Lösung von (5.10), die die starken hinreichenden Optimalitätsbedingungen zweiter Ordnung erfüllt, sicherzustellen, wird folgende Strategie angewendet:

Bezeichne j den Iterationszähler der „äußeren Schleife“ des Algorithmus. Es wird gefordert, dass die approximierte Lösung $(\tilde{x}_{*,j+1}, \tilde{\lambda}_{*,j+1}, \tilde{z}_{*,j+1})$ des Barriere-Problems (5.11) für einen gegebenen Wert von μ_j die Toleranz

$$E_{\mu_j}(\tilde{x}_{*,j+1}, \tilde{\lambda}_{*,j+1}, \tilde{z}_{*,j+1}) \leq \kappa_\epsilon \mu_j \quad (5.15)$$

für eine Konstante $\kappa_\epsilon > 0$ erfüllt, bevor der Algorithmus mit der Lösung des Barriere-Problems fortfährt. Den neuen Barriereparameter erhält man aus

$$\mu_{j+1} = \max \left\{ \frac{\epsilon_{tol}}{10}, \min \left\{ \kappa_\mu \mu_j, \mu_j^{\theta_\mu} \right\} \right\}, \quad (5.16)$$

wobei $\kappa_\mu \in (0, 1)$ und $\theta_\mu \in (1, 2)$ Konstanten darstellen.

Einerseits unterbindet diese Updateregeln, dass μ bei gegebener Toleranz ϵ_{tol} kleiner als nötig wird, was numerische Schwierigkeiten am Ende der Optimierungsroutine verhindert, andererseits zieht diese Regel ein superlineares Verhalten des Barriereparameters nach sich.

Ein weiterer Parameter, der später von Nutzen sein wird, ist

$$\tau_j = \max \{ \tau_{min}, 1 - \mu_j \} \quad (5.17)$$

mit Minimalwert $\tau_{min} \in (0, 1)$.

Lösung des Barriereproblems

Um das Barriere-Problem (5.11) für einen festen Wert μ_j zu lösen, wird bei IPOPT auf die primal-dualen Gleichungen (5.12) ein *gedämpftes Newton-Verfahren* angewendet. k bezeichnet im Folgenden den Iterationszähler der „inneren Schleife“ des Algorithmus.

Bei gegebener Iterierten (x_k, λ_k, z_k) mit $x_k, z_k > 0$ erhält man die Suchrichtungen $(d_k^x, d_k^\lambda, d_k^z)$ durch Linearisierung von (5.12) im Punkt (x_k, λ_k, z_k) , also

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^T & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{pmatrix} d_k^x \\ d_k^\lambda \\ d_k^z \end{pmatrix} = - \begin{pmatrix} \nabla f(x_k) + A_k \lambda_k - z_k \\ h(x_k) \\ X_k Z_k e - \mu_j e \end{pmatrix}. \quad (5.18)$$

Dabei ist $A_k := \nabla h(x_k)$ und W_k bezeichnet die Hessematrix der Lagrange-Funktion (des Originalproblems (5.10)) $\nabla^2 \mathcal{L}(x_k, \lambda_k, z_k)$

$$\mathcal{L}(x, \lambda, z) := f(x) + h(x)^T \lambda - z.$$

Statt das nichtlineare System (5.18) direkt zu lösen, wird die Lösung auf äquivalente Art berechnet, indem zuerst ein kleineres, symmetrisches, lineares System

$$\begin{bmatrix} W_k + \Sigma_k & A_k \\ A_k^T & 0 \end{bmatrix} \begin{pmatrix} d_k^x \\ d_k^\lambda \end{pmatrix} = - \begin{pmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ h(x_k) \end{pmatrix} \quad (5.19)$$

mit $\Sigma_k := X_k^{-1} Z_k$, abgeleitet von (5.18) durch Elimination der letzten Blockreihe, verwendet wird. Den Vektor d_k^z erhält man dann aus

$$d_k^z = \mu_j X_k^{-1} e - z_k - \Sigma_k d_k^x. \quad (5.20)$$

Wie bei den meisten Line-Search-Methoden muss sichergestellt werden, dass die Matrix im linken oberen Block der Matrix in (5.19), projiziert auf den Nullraum der Jacobimatrix der Nebenbedingungen A_k^T , positiv definit ist. Das ist gemäß [28] notwendig, um bestimmte Abstiegseigenschaften der nachfolgenden Filter-Line-Search-Prozedur zu garantieren. Genauso ist, falls die Matrix A_k nicht vollen Rang hat, die Iterationsmatrix in (5.19) singulär und es existiert möglicherweise keine Lösung für (5.19). Für diese Fälle muss die Iterationsmatrix abgeändert werden. In IPOPT werden deshalb zwei Skalare $\delta_w, \delta_c \geq 0$ eingeführt und das lineare System

$$\begin{bmatrix} W_k + \Sigma_k + \delta_w I & A_k \\ A_k^T & -\delta_c I \end{bmatrix} \begin{pmatrix} d_k^x \\ d_k^\lambda \end{pmatrix} = - \begin{pmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{pmatrix} \quad (5.21)$$

gelöst. Auf die Wahl der Skalare δ_w und δ_c wird hier nicht genauer eingegangen, sie ist in [27, Abschnitt 3.1] diskutiert.

Nach Berechnung der Suchrichtungen in (5.20) und (5.21) bleiben noch die Schrittweiten $\alpha_k, \alpha_k^z \in (0, 1]$ zu bestimmen, um die nächsten Iterierten

$$x_{k+1} := x_k + \alpha_k d_k^x \quad (5.22)$$

$$\lambda_{k+1} := \lambda_k + \alpha_k d_k^\lambda \quad (5.23)$$

$$z_{k+1} := z_k + \alpha_k^z d_k^z \quad (5.24)$$

zu erhalten. Die Variablen z haben aus Effizienzgründen andere Schrittweiten als die Variablen x und λ , weil so deren Schrittweite nicht unnötig eingeschränkt wird.

Da x und z positiv an einem Optimalpunkt des Barriere-Problems (5.11) sind, sollen auch alle Iterierten diese Eigenschaft besitzen. Das erreicht man, indem man die *fraction-to-the-boundary*-Regel

$$\alpha_k^{max} := \max \{ \alpha \in (0, 1] : x_k + \alpha d_k^x \geq (1 - \tau_j) x_k \} \quad (5.25)$$

$$\alpha_k^z := \max \{ \alpha \in (0, 1] : z_k + \alpha d_k^z \geq (1 - \tau_j) z_k \} \quad (5.26)$$

mit Parametern $\tau_j \in (0, 1)$ aus (5.17) verwendet.

Um globale Konvergenz zu sichern, wird die Schrittweite $\alpha_k \in (0, \alpha_k^{max}]$ für die Variablen außer z durch ein Backtracking-Line-Search-Verfahren, das eine absteigende Folge von Schrittweiten $\alpha_{k,l} = 2^{-l} \alpha_k^{max}$ (mit $l = 0, 1, 2, \dots$) untersucht, bestimmt. Auf dieses Verfahren wird im nächsten Abschnitt eingegangen.

Eine wichtige Forderung für den globalen Konvergenzbeweis des Verfahrens ist, dass die „Hessematrix des primal-dualen Barriereterms“ Σ_k nicht beliebig weit von der „primalen Hessematrix“ $\mu_j X_k^{-2}$ abweichen darf. Das wird durch das Setzen von

$$z_{k+1}^{(i)} := \max \left\{ \min \left\{ z_k^{(i)}, \frac{\kappa_\Sigma \mu_j}{x_k^{(i)}} \right\}, \frac{\mu_j}{\kappa_\Sigma x_k^{(i)}} \right\}, \quad i = 1, \dots, n \quad (5.27)$$

mit festem $\kappa_\Sigma \geq 1$ nach jedem Schritt gewährleistet. Es garantiert, dass jede Komponente $\sigma_{k+1}^{(i)}$ von Σ_{k+1} innerhalb des Intervalls $\left[\frac{1}{\kappa_\Sigma} \mu_j / (x_k^{(i)})^2, \kappa_\Sigma \mu_j / (x_k^{(i)})^2 \right]$ liegt.

Eine Line-Search-Filter-Methode

Die Grundidee der Filtermethode im Zusammenhang mit der Lösung des Barriere-Problems für μ_j besteht darin, (5.11) als Zweiziel-Optimierungsproblem zu betrachten. Die Zielsetzung ist es dann, sowohl die „normale“ Zielfunktion $\varphi_{\mu_j}(x)$ als auch die Nebenbedingungsverletzung $\theta(x) := \|h(x)\|$ zu minimieren. Ein Testpunkt $x_k(\alpha_{k,l}) := x_k + \alpha_{k,l}d_k^x$ wird während der Backtracking-Line-Search als akzeptierbar angesehen, wenn er einen hinreichenden Fortschritt gegenüber der aktuellen Iterierten hinsichtlich eines der Ziele bringt; das heißt also, falls

$$\theta(x_k(\alpha_{k,l})) \leq (1 - \gamma_\theta)\theta(x_k) \quad (5.28)$$

$$\text{oder } \varphi_{\mu_j}(x_k(\alpha_{k,l})) \leq \varphi_{\mu_j}(x_k) - \gamma_\varphi\theta(x_k) \quad (5.29)$$

bei festen Konstanten $\gamma_\theta, \gamma_\varphi \in (0, 1)$.

Allerdings wird das obige Kriterium durch die Forderung nach ausreichendem Fortschritt in der Barrierefunktion ersetzt, wenn für die aktuelle Iterierte die Bedingung $\theta(x_k) \leq \theta^{min}$ mit Konstante $\theta^{min} \in (0, \infty]$ erfüllt ist und die folgende *Switching-Bedingung*

$$\nabla\varphi_{\mu_j}(x_k)^T d_k^x < 0 \text{ und } \alpha_{k,l} [-\nabla\varphi_{\mu_j}(x_k)^T d_k^x]^{s_\varphi} > \delta [\theta(x_k)]^{s_\theta} \quad (5.30)$$

mit Konstanten $\delta > 0, s_\theta > 1, s_\varphi \geq 1$ gilt. Falls $\varphi(x_k) \leq \varphi^{min}$ und (5.30) für die aktuelle Schrittweite $\alpha_{k,l}$ gelten, muss der Testpunkt statt (5.28) die Armijo-Bedingung

$$\varphi_{\mu_j}(x_k(\alpha_{k,l})) \leq \varphi_{\mu_j}(x_k) + \eta_\varphi \alpha_{k,l} \nabla\varphi_{\mu_j}(x_k)^T d_k^x \quad (5.31)$$

erfüllen, wobei $\eta_\varphi \in (0, \frac{1}{2})$ eine Konstante darstellt.

Des Weiteren benutzt der IPOPT-Algorithmus in jeder Iteration k einen „Filter“, die Menge $\mathcal{F}_k \subseteq \{(\theta, \varphi) \in \mathbb{R}^2 : \theta \geq 0\}$. Der Filter $\mathcal{F}_k$ beinhaltet alle Kombinationen von Nebenbedingungsverletzungswerten θ und Barriere-Zielfunktionswerten φ , die für einen zulässigen Testpunkt in der k -ten Iteration verboten sind. Während der Line-Search wird ein Testpunkt $x_k(\alpha_{k,l})$ für den Filter $\mathcal{F}_k$ als nicht gültig angesehen, falls gilt $(\theta(x_k(\alpha_{k,l})), \varphi_{\mu_j}(x_k(\alpha_{k,l}))) \in \mathcal{F}_k$. Die Initialisierung des Filters erfolgt mit

$$\mathcal{F}_0 := \{(\theta, \varphi) \in \mathbb{R}^2 : \theta \geq \theta^{max}\} \quad (5.32)$$

bei fest vorgegebenem θ^{max} . Folglich erlaubt der gesamte Algorithmus niemals Testpunkte, deren Nebenbedingungsverletzung größer als θ^{max} sind, denn der Filter wird nach jeder Iteration, in der für die akzeptierte Schrittweite die Switching-Bedingung (5.30) oder die Armijo-Bedingung (5.31) nicht gilt, mit dieser Update-Formel erweitert:

$$\mathcal{F}_{k+1} := \mathcal{F}_k \cup \{(\theta, \varphi) \in \mathbb{R}^2 : \theta \geq (1 - \gamma_\theta)\theta(x_k) \text{ und } \varphi \geq \varphi_{\mu_j}(x_k) - \gamma_\varphi\theta(x_k)\}. \quad (5.33)$$

Dadurch ist garantiert, dass die Iterierten nicht in die Umgebung von x_k zurückspringen. Wenn die Bedingungen (5.30) und (5.31) jedoch erfüllt sind, bleibt der Filter unverändert. Somit stellt diese Routine also sicher, dass der Algorithmus nicht zyckelt, beispielsweise zwischen zwei Punkten, die abwechselnd die Nebenbedingungsverletzung und die Barriere-Zielfunktion verringern.

In manchen Fällen ist es nicht möglich, eine Schrittweite $\alpha_{k,l}$ zu finden, die das obere Kriterium (5.28) erfüllt. Eine minimal gewünschte Schrittweite wird durch Verwendung von Linearisierungen der beteiligten Funktionen approximiert. Dazu definiert man

$$\alpha_k^{min} := \gamma_\alpha \cdot \begin{cases} \min \left\{ \gamma_\theta, \frac{\gamma_\varphi\theta(x_k)}{-\nabla\varphi_{\mu_j}(x_k)^T d_k^x}, \frac{\delta[\theta(x_k)]^{s_\theta}}{[-\nabla\varphi_{\mu_j}(x_k)^T d_k^x]^{s_\varphi}} \right\}, & \text{falls } \nabla\varphi_{\mu_j}(x_k)^T d_k^x < 0 \text{ und } \theta(x_k) \leq \theta^{min} \\ \min \left\{ \gamma_\theta, \frac{\gamma_\varphi\theta(x_k)}{-\nabla\varphi_{\mu_j}(x_k)^T d_k^x} \right\}, & \text{falls } \nabla\varphi_{\mu_j}(x_k)^T d_k^x < 0 \text{ und } \theta(x_k) > \theta^{min} \\ \gamma_\theta, & \text{sonst} \end{cases} \quad (5.34)$$

mit einem „Sicherheitsfaktor“ $\gamma_\alpha \in (0, 1]$. Wenn die Backtracking-Line-Search eine Schrittweite $\alpha_{k,l} \leq \alpha_k^{min}$ liefert, greift der Algorithmus auf eine *Zulässigkeits-Zurückgewinnungs-Phase* zurück. Dort wird durch Reduktion der Nebenbedingungsverletzung mit einer iterativen Methode versucht, eine neue Iterierte $x_{k+1} > 0$, die für den aktuellen Filter zulässig ist und für die (5.28) erfüllt ist, zu erzeugen. Weitere Details dazu sind in [27, Abschnitt 3.3] geschildert.

Um globale Konvergenz des gesamten Verfahrens zu gewährleisten, ist es ausreichend, globale Konvergenz für jeden Barriereparameter μ_l sicherzustellen. Dafür wird der Filter $\mathcal{F}_k$ immer auf seinen Initialwert (5.32) gesetzt, sobald μ_l einen niedrigeren Wert annimmt.

Korrekturen zweiter Ordnung

Bei vielen nichtlinearen Optimierungsmethoden werden Korrekturen zweiter Ordnung zur Verbesserung des vorgeschlagenen Schritts verwendet, wenn der aktuelle Testpunkt nicht akzeptiert wurde. Eine Korrektur zweiter Ordnung (SOC) bei einem Schritt $\tilde{d}_k^x$ dient dazu, die Unzulässigkeit durch Anwenden eines weiteren Newtonschritts für die Nebenbedingungen am Punkt $x_k + \tilde{d}_k^x$ zu verringern, indem bei x_k die Jacobimatrix A_k^T verwendet wird. So wird in IPOPT eine Korrektur zweiter Ordnung $d_k^{x,soc}$ (beim Schritt $\tilde{d}_k^x = \alpha_{k,0}d_k^x$) eingesetzt, wenn die erste Testschrittweite $\alpha_{k,0}$ verworfen wurde und $\theta(x_k(\alpha_{k,0})) \geq \theta(x_k)$ gilt. $d_k^{x,soc}$ wird so berechnet, dass die Bedingung

$$A_k^T d_k^{x,soc} + h(x_k + \alpha_{k,0}d_k^x) = 0 \quad (5.35)$$

erfüllt ist. Die neue, korrigierte Suchrichtung lautet dann

$$d_k^{x,cor} = \alpha_{k,0}d_k^x + d_k^{x,soc}. \quad (5.36)$$

Jedoch ist die durch (5.35) definierte Korrektur zweiter Ordnung nicht eindeutig, und es wären mehrere Auswahlen möglich. Um weitere Matrixfaktorisierungen zu verhindern, nimmt die in IPOPT implementierte Methode die gleiche Matrix wie in (5.21), um den korrigierten Schritt (5.36) aus

$$\begin{bmatrix} W_k + \Sigma_k + \delta_w I & A_k \\ A_k^T & -\delta_c I \end{bmatrix} \begin{pmatrix} d_k^{x,cor} \\ d_k^\lambda \end{pmatrix} = - \begin{pmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ h_k^{soc} \end{pmatrix} \quad (5.37)$$

zu errechnen. Man wählt nun

$$h_k^{soc} = \alpha_{k,0}h(x_k) + h(x_k + \alpha_{k,0}d_k^x), \quad (5.38)$$

was man aus (5.21), (5.35) und (5.36) erhält.

Hat man die korrigierte Suchrichtung $d_k^{x,cor}$ berechnet, wird erneut die fraction-to-boundary-Regel angewendet

$$\alpha_k^{soc} := \max \{ \alpha \in (0, 1] : x_k + \alpha d_k^{x,cor} \geq (1 - \tau_j)x_k \} \quad (5.39)$$

und überprüft, ob der resultierende Testpunkt $x_k^{soc} := x_k + \alpha d_k^{x,cor}$ vom Filter akzeptiert wird. Besteht der Punkt diese Tests, so wird er zur neuen Iterierten. Ansonsten sind möglicherweise weitere Korrekturen zweiter Ordnung notwendig, es sei denn der Korrekturschritt hat die Nebenbedingungsverletzung nicht mindestens um einen festen Anteil $\kappa_{soc} \in (0, 1)$ verringert oder die maximale Anzahl p^{max} an Korrekturen zweiter Ordnung wurde bereits erreicht. In diesem Fall wird auf die alte Suchrichtung d_k^x zurückgegriffen und die normale Backtracking-Line-Search wird mit einer kürzeren Schrittweite $\alpha_{k,1} = \frac{1}{2}\alpha_{k,0}$ durchgeführt.

Der Algorithmus

Nachdem die wichtigsten Bestandteile des in IPOPT implementierten Algorithmus beleuchtet wurden, wird der Filter-Line-Search-Algorithmus zur Lösung des Barriere-Problems (5.11) formal dargestellt:

Algorithmus 5.2.1 (LINE-SEARCH-FILTER-BARRIEREMETHODE)

- Gegeben:* Startpunkt (x_0, λ_0, z_0) mit $x_0, z_0 > 0$,
 Initialisierungswert für den Barriereparameter $\mu_0 > 0$,
 Konstanten $\epsilon_{tol} > 0$, $s_{max} \geq 1$, $\kappa_\epsilon > 0$, $\kappa_\mu \in (0, 1)$, $\theta_\mu \in (1, 2)$, $\tau_{min} \in (0, 1)$,
 $\kappa_\Sigma > 1$, $\theta_{max} \in (\theta(x_0), \infty]$, $\theta_{min} > 0$, $\gamma_\theta, \gamma_\varphi \in (0, 1)$, $\delta > 0$, $\gamma_\alpha \in (0, 1]$, $s_\theta > 1$,
 $s_\varphi \geq 1$, $\eta_\varphi \in (0, \frac{1}{2})$, $\kappa_{soc} \in (0, 1)$, $p^{max} \in \{0, 1, 2, \dots\}$.
- Schritt 1:* Initialisierung
 Setze $j := 0$, $k := 0$,
 setze $\mathcal{F}_0$ wie in (5.32) angegeben,
 wähle τ_0 entsprechend (5.17).
- Schritt 2:* Konvergenzcheck des Gesamtproblems
 Falls $E_0(x_k, \lambda_k, z_k) \leq \epsilon_{tol}$ (mit Optimalitätsfehler E_0 entsprechend (5.13)):
 STOP [CONVERGED].
- Schritt 3:* Konvergenzcheck des Barriere-Problems
 Falls $E_{\mu_j}(x_k, \lambda_k, z_k) \leq \kappa_\epsilon \mu_j$:
 Schritt 3.1: Berechne μ_{j+1} und τ_{j+1} aus (5.16) und (5.17), setze $j := j + 1$;
 Schritt 3.2: Re-Initialisierung des Filter $\mathcal{F}_k := \{(\theta, \varphi) \in \mathbb{R}^2 : \theta \geq \theta_{max}\}$;
 Schritt 3.3: Falls $k = 0$, wiederhole Schritt 3, ansonsten gehe zu Schritt 4.
- Schritt 4:* Berechnung der Suchrichtung
 Berechne $(d_k^x, d_k^\lambda, d_k^z)$ aus (5.21), wobei man δ_w und δ_c aus Algorithmus IC in [27, Abschnitt 3.1] erhält.
- Schritt 5:* Backtracking-Line-Search
 Schritt 5.1: Initialisierung der Line-Search
 Setze $\alpha_{k,0} := \alpha_k^{max}$ aus (5.25) und $l := 0$.
 Schritt 5.2: Berechnung des neuen Testpunkts
 Setze $x_k(\alpha_{k,l}) := x_k + \alpha_{k,l} d_k^x$.
 Schritt 5.3: Zulässigkeitscheck hinsichtlich des Filters
 Falls $(\theta(x_k(\alpha_{k,l})), \varphi_{\mu_j}(x_k(\alpha_{k,l}))) \in \mathcal{F}_k$, akzeptiere Testschritt nicht und gehe zu Schritt 5.5.
 Schritt 5.4: Überprüfung des hinreichenden Abstiegs hinsichtlich aktueller Iterierten
 Fall 1: $\theta(x_k) \leq \theta^{min}$ und (5.30) sind erfüllt:
 falls (5.31) gilt, akzeptiere den Testschritt $x_{k+1} := x_k(\alpha_{k,l})$ und gehe zu Schritt 6, ansonsten gehe zu Schritt 5.5;
 Fall 2: $\theta(x_k) > \theta^{min}$ oder (5.30) ist nicht erfüllt:
 falls (5.28) gilt, akzeptiere den Testschritt $x_{k+1} := x_k(\alpha_{k,l})$ und gehe zu Schritt 6, ansonsten gehe zu Schritt 5.5.
 Schritt 5.5: Initialisierung der Korrektur zweiter Ordnung
 Falls $l > 0$ oder $\theta(x_{k,0}) < \theta(x_k)$, überspringe (SOC) und gehe zu Schritt 5.10, ansonsten setze $p := 1$ und h_k^{soc} in (5.38);
 initialisiere $\theta_{old}^{soc} := \theta(x_k)$
 Schritt 5.6: Berechnung der Korrektur zweiter Ordnung
 Berechne $d_k^{x,cor}$ und d_k^λ aus (5.37), α_k^{soc} aus (5.39) und setze dann
 $x_k^{soc} := x_k + \alpha_k^{soc} d_k^{x,cor}$.

- Schritt 5.7:* Zulässigkeitscheck hinsichtlich des Filters (in SOC)
 Falls $(\theta(x_k^{\text{soc}}), \varphi_{\mu_j}(x_k^{\text{soc}})) \in \mathcal{F}_k$, akzeptiere Testschrittweite nicht und gehe zu Schritt 5.10.
- Schritt 5.8:* Überprüfung des hinreichenden Abstiegs hinsichtlich aktueller Iterierten (in SOC)
 Fall 1: $\theta(x_k) \leq \theta^{\min}$ und (5.30) ist erfüllt (für $\alpha_{k,0}$):
 falls (5.31) gilt mit „ $x_k(\alpha_{k,l})$ “ ersetzt durch „ x_k^{soc} “, akzeptiere den Testschritt $x_{k+1} := x_k^{\text{soc}}$ und gehe zu Schritt 6, ansonsten gehe zu Schritt 5.5.
 Fall 2: $\theta(x_k) > \theta^{\min}$ oder (5.30) ist nicht erfüllt (für $\alpha_{k,0}$):
 Falls (5.28) gilt mit „ $x_k(\alpha_{k,l})$ “ ersetzt durch „ x_k^{soc} “, akzeptiere den Testschritt $x_{k+1} := x_k(\alpha_{k,l})$ und gehe zu Schritt 6, ansonsten gehe zu Schritt 5.5.
- Schritt 5.9:* Weitere Korrektur zweiter Ordnung
 Falls $p = p^{\max}$ oder $\theta(x_k^{\text{soc}}) > \kappa_{\text{soc}} \theta_{\text{old}}^{\text{soc}}$, brich SOC ab und gehe zu Schritt 5.10,
 ansonsten setze $p := p + 1$, $h_k^{\text{soc}} := \alpha_k^{\text{soc}} h_k^{\text{soc}} + h(x_k^{\text{soc}})$ und $\theta_{\text{old}}^{\text{soc}} := \theta_k^{\text{soc}}$ und gehe zurück zu Schritt 5.6.
- Schritt 5.10:* Bestimmung der neuen Testschrittweite
 Setze $\alpha_{k,l+1} := \frac{1}{2} \alpha_{k,l}$ und $l := l + 1$;
 falls die Testschrittweite zu klein wird, d. h. $\alpha_{k,l} < \alpha_k^{\min}$ mit $\alpha_k^{\min}$ definiert in (5.34), gehe zur Zulässigkeits-Wiederherstellungs-Phase Schritt 9, ansonsten gehe zurück zu Schritt 5.2.
- Schritt 6:* Akzeptieren des Testpunkts
 Setze $\alpha_k := \alpha_{k,l}$ (oder $\alpha_k := \alpha_k^{\text{soc}}$, falls der SOC-Punkt in Schritt 5.8 akzeptiert wurde), aktualisiere die Multiplikatoren λ_{k+1} und z_{k+1} aus (5.23) und (5.24) mit α_k^z aus (5.26),
 falls nötig, wende (5.27) an, um z_{k+1} zu korrigieren.
- Schritt 7:* Erweiterung des Filters falls nötig
 Falls (5.30) und (5.31) für α_k nicht gelten, erweitere den Filter entsprechend (5.33), ansonsten behalte den Filter unverändert bei, $\mathcal{F}_{k+1} := \mathcal{F}_k$.
- Schritt 8:* Fortsetzung mit nächster Iterierten
 Setze $k := k + 1$ und gehe zu Schritt 2.
- Schritt 9:* Zulässigkeits-Wiederherstellungs-Phase
 Erweitere den Filter durch (5.33) und berechne eine neue Iterierte x_{k+1} durch Verringerung des Unzulässigkeitsmaßes $\theta(x)$, so dass x_{k+1} zulässig für den erweiterten Filter ist, d. h. $(\theta(x_{k+1}), \varphi_{\mu_j}(x_{k+1})) \notin \mathcal{F}_{k+1}$,
 gehe zu Schritt 8.

5.2.3 Methode der beweglichen Asymptoten

Die Methode der beweglichen Asymptoten wurde im Jahr 1987 von K. Svanberg eingeführt. Die fehlende Eigenschaft globaler Konvergenz wurde 1993 im Verfahren SCP (sequentielle konvexe Programmierung) durch Hinzufügen einer Line-Search behoben.

Allgemeines Problem und grundsätzliche Vorgehensweise

Ausgegangen wird von einem allgemeinen nichtlinearen Optimierungsproblem ähnlicher Form wie das des (SOP), jedoch werden, um die Beschreibung des Algorithmus nicht unnötig zu erschweren, die Ungleichungsnebenbedingungen nicht mit $g \leq 0$, sondern auch mit $h_j \leq 0$ dargestellt:

$$\begin{aligned} & \text{Minimiere} && f(x), && x \in \mathbb{R}^n \\ & \text{unter} && h_j(x) = 0, && j = 1, \dots, m_{eq}, \\ & && h_j(x) \leq 0, && j = m_{eq} + 1, \dots, M, \\ & && x_L \leq x_i \leq x_U, && i = 1, \dots, n. \end{aligned} \quad (\text{NLP})$$

Die Funktionen f und h_j ($j = 1, \dots, M$) sind dabei innerhalb des zulässigen Bereichs $X := \{x \mid x_L \leq x_i \leq x_U, i = 1, \dots, n\}$ definiert und im Inneren von X als mindestens zweimal stetig differenzierbar vorausgesetzt. Der zulässige Bereich wird als nichtleer angenommen.

Bei konvexer Optimierung wird die Zielfunktion des Problems durch eine konvexe Funktion approximiert, Ungleichungsnebenbedingungen ebenfalls durch konvexe Funktionen und Gleichungsnebenbedingungen durch lineare Funktionen. Der Begriff „sequentiell“ gibt an, dass das (NLP) durch eine Folge von einfacheren Subproblemen ersetzt wird. Jedes der Subprobleme hat die oben beschriebene Form — konvexes Zielfunktional und konvexe Ungleichungsnebenbedingungen sowie lineare Gleichungsnebenbedingungen und eventuell Box-Constraints.

Im Folgenden bezeichnet $I_{j,k}^+$ die Menge der Indizes aller Komponenten mit nicht-negativen ersten partiellen Ableitungen im Punkt x_k ⁵, $I_{j,k}^-$ entsprechend die Menge der Indizes der Komponenten mit negativen ersten partiellen Ableitungen:

$$I_{j,k}^+ := \left\{ i \mid \frac{\partial h_j(x_k)}{\partial x_i} \geq 0 \right\}, \quad I_{j,k}^- := \left\{ i \mid \frac{\partial h_j(x_k)}{\partial x_i} < 0 \right\}.$$

Die Zielfunktion f ($h_0 := f$) wird dargestellt als

$$\begin{aligned} f_k(x) := f(x_k) &+ \sum_{i \in I_{0,k}^+} \left[\frac{\partial f(x_k)}{\partial x_i} \left(\frac{(U_{i,k} - x_{i,k})^2}{U_{i,k} - x_i} - (U_{i,k} - x_{i,k}) \right) + \tau_{i,k} \frac{(x_i - x_{i,k})^2}{U_{i,k} - x_i} \right] \\ &- \sum_{i \in I_{0,k}^-} \left[\frac{\partial f(x_k)}{\partial x_i} \left(\frac{(x_{i,k} - L_{i,k})^2}{x_i - L_{i,k}} - (x_{i,k} - L_{i,k}) \right) - \tau_{i,k} \frac{(x_i - x_{i,k})^2}{x_i - L_{i,k}} \right]. \end{aligned} \quad (5.40)$$

Dabei garantiert der zusätzliche Term, multipliziert mit positivem Parameter $\tau_{i,k}$, ($i = 1, \dots, n$), dass die entstehende Approximation nicht nur konvex, sondern sogar streng konvex ist.

Eine Ungleichungsnebenbedingung h_j ($j = m_{eq} + 1, \dots, M$) wird ersetzt durch

$$\begin{aligned} h_{j,k}(x) := h_j(x_k) &+ \sum_{i \in I_{j,k}^+} \frac{\partial h_j(x_k)}{\partial x_i} \left(\frac{(U_{i,k} - x_{i,k})^2}{U_{i,k} - x_i} - (U_{i,k} - x_{i,k}) \right) \\ &- \sum_{i \in I_{j,k}^-} \frac{\partial h_j(x_k)}{\partial x_i} \left(\frac{(x_{i,k} - L_{i,k})^2}{x_i - L_{i,k}} - (x_{i,k} - L_{i,k}) \right). \end{aligned} \quad (5.41)$$

⁵Der Index k gibt die Iterationsnummer wieder, Index i steht für die i -te Komponente des Vektors. $x_{i,k}$ bezeichnet also die i -te Komponente des Vektors x in der k -ten Iteration.

f_k und $h_{j,k}$ ($j = m_{eq} + 1, \dots, M$) sind auf

$$D_k := \{x \mid L_{i,k} < x_i < U_{i,k}, \quad i = 1, \dots, n\}$$

definiert, $L_{i,k}$ und $U_{i,k}$ stellen Parameter mit der Eigenschaft $L_{i,k} \leq x_{i,k} \leq U_{i,k}$ dar. Sie sind Asymptoten des jeweiligen Teilproblems und variieren in den verschiedenen Iterationen, wovon der Algorithmus seinen Namen hat. Auf die verschiedenen Möglichkeiten der Asymptotenbestimmung wird anschließend eingegangen.

Gleichungsnebenbedingungen h_j ($j = 1, \dots, m_{eq}$) werden durch

$$h_{j,k}(x) := h_j(x_k) + \sum_{i=1}^n \frac{\partial h_j(x_k)}{\partial x_i} (x_i - x_{i,k}). \quad (5.42)$$

approximiert. Bei ihnen werden keine weiteren Restriktionen auf D_k angewendet.

Folglich werden Gleichungsnebenbedingungen in herkömmlicher Weise linearisiert, Ungleichungsnebenbedingungen und das Zielfunktional werden nach transformierten Variablen

$$\frac{1}{x_i - L_{i,k}} \quad (\text{für } i \in I_{j,k}^-) \quad \text{und} \quad \frac{1}{U_{i,k} - x_i} \quad (\text{für } i \in I_{j,k}^+)$$

linearisiert. Da es sich bei f_k und $h_{j,k}$ um Approximationen erster Ordnung handelt, gilt für $j = 1, \dots, M$

$$\begin{aligned} f_k(x_k) &= f(x_k), & h_{j,k}(x_k) &= h_j(x_k), \\ \nabla f_k(x_k) &= \nabla f(x_k), & \nabla h_{j,k}(x_k) &= \nabla h_j(x_k). \end{aligned}$$

Ein Subproblem hat somit folgende Form:

$$\begin{aligned} &\text{Minimiere} && f_k(x), && x \in \mathbb{R}^n \\ &\text{unter} && h_{j,k}(x) = 0, && j = 1, \dots, m_{eq}, \\ &&& h_{j,k}(x) \leq 0, && j = m_{eq} + 1, \dots, M, \\ &&& x'_{i,L} \leq x_i \leq x'_{i,U}, && i = 1, \dots, n, \end{aligned} \quad (NLP_k^{sub})$$

wobei $x'_{i,L} := \max \{x_{i,L}, x_{i,k} - \omega(x_{i,k} - L_{i,k})\}$ und $x'_{i,U} := \min \{x_{i,U}, x_{i,k} + \omega(U_{i,k} - x_{i,k})\}$ mit festem $\omega \in]0, 1[$ ist.

Folglich gilt also $x_{i,L} \leq x'_{i,L} \leq x_{i,k} \leq x'_{i,U} \leq x_{i,U}$, wobei $x_{i,k}$ die i -te Komponente des aktuellen Auswertungspunkts bezeichnet.

Asymptotenwahl

Die Wahl der Asymptoten beeinflusst die Approximationen wesentlich. Näher zusammenliegende Asymptoten haben nicht nur einen kleineren zulässigen Bereich des Ersatzproblems zur Folge, sie erhöhen auch die Krümmung der Approximationen.

Satz 5.2.1 (Zulässige Folge von Asymptoten)

Eine Folge von Asymptoten heißt zulässig, wenn gilt

1. $L_{i,k} \leq x_{i,k} - \xi$, $U_{i,k} \geq x_{i,k} + \xi$, $\forall i = 1, \dots, n$ und k , wobei ξ eine positive Konstante darstellt.
2. $L_{i,k} \geq L_{min}$, $U_{i,k} \leq U_{max}$ $\forall k \geq 0$, L_{min}, U_{max} endlich.

Der erste Punkt des Satzes verhindert, dass die Krümmung der Approximationen nach unendlich geht, der zweite Teil sorgt dafür, dass die Approximation nicht linear oder fast-linear wird, was eigentlich nur für die Zielfunktion notwendig ist.

Zur Festlegung der Asymptoten stehen unter SCIP vier verschiedene Möglichkeiten zur Auswahl:

- Die Standard-Asymptotenwahl in SCIPIP sieht in formaler Schreibweise wie folgt aus:

Algorithmus 5.2.2 (Standard-Asymptotenwahl unter SCIPIP)

$$\begin{aligned}
 k = 0, 1 : \quad & \text{falls } x_{i,L} = -\infty \text{ oder } x_{i,U} = \infty : \\
 & L_{i,k} := x_{i,k} - \max\{1, |x_{i,k}|\}, \\
 & U_{i,k} := x_{i,k} + \max\{1, |x_{i,k}|\}, \\
 & \text{ansonsten:} \\
 & L_{i,k} := x_{i,k} - 0.5(x_{i,U} - x_{i,L}), \\
 & U_{i,k} := x_{i,k} + 0.5(x_{i,U} - x_{i,L}), \\
 k = 2, 3, \dots : \quad & \text{falls } \operatorname{sgn}(x_{i,k} - x_{i,k-1}) = \operatorname{sgn}(x_{i,k-1} - x_{i,k-2}) : \\
 & L_{i,k} := x_{i,k} - 1.15 \cdot (x_{i,k-1} - L_{i,k-1}), \\
 & U_{i,k} := x_{i,k} + 1.15 \cdot (U_{i,k-1} - x_{i,k-1}), \\
 & \text{falls } \operatorname{sgn}(x_{i,k} - x_{i,k-1}) \neq \operatorname{sgn}(x_{i,k-1} - x_{i,k-2}) : \\
 & L_{i,k} := x_{i,k} - 0.7(x_{i,k-1} - L_{i,k-1}), \\
 & U_{i,k} := x_{i,k} + 0.7(U_{i,k-1} - x_{i,k-1}).
 \end{aligned}$$

- Im Jahr 1987 veröffentlichte Svanberg eine einfache Asymptotenwahl, die sinnvoll ist, wenn untere Schranken der Variablen vorhanden sind:

Algorithmus 5.2.3 (Asymptotenwahl nach Svanberg)

$$\begin{aligned}
 k = 0, 1, \dots : \quad & L_{i,k} := 0.7 \cdot x_{i,k}, \\
 & U_{i,k} := \frac{x_{i,k}}{0.7}.
 \end{aligned}$$

- Svanbergs Strategie (5.2.3) wurde in Schenks Diplomarbeit zu einer verbesserten Asymptotenwahl erweitert. Dabei wird ab der zweiten Iteration überprüft, ob Oszillation oder glatte Konvergenz erkennbar ist. Im Falle von Oszillation werden die Schranken enger gesetzt.

Algorithmus 5.2.4 (Asymptotenwahl nach Schenk)

$$\begin{aligned}
 k = 0, 1 : \quad & L_{i,k} := x_{i,L} - 0.1(x_{i,U} - x_{i,L}), \\
 & U_{i,k} := x_{i,U} + 0.1(x_{i,U} - x_{i,L}), \\
 k = 2, 3, \dots : \quad & \text{falls } \operatorname{sgn}(x_{i,k} - x_{i,k-1}) = \operatorname{sgn}(x_{i,k-1} - x_{i,k-2}) \text{ (glatte Konvergenz)} : \\
 & L_{i,k} := x_{i,k} - \frac{x_{i,k-1} - L_{i,k-1}}{0.7}, \\
 & U_{i,k} := x_{i,k} + \frac{U_{i,k-1} - x_{i,k-1}}{0.7}, \\
 & \text{falls } \operatorname{sgn}(x_{i,k} - x_{i,k-1}) \neq \operatorname{sgn}(x_{i,k-1} - x_{i,k-2}) \text{ (Oszillation)} : \\
 & L_{i,k} := x_{i,k} - 0.7(x_{i,k-1} - L_{i,k-1}), \\
 & U_{i,k} := x_{i,k} + 0.7(U_{i,k-1} - x_{i,k-1}).
 \end{aligned}$$

Wichtig bei diesem Verfahren ist, dass „vernünftige“ Schranken der Variablen vorhanden sind. „Vernünftig“ heißt in diesem Zusammenhang, dass die oberen und unteren Schranken von gleicher Größenordnung sind und nicht „sehr kleine“ beziehungsweise „sehr große“ Werte annehmen.

- Existieren für die Variablen untere Schranken größer Null, so kann folgende simple Strategie verwendet werden:

Algorithmus 5.2.5 (Asymptotenwahl zur CONLIN⁶-Approximation)

$$\begin{aligned}
 k = 0, 1, \dots : \quad & L_{i,k} := 0, \\
 & U_{i,k} := \infty.
 \end{aligned}$$

⁶CON^{ve}x LINearisation-Methode nach Fleury

Der Algorithmus

Da die Vorgehensweise der MMA geschildert wurde und auch die Aspekte der Asymptotenwahl untersucht wurden, kann der Algorithmus nun formal dargestellt werden:

Algorithmus 5.2.6 (MMA)

- Schritt 0:* Wähle $x_0 \in X$, $\lambda_{0,j} \geq 0$, $j = 1, \dots, M$;
 berechne $f(x_0)$, $\nabla f(x_0)$, $h_j(x_0)$, $\nabla h_j(x_0)$, $j = 1, \dots, M$;
 setze $k := 0$.
- Schritt 1:* Berechne $L_{i,k}$ und $U_{i,k}$ ($i = 1, \dots, n$) durch eines der Schemata (5.2.2), (5.2.3), (5.2.4), (5.2.5);
 definiere $f_k(x)$, $h_{j,k}(x)$, $j = 1, \dots, M$ gemäß (5.40), (5.41) und (5.42).
- Schritt 2:* Löse (NLP_k^{sub}) ;
 (x_{k+1}, λ_{k+1}) sei die Lösung von (NLP_k^{sub}) , wobei λ_{k+1} dem zugehörigen Lagrange-Multiplikatoren-Vektor entspricht.
- Schritt 3:* Falls $x_{k+1} = x_k$, STOP mit $\begin{pmatrix} x_k \\ \lambda_k \end{pmatrix}$ als Lösung.
- Schritt 4:* Berechne $f(x_{k+1})$, $\nabla f(x_{k+1})$, $h_j(x_{k+1})$, $\nabla h_j(x_{k+1})$, $j = 1, \dots, M$;
 setze $k := k + 1$, gehe zu Schritt 1.

5.2.4 Sequentielle konvexe Optimierung

Allgemeines Problem und grundsätzliche Vorgehensweise

Wählt man die Lösung von (NLP_k^{sub}) als neuen Iterationspunkt, ist der im vorherigen Abschnitt beschriebene Algorithmus (5.2.6) nicht global konvergent. Man müsste dafür an die Modellfunktionen starke Annahmen stellen, die fernab der Realität sind. Eine Möglichkeit, um globale Konvergenz von Algorithmen sicherzustellen, ist es, sogenannte Meritfunktionen einzuführen und hinsichtlich einer solchen Funktion eine Line-Search durchzuführen. Hintergrund dazu ist, dass stationäre Punkte eines Problems wie (NLP) an stationäre Punkte der unrestringierten Meritfunktion gekoppelt sind. Die Abnahme der Meritfunktion stellt somit ein Maß für den Iterationsfortschritt dar.

SCIP verwendet die erweiterte Lagrange-Funktion als Meritfunktion, die weiter unten für diese spezielle Problemklasse definiert ist.

Um jedoch die Notation etwas zu vereinfachen, wird das (NLP) umformuliert, indem Box-Constraints als gewöhnliche Ungleichungsnebenbedingungen behandelt werden:

$$\begin{aligned} &\text{Minimiere} && f(x), && x \in \mathbb{R}^n \\ &\text{unter} && h_j(x) = 0, && j = 1, \dots, m_{eq}, \\ &&& h_j(x) \leq 0, && j = m_{eq} + 1, \dots, m, \end{aligned} \quad (\text{NLP2})$$

wobei $m = M + 2n$.

Definition 5.2.1 (Erweiterte Lagrange-Funktion)

Die erweiterte Lagrange-Funktion $\Phi_\rho : \mathbb{R}^{n+m} \rightarrow \mathbb{R}$ von (NLP2) für feste Parameter $\rho_j > 0$ ($j = 1, \dots, m$) lautet

$$\begin{aligned} \Phi_\rho \begin{pmatrix} x \\ y \end{pmatrix} &= f(x) + \sum_{j=1}^{m_{eq}} \left(\lambda_j h_j(x) + \frac{\rho_j}{2} h_j^2(x) \right) \\ &+ \sum_{j=m_{eq}+1}^m \begin{cases} \lambda_j h_j(x) + \frac{\rho_j}{2} h_j^2(x), & \text{falls } -\frac{\lambda_j}{\rho_j} \leq h_j(x) \\ -\frac{\lambda_j^2}{2\rho_j}, & \text{ansonsten} \end{cases}. \end{aligned} \quad (5.43)$$

Zwei weitere für den Algorithmus SCP notwendige Größen sind

$$\eta_{i,k} := \begin{cases} \left(\frac{\partial f(x_k)}{\partial x_i} + \tau_{i,k} \right) \frac{2U_{i,k} - z_{i,k} - x_{i,k}}{(U_{i,k} - z_{i,k})^2}, & \text{falls } \frac{\partial f(x_k)}{\partial x_i} \geq 0, \\ \left(\tau_{i,k} - \frac{\partial f(x_k)}{\partial x_i} \right) \frac{-2L_{i,k} + z_{i,k} + x_{i,k}}{(z_{i,k} - L_{i,k})^2}, & \text{falls } \frac{\partial f(x_k)}{\partial x_i} \leq 0, \end{cases}$$

mit z_k als Lösung des konvexen Ersatzproblems (NLP_k^{sub}) und

$$\eta_k := \min_{i=1,\dots,n} \eta_{i,k}. \quad (5.44)$$

η_k geht in die Berechnung der „Abstiegsrichtung“ der erweiterten Lagrange-Funktion (siehe Algorithmus(5.2.8), Schritt 6) ein. Wie bei der Line-Search üblich wird jeder Iterationspunkt überprüft, ob er stationär ist. Falls das nicht so ist, wird eine neue Abstiegsrichtung $s_k \in \mathbb{R}^n$ der erweiterten Lagrange-Funktion berechnet. Ob es sich bei s_k tatsächlich um eine Abstiegsrichtung handelt, wird in SCPIP durch folgendes Kriterium, oben als „Abstiegsrichtung“ bezeichnet, entschieden:

$$\nabla \Phi_\rho \begin{pmatrix} x_k \\ y_k \end{pmatrix}^T s_k < -\frac{\eta_k(\delta_k)^2}{2}, \quad (5.45)$$

wobei δ_k der Abstand (die euklidische Norm) der beiden letzten Iterierten ist.

Ein weiterer wichtiger Bestandteil des SCP-Algorithmus ist die Berechnung der Änderung des Penalty-Parameters ρ_j für jede Nebenbedingung h_j , falls die Abstiegsbedingung (5.45) nicht erfüllt ist. Dazu seien folgende Mengen definiert:

$$J := \{j \mid 1 \leq j \leq m_{eq}\} \cup \left\{ j \mid m_{eq} + 1 \leq j \leq m; -\frac{\lambda_j}{\rho_j} \leq h_j(x) \right\}, \quad K := \{1, \dots, m\} \setminus J.$$

Der Algorithmus zum Update der Penalty-Parameter lautet dann wie folgt:

Algorithmus 5.2.7 (Update der Penalty-Parameter)

Sei $\kappa_1 > 1$ const, $\kappa_2 > \kappa_1$ const.

$j \in J$: falls $(h_j(x_k) > 0 \text{ und } \nabla h_j(x_k)^T(z_k - x_k) \neq 0) \text{ oder}$
 $(h_j(x_k) < 0 \text{ und } \nabla h_j(x_k)^T(z_k - x_k) > 0)$:

$$\rho_j := \min \left\{ \kappa_2 \rho_j, \max \left\{ \kappa_1 \rho_j, \left| \frac{2(v_j - \lambda_j)}{h_j(x_k)} \right| \right\} \right\},$$

sonst: $\rho_j := \kappa_1 \rho_j$,

$j \in K$: falls $v_j - \lambda_j < 0$:

$$\rho_j := \min \left\{ \kappa_2 \rho_j, \max \left\{ \kappa_1 \rho_j, \left| \frac{\lambda_j(v_j - \lambda_j)4m}{\eta_k(\delta_k)^2} \right| \right\} \right\},$$

sonst: $\rho_j := \kappa_1 \rho_j$.

κ_1 unterbindet, dass der Penalty-Parameter langsam gegen eine obere Grenze konvergiert, κ_2 verhindert, dass er zu schnell große Werte annimmt. v entspricht dem zu z gehörigen Lagrange-Multiplikator der erweiterten Lagrange-Funktion.

Der Algorithmus

Im vorherigen Abschnitt wurden die Mittel für den SCP-Algorithmus bereitgestellt. Allerdings wurde nicht darauf eingegangen, wie die konvexen Ersatzprobleme gelöst werden und wie beispielsweise die Barrieremethode eingesetzt wird. Dafür sei auf [30, Abschnitt 3] verwiesen. Auf ein weiteres explizites Ausführen der Details wird in dieser Arbeit verzichtet.

Algorithmus 5.2.8 (SCP)

- Schritt 0:* Wähle $x_0 \in X$, $\lambda_{j,0} \in \mathbb{R}$, $j = 1, \dots, m_{eq}$, $\lambda_{j,0} \geq 0$, $j = m_{eq}+1, \dots, m$, $0 < c < 1$;
 berechne $f(x_0)$, $\nabla f(x_0)$, $h_j(x_0)$, $\nabla h_j(x_0)$, $j = 1, \dots, M$;
 setze $k := 0$.
- Schritt 1:* Berechne $L_{i,k}$ und $U_{i,k}$ ($i = 1, \dots, n$) durch eines der Schemata (5.2.2), (5.2.3), (5.2.4), (5.2.5);
 definiere $f_k(x)$, $h_{j,k}(x)$, $j = 1, \dots, M$ gemäß (5.40), (5.41) und (5.42)
- Schritt 2:* Löse (NLP_k^{sub}) ;
 (z_k, v_k) sei die Lösung von (NLP_k^{sub}) , wobei v_k dem zugehörigen Lagrange-Multiplikatoren-Vektor entspricht.
- Schritt 3:* Falls $z_k = x_k$, STOP mit $\begin{pmatrix} x_k \\ \lambda_k \end{pmatrix}$ als Lösung.
- Schritt 4:* Setze $p_k := \begin{pmatrix} z_k - x_k \\ v_k - \lambda_k \end{pmatrix}$, $\delta_k := \|z_k - x_k\|$, η_k wie in (5.44) definiert.
- Schritt 5:* Berechne $\Phi_\rho \begin{pmatrix} x_k \\ \lambda_k \end{pmatrix}$, $\nabla \Phi_\rho \begin{pmatrix} x_k \\ \lambda_k \end{pmatrix}$, $\nabla \Phi_\rho \begin{pmatrix} x_k \\ \lambda_k \end{pmatrix}^T p_k$.
- Schritt 6:* Falls $\nabla \Phi_\rho \begin{pmatrix} x_k \\ \lambda_k \end{pmatrix}^T p_k > -\frac{\eta_k(\delta_k)^2}{2}$, berechne ρ_j ($j = 1, \dots, m$) entsprechend Algorithmus (5.2.7) und gehe zu Schritt 5;
 ansonsten setze $\sigma := 1$.
- Schritt 7:* Berechne $f(x_k + \sigma(z_k - x_k))$, $h_j(x_k + \sigma(z_k - x_k))$, $j = 1, \dots, M$;
 berechne $\Phi_\rho \left(\begin{pmatrix} x_k \\ \lambda_k \end{pmatrix} + \sigma p_k \right)$.
- Schritt 8:* Armijo-Bedingung
 Falls $\Phi_\rho \begin{pmatrix} x_k \\ \lambda_k \end{pmatrix} - \Phi_\rho \left(\begin{pmatrix} x_k \\ \lambda_k \end{pmatrix} + \sigma p_k \right) < -c\sigma \nabla \Phi_\rho \begin{pmatrix} x_k \\ \lambda_k \end{pmatrix}^T p_k$, setze $\sigma := \sigma \cdot \psi$
 und gehe zu Schritt 7;
 ansonsten setze $\sigma_k := \sigma$.
- Schritt 9:* Setze $\begin{pmatrix} x_{k+1} \\ \lambda_{k+1} \end{pmatrix} := \begin{pmatrix} x_k \\ \lambda_k \end{pmatrix} + \sigma_k p_k$, $k := k + 1$.
- Schritt 10:* Berechne $\nabla f(x_k)$, $\nabla h_j(x_k)$, $j = 1, \dots, m$;
 gehe zu Schritt 1.

6 Implementierung

In diesem Kapitel wird die Implementierung des Löser für Optimalsteuerungsprobleme mit parabolischen Differentialgleichungen beschrieben. Die konkrete Umsetzung der erläuterten Methoden durch Programmierung in C++ wird hier dargestellt.

Die C++-Klassenbibliothek DIFFPACK stellt gewissermaßen die Umgebung für den Löser. Sie wird zur Lösung der auftretenden PDEs eingesetzt. Angaben, die der Benutzer macht bzw. die durch Files vorgegeben sind, werden durch sie eingelesen und aus ihr werden die Optimierer IPOPT und SCIP aufgerufen. Da SCIP in Fortran77 implementiert ist, wird eine Wrapper-Klasse verwendet, durch die der Fortran-Code aus der C++-Umgebung angesprochen wird. Zunächst wird aber auf zwei mathematische Verfahren eingegangen, die bei der Implementierung benötigt werden. Neben dem bereits angesprochenen Newton-Raphson-Verfahren, mit dem nichtlineare Gleichungssysteme gelöst werden können, wird eine Methode vorgestellt, mit der sich relativ leicht Gradienten bestimmen lassen. Die zur Zeitdiskretisierung verwendeten Finiten Differenzen kommen hier wieder ins Spiel und es wird in diesem Kapitel ausführlicher auf die Methode, Vorteile und Problematiken eingegangen.

Die Informationen zu DIFFPACK stammen zum größten Teil aus [17] und anderen Diplomarbeiten, in denen DIFFPACK bereits erfolgreich zur Anwendung kam [11, 15, 21, 22]. Auch das Newton-Raphson-Verfahren ist ausführlich in [17] erläutert. Der Hintergrund zur Gradientenbestimmung mittels Finiten Differenzen ist in „Elements of Structural Optimization“ von Haftka und Gürdal [13] beschrieben. Wie man IPOPT mittels C++-Funktionen aufrufen kann, ist der Dokumentation zu IPOPT auf der Webseite [3] zu entnehmen. Der SCIP-Wrapper wurde dankenswerterweise von der inuTech GmbH zur Verfügung gestellt, weitere Informationen, wie Einstellungen für den Optimierer vorzunehmen sind, finden sich in der Dokumentation von Zillober [30].

6.1 Bei der Implementierung eingesetzte mathematische Verfahren

Finite Differenzen zur Berechnung erster Ableitungen und das Newton-Raphson-Verfahren zur Lösung nichtlinearer Gleichungssysteme stellen wichtige Hilfsmittel bei der Implementierung dieser direkten Methode dar und werden nun präsentiert.

6.1.1 Gradientenbestimmung mittels Finiten Differenzen

Bei der Finite-Differenzen-Approximation handelt es sich um eine Technik, Ableitungen nach (Design-)Variablen zu berechnen. Sie zieht zwar relativ lange Laufzeiten der Algorithmen nach sich, ist aber leicht zu implementieren und sehr flexibel einsetzbar. In dieser Arbeit wird dieses Verfahren verwendet, Alternativen werden im Ausblick aufgezeigt.

Vorwärts-, Rückwärts- und zentrale Differenzen

Es sei eine Funktion $y(x)$ gegeben und sie hänge von einer eindimensionalen Variablen x ab. Für die Ableitung $\frac{d}{dx}y(x)$ gilt bekanntlich:

$$\frac{d}{dx}y(x) = \lim_{\Delta x \rightarrow 0} \frac{y(x + \Delta x) - y(x)}{\Delta x}.$$

Um in numerischen Verfahren diese Ableitung annähern zu können, verwendet man Finite-Differenzen-Approximationen. Die einfachste Art dieser Klasse ist die *Vorwärtsdifferenzen-Approximation erster Ordnung*. Mit ihrer Hilfe lässt sich obige Ableitung wie folgt approximieren:

$$\frac{d}{dx}y(x) \approx \frac{y(x + \Delta x) - y(x)}{\Delta x}, \quad (6.1)$$

wobei Δx ein kleiner Wert ist ($0 \leq \Delta x < 1$).

Eine weitere Approximation der ersten Ableitung ist die sogenannte *Rückwärtsdifferenzen-Approximation erster Ordnung*:

$$\frac{d}{dx}y(x) \approx \frac{y(x) - y(x - \Delta x)}{\Delta x}. \quad (6.2)$$

Die *Zentrale Differenzen-Approximation zweiter Ordnung* stellt eine um einen Grad höhere Annäherung dar:

$$\frac{d}{dx}y(x) \approx \frac{y(x + \Delta x) - y(x - \Delta x)}{2\Delta x}. \quad (6.3)$$

Natürlich lassen sich zur Bestimmung der Ableitungen auch Finite-Differenzen-Approximationen höherer Ordnung verwenden. Sie spielen jedoch in dieser Arbeit keine Rolle und wurden nicht implementiert.

Benötigt man die Ableitung einer Funktion nach n Variablen, so sind also bei den Vorwärts- und Rückwärtsdifferenzen je n zusätzliche Auswertungen, bei den zentralen Differenzen sogar $2n$ weitere Auswertungen von y notwendig. Approximationen höherer Ordnung sind natürlich noch „teurer“.

Herleitung Finiter-Differenzen-Verfahren

Grundlage für die Herleitung der Finite-Differenzen-Approximation ist die Taylorentwicklung der Funktion y . Mit Hilfe der Taylorreihe lässt sich bekanntlich der Wert der Funktion $y(x)$ an den Stellen $x + \Delta x$ und $x - \Delta x$ berechnen, wenn die Ableitungen an der Stelle x bekannt sind:

$$y(x + \Delta x) = y(x) + \frac{\Delta x}{1!}y'(x) + \frac{(\Delta x)^2}{2!}y''(x) + \dots \quad (6.4)$$

$$y(x - \Delta x) = y(x) - \frac{\Delta x}{1!}y'(x) + \frac{(\Delta x)^2}{2!}y''(x) - \dots \quad (6.5)$$

Durch Auflösen von (6.4) erhält man für die erste Ableitung von y nach x folgende Formel:

$$y'(x) = \frac{y(x + \Delta x) - y(x)}{\Delta x} + \left(-\frac{\Delta x}{2!}y''(x) - \frac{\Delta x^2}{3!}y'''(x) - \dots \right). \quad (6.6)$$

Fasst man die Glieder mit zweiten und höheren Ableitungen als Rest $\mathcal{O}(\Delta x)$ zusammen, so hat man bereits die Formel für die Vorwärtsdifferenzen-Approximation erster Ordnung:

$$y'(x) = \frac{y(x + \Delta x) - y(x)}{\Delta x} + \mathcal{O}(\Delta x). \quad (6.7)$$

Maßgeblich für die Größe $\mathcal{O}(\Delta x)$ ist die Schrittweite Δx und deren Potenz im dominanten Term des Rests, die man als *lokale Fehlerordnung* $\mathcal{O}_{lok}$ bezeichnet¹. Man sagt deshalb, dass das Verfahren die lokale Fehlerordnung $\mathcal{O}_{lok}(\Delta x)$ hat.

¹Der dominante Teil des Rests von (6.6) ist übrigens $\frac{\Delta x}{2!}y''(x)$.

Die lokale Fehlerordnung des Rückwärtsdifferenzen-Verfahrens ist natürlich betragsmäßig die gleiche. Die Herleitung der Formel erfolgt analog, ausgehend von (6.5).

Die Fehlerordnung der zentralen Differenzen-Approximation ist, wie bereits erwähnt, um einen Grad höher. Die Formel erhält man, indem man die Differenz von (6.4) und (6.5) bildet und nach y' auflöst:

$$y'(x) = \frac{y(x + \Delta x) - y(x - \Delta x)}{2\Delta x} + \mathcal{O}(\Delta x^2). \quad (6.8)$$

Fehlerbestimmung

Im Zusammenhang mit Finiten Differenzen unterscheidet man zwischen zwei verschiedenen Fehlerarten, nämlich *Rundungs-* und *Konditionierungsfehlern*.

Rundungsfehler sind die in obiger Herleitung beschriebenen Fehler, die aus dem Weglassen von Termen der Taylorentwicklung resultieren. Beispielsweise beträgt der Rundungsfehler $e_R(\Delta x)$ der Vorwärtsdifferenzen-Approximation an einer Zwischenstelle ζ

$$e_R(\Delta x) = \frac{\Delta x}{2} y''(x + \zeta \Delta x). \quad (6.9)$$

Konditionierungsfehler hingegen definiert man als Differenz zwischen der numerisch ausgewerteten Funktion und dem exakten Wert. Numerische Fehler in der Berechnung von $\frac{dy}{dx}$ am Originalwert x und am perturbierten Wert $x + \Delta x$, eingesetzt in y , tragen zu ihm bei. Bei den meisten Berechnungen² ist dieser Fehler gering, außer Δx ist extrem klein. Wird $y(x)$ jedoch durch einen übermäßig langen und schlecht konditionierten Prozess berechnet, kann der Beitrag des Rundungsfehlers erheblich sein.

Hat man durch ϵ_y eine obere Schranke für den absoluten Fehler in der berechneten Funktion y , so lässt sich auch der Konditionierungsfehler abschätzen. Der Konditionierungsfehler $e_K(\Delta x)$ bei den Vorwärtsdifferenzen erster Ordnung lässt sich durch

$$e_K(\Delta x) = \frac{2}{\Delta x} \epsilon_y$$

angeben. Diese Formel zusammen mit (6.9) führt zum sogenannten *Schrittweiten-Dilemma*. Wählt man die Schrittweite extrem klein, um Rundungsfehler zu verringern, führt das möglicherweise zu übermäßig großen Konditionierungsfehlern. Aus diesem Grund gibt es Verfahren zur Berechnung der optimalen Schrittweite. Sie liefern hinsichtlich der beiden Fehlerarten bessere Ergebnisse, einher geht aber auch ein wachsender numerischer Aufwand.

6.1.2 Behandlung von Nichtlinearitäten mit dem Newton-Raphson-Verfahren

Das Newton-Raphson-Verfahren ist die Standardmethode zur Lösung nichtlinearer Gleichungen und Gleichungssysteme. Eine ausführliche Beschreibung des Verfahrens ist in [17, Kapitel 4.1] zu finden.

Durch Diskretisierung nichtlinearer PDEs bzw. linearer PDEs mit nichtlinearen Randbedingungen mittels Finiten Differenzen oder Finiten Elemente erhält man Systeme nichtlinearer algebraischer Gleichungen. Diese werden durch Folgen linearer Systeme gelöst. Eine weitere iterative Methode, die zu diesem Zweck eingesetzt wird, ist die *Picard-Iteration* (oder *Successive Substitutions*). Für diese sei auf [17] verwiesen, das Newton-Raphson-Verfahren wird nun dargestellt.

Ausgegangen wird von einem nichtlinearen System

$$F(y) = 0,$$

²insbesondere bei Computern mit hoher Maschinengenauigkeit

das beispielsweise durch die FEM, wie in (4.2.2, Unterabschnitt Variationsformulierung und Galerkin-Ansatz) erläutert, erhalten wurde. Zusätzlich sei eine Approximation y^k von y bekannt. Das Ziel ist es nun, eine verbesserte Approximation der Lösung y des nichtlinearen Systems zu berechnen. Die Idee des Newton-Raphson-Verfahrens besteht darin, $F(y)$ in der Umgebung von y^k zu approximieren ($F(y) \approx M(y; y^k)$), so dass das System $M(y; y^k) = 0$ eine Gleichung bzw. ein Gleichungssystem darstellt, das leicht zu lösen ist. Die Lösung dieses Systems ist dann die verbesserte Approximation y^{k+1} der Lösung des ursprünglichen Systems $F(y) = 0$. Falls kein Abbruchkriterium wie beispielsweise $\|y^{k+1} - y^k\| \leq \epsilon_y$ mit vorgegebener Toleranz ϵ_y erfüllt ist, nimmt y^{k+1} die Bedeutung von y^k ein und ist neue Ausgangsapproximierende der Lösung für einen weiteren Iterationsschritt.

Das System $F(y) = 0$ wird in der Umgebung von y^k durch eine lineare Funktion $M(y; y^k)$ approximiert:

$$M(y; y^k) = F(y^k) + J(y^k) \cdot (y - y^k),$$

wobei $J \equiv \nabla F$ die Jacobimatrix von F darstellt. Sei $F = (F_1, \dots, F_n)^T$ und $y = (y_1, \dots, y_n)^T$, so entspricht der Eintrag (i, j) von J dem Ausdruck $\frac{\partial F_i}{\partial y_j}$. Um eine weitere Approximierende y^{k+1} aus $M(y^{k+1}; y)$ zu finden, muss somit ein lineares System mit J als Koeffizientenmatrix gelöst werden.

Zusammengefasst ergibt sich folgender Algorithmus:

Algorithmus 6.1.1 (Newton-Raphson-Verfahren)

Gegeben sei eine Schätzung y^0 für die Lösung von $F(y) = 0$.

Für $k = 0, 1, 2, \dots$, solange kein Abbruchkriterium erfüllt ist,

löse das lineare System $J(y^k)\delta y^{k+1} = -F(y^k)$ nach δy^{k+1} und

setze $y^{k+1} = y^k + \delta y^{k+1}$.

Weitere mögliche Abbruchkriterien, neben dem oben genannten, sind:

- $\|F(y^{k+1})\| \leq \epsilon_r$ mit vorgegebener Toleranz ϵ_r ,
- $\frac{\|y^{k+1} - y^k\|}{\|y^k\|} \leq \epsilon_u$,
- $\frac{\|F(y^{k+1})\|}{\|F(y^0)\|} \leq \epsilon_r$.

6.2 DIFFPACK

DIFFPACK ist ein sehr umfangreiches, objektorientiertes Softwaresystem zur Entwicklung numerischer Programme. Es zeichnet sich durch hohe Flexibilität und Effizienz aus und kombiniert die Vorteile objektorientierter Programmierung mit performanter Numerik. Ein besonderer Schwerpunkt ist auf Finite-Elemente-Modellierung und -Berechnung gelegt, weshalb es sich sehr gut zur Lösung partieller Differentialgleichungen eignet. Mit seinen über 600 C++-Klassen stellt es Hilfsmittel zur Gittergenerierung (sogar mit adaptiven Gittern), zur effektiven Lösung linearer Gleichungssysteme durch iterative Löser, zum Lösen nichtlinearer Gleichungssysteme, zur Dokumentation der Ergebnisse, etc. bereit. Die Bedienung des Programms erfordert Hintergrundwissen in objektorientierter Programmierung mit C++ und Einblick in die mathematische Modellierung des entsprechenden Problems.

Im Jahre 1991 begann die Entwicklung von DIFFPACK unter der Leitung von Hans Petter Langtangen und Are Magnus Bruaset an der Universität Oslo. Hans Petter Langtangen war es auch, der das Buch „Computational Partial Differential Equations“ schrieb, das als Referenz

für DIFFPACK-Anwender gilt. Seit dem Jahr 2003 liegen Vertrieb, Wartung und Weiterentwicklung von DIFFPACK komplett in der Hand der inuTech GmbH in Nürnberg. Zahlreiche namhafte Unternehmen wie DaimlerChrysler, die Robert Bosch GmbH oder Siemens setzen DIFFPACK ein, aber auch Forschungszentren wie die Universitäten Cambridge und Virginia Tech nutzen die Bibliothek. Weitere Informationen zu DIFFPACK sind auf der Webseite [1] zu finden.

6.2.1 Prinzipieller Aufbau eines DIFFPACK-Simulators

Dieser Abschnitt liefert einen Überblick, wie ein DIFFPACK-Solver zur Lösung einer PDE grundsätzlich aufgebaut ist. In den Abbildungen 6.1 und 6.3 sind die einzelnen Lösungsschritte eines DIFFPACK-Solvers für lineare und nichtlineare Programme dargestellt. Im Anhang (A.2) sind die für diese Arbeit wichtigsten Datentypen, Klassen und Funktionen aufgeführt. Im Text wird nicht auf die Bedeutung jeder einzelnen Klasse eingegangen, sondern es wird empfohlen, diese im Anhang oder unter [1] nachzulesen.

Eine Klasse für einen PDE-Löser wird von der **FEM**-Klasse abgeleitet, die die wichtigsten Funktionen und Datenstrukturen für den Einsatz der Finiten-Element-Methode bereitstellt. Einige Funktionen sind in der **FEM**-Klasse als **virtual** deklariert und müssen in der abgeleiteten Klasse reimplementiert werden. Dazu gehören die beiden wichtigen Funktionen **integrands()** und **integrands4side()**, auf deren Bedeutung später eingegangen wird.

Allgemeines Vorgehen in DIFFPACK

Die **main()**-Funktion bei Verwendung von DIFFPACK hat standardmäßig folgende Form:

```
int main(int arg, const char * argv[])
{
    initDiffpack(argc, argv);
    global_menu.init("new simulator", "Solver");
    Solver problem;
    global_menu.multipleLoop(problem);
    return 0;
}
```

Zu Beginn des Programms muss, um DIFFPACK verwenden zu können, **initDiffpack()** stehen. Direkt im Anschluss wird das globale Menü in Form des Objekts **global_menu** initialisiert und ein Objekt (**problem**) der Klasse des Löser, hier mit **Solver** bezeichnet, angelegt. Den weiteren Ablauf bestimmt die Methode **multipleLoop()**, der das **Solver**-Objekt **problem** übergeben wird (grafisch dargestellt in Abbildung 6.1).

Durch die Methode **multipleLoop()** werden nacheinander folgende virtuelle Funktionen aufgerufen, die im neuen Löser zu reimplementieren sind:

- **adm(MenuSystem &menu):**
Die von der Klasse **SimCase** reimplementierte Methode verwaltet die Funktionen **define()** und **scan()**, um Benutzerdaten einzulesen.
- **define(MenuSystem &menu, int level):**
Menüeinträge wie beispielsweise **LinEqAdm**, **initial control**, etc. werden definiert.
- **scan():**
Nachdem die Menüeinträge durch eine Funktion **prompt()** in den Zwischenspeicher eingelesen wurden, werden die Daten im Programm verarbeitet. Die **scan()**-Funktion wird auch zur Initialisierung einiger Membervariablen genutzt.

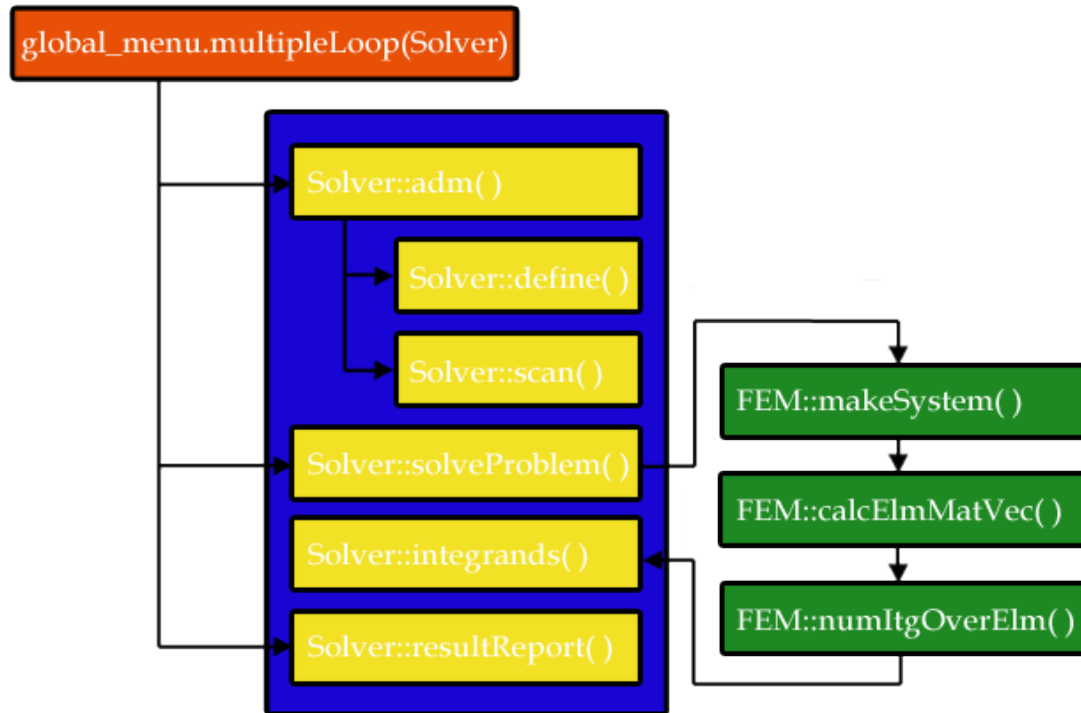


Abbildung 6.1: DIFFPACK-Programmablauf bei linearen Problemen

- `solveProblem()`:

Ebenfalls von `SimCase` reimplementiert, löst diese Funktion das vorgegebene Problem mit festgelegten Menüeinträgen. Sie ist gewissermaßen für die Steuerung des Funktionsaufrufs zur Lösung des Problems verantwortlich. Neben `FEM::makeSystem()` zur Erstellung des linearen Systems ruft sie `LinEqAdmFe::solve()` zur Lösung des Systems auf. `FEM::makeSystem()` ist für die Erstellung des linearen Systems verantwortlich und ruft die Funktionen `Solver::calcElmMatVec()`, `FEM::calcElmMatVec()` und `FEM::numItgOverElm()` auf. Sie steuern die Berechnung der Elementmatrix und des -vektors. Die schwachen Gleichungen sind in `Solver::integrands()` implementiert.

- `resultReport()`:

Die Ergebnisse des Solvers zum Problem können mit Hilfe dieser Methode dokumentiert werden. Außerdem ermöglicht sie es, Daten, die es erlauben, die Resultate zu visualisieren, in Files zu schreiben oder auszugeben.

Weitere für einen Löser zu implementierende Methoden sind:

- `calcElmMatVec(int e, ElmMatVec& elmat, FiniteElement& fe)`:

Sie berechnet die Elementarmatrix und die rechte Seite des linearen Gleichungssystems. Allerdings muss sie neu implementiert werden, wenn Randintegrale berechnet werden müssen. Liegen keine oder nur wesentliche Randbedingungen vor, so kann die von der Basisklasse bereitgestellte Funktionalität verwendet werden.

- `integrands(ElmMatVec& elmat, const FiniteElement& fe)` und `integrands4side(int side, int boind, ElmMatVec& elmat, const FiniteElement& fe)`:

Diese beiden Funktionen bilden das Herzstück eines Lölers bei der Programmierung mit DIFFPACK. Sie werden von `makeSystem()` aus der Basisklasse und `calcElmMatVec()`

aufgerufen, um die in der schwachen Formulierung auftretenden Integrale numerisch zu berechnen. Für die Integration über das Gebiet Ω zeichnet sich die Funktion `integrands()` verantwortlich, für Randintegrale über Γ ist `integrands4side()` zuständig.

Bei zeitabhängigen Problemen benötigt man eine Zeitschleife über die diskreten Zeitpunkte, an denen das System ausgewertet wird. `solveProblem()` ruft dazu direkt die Methode `timeLoop()` auf:

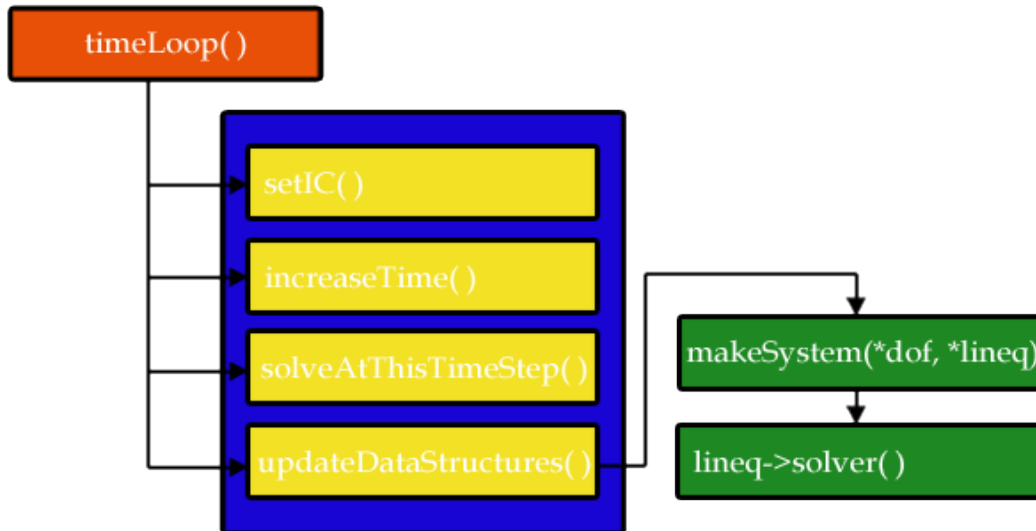


Abbildung 6.2: Zeitschleife zum Lösen einer zeitabhängigen linearen PDE

Es sind deshalb folgende weitere Funktionen zu implementieren:

- `timeLoop()`:
Diese Methode ist für das Simulieren der Zeitschleife verantwortlich. Nach Setzen der Anfangsbedingungen durch `setIC()` inkrementiert sie den aktuellen Zeitschritt durch die Funktion (`increaseTime()`) und ruft `solveAtThisTimeStep()` auf. Die Methode `updateDataStructures()` aktualisiert anschließend die Datenstrukturen für den nächsten Zeitschritt.
- `setIC()`:
Diese Funktion dient zum Setzen von Anfangsbedingungen für den Zeitpunkt $t = 0$.
- `solveAtThisTimeStep()`:
Durch die Rothe-Methode wird, wie in Abschnitt (4) beschrieben, eine parabolische PDE in eine Folge von elliptischen PDEs umgewandelt. Zu jedem Zeitpunkt muss eine elliptische PDE gelöst werden. Die Funktion `solveAtThisTimeStep()` wird von `solveProblem()` aufgerufen und, wie der Name der Methode nahelegt, die Lösung der PDE zu einem diskreten Zeitpunkt berechnet. Dazu ist es natürlich wieder notwendig, andere Funktionen wie `makeSystem()`, `calcElmMatVec()`, etc. aufzurufen.
- `updateDataStructures()`:
Die Datenstrukturen werden für den nächsten Zeitschritt angepasst.

Besonderheiten bei nichtlinearen Problemen

Grundsätzlich ist der Aufbau eines Löser für ein nichtlineares Problem der gleiche wie der für ein lineares Problem. Nichtlineare Gleichungen bzw. Gleichungssysteme werden, wie in (6.1.2)

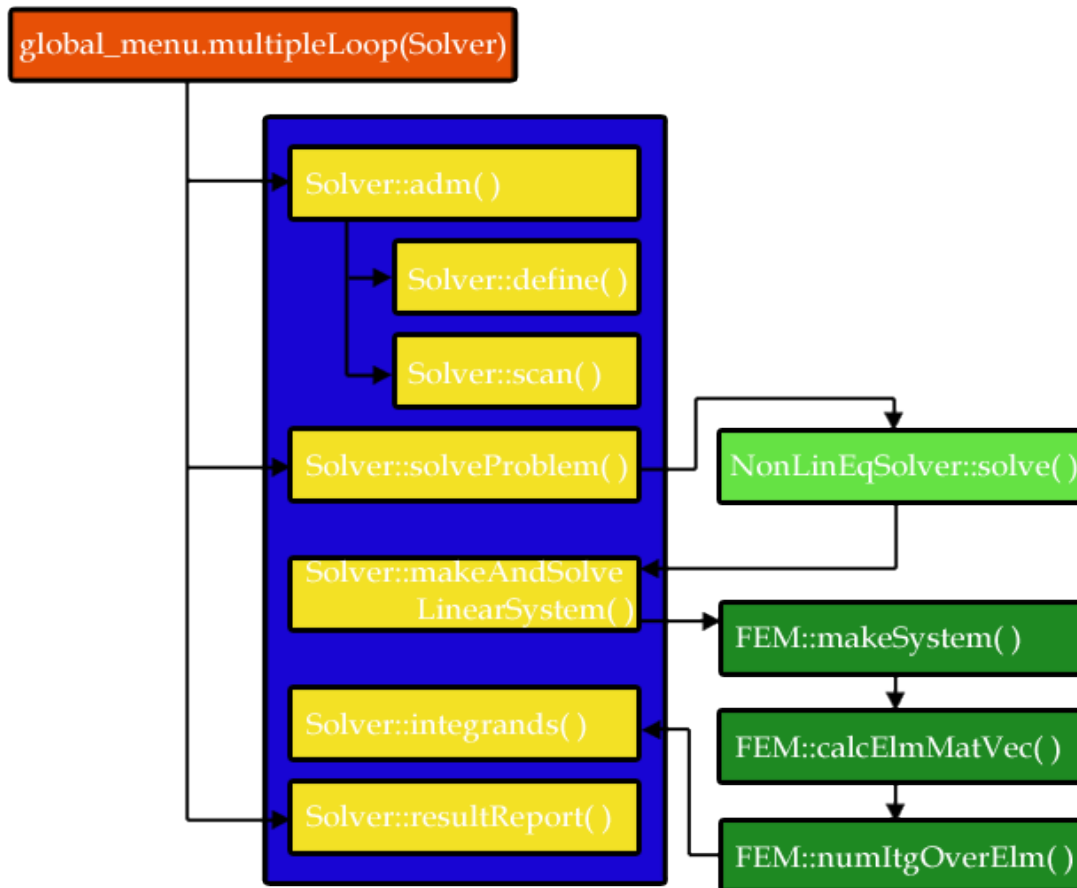


Abbildung 6.3: DIFFPACK-Programmablauf bei nichtlinearen Problemen

beschrieben, iterativ durch eine Folge von linearen Gleichungen bzw. Gleichungssystemen gelöst. Der Ablauf bis zur Funktion `Solver::solveProblem()` ist identisch zum linearen Fall. Der Unterschied besteht darin, dass der Fortgang des Verfahrens von der Funktion `solve()` der Klasse `NonLinEqSolver` gesteuert wird. Sie ruft `Solver::makeAndSolveLinearSystem()` auf, die mit Hilfe der Funktionen `makeSystem()`, `calcElmMatVec()`, `numItgOverElm()` und `integrands()` das zur aktuellen Iterierten gehörende lineare Gleichungssystem erstellt und löst. Diese Lösung wird dann vom nichtlinearen Löser-Objekt der Klasse `NonLinEqSolver` überprüft, ob sie das Abbruchkriterium erfüllt. Ist das der Fall, so wurde eine Lösung des nichtlinearen Problems gefunden, ansonsten ist die Lösung Ausgangspunkt eines neuen Iterationsschritts und es wird erneut `makeAndSolveLinearSystem()` aufgerufen.

Bei zeitabhängigen Problemen wird innerhalb der `timeLoop()` für jeden Zeitschritt der nichtlineare Löser der Klasse `NonLinEqSolver` durch die Funktion `solveAtThisTimeStep()` aufgerufen. In jedem Zeitschritt wird also ein nichtlineares System mit Hilfe des Newton-Raphson-Verfahrens in der im vorherigen Absatz beschriebenen Weise berechnet. Als Startwert für einen Iterationsschritt bietet es sich an, die Lösung des nichtlinearen Problems im vorhergehenden Zeitschritt zu wählen. In Abbildung 6.4 ist der Ablauf innerhalb der `timeLoop()` im nichtlinearen Fall zu sehen.

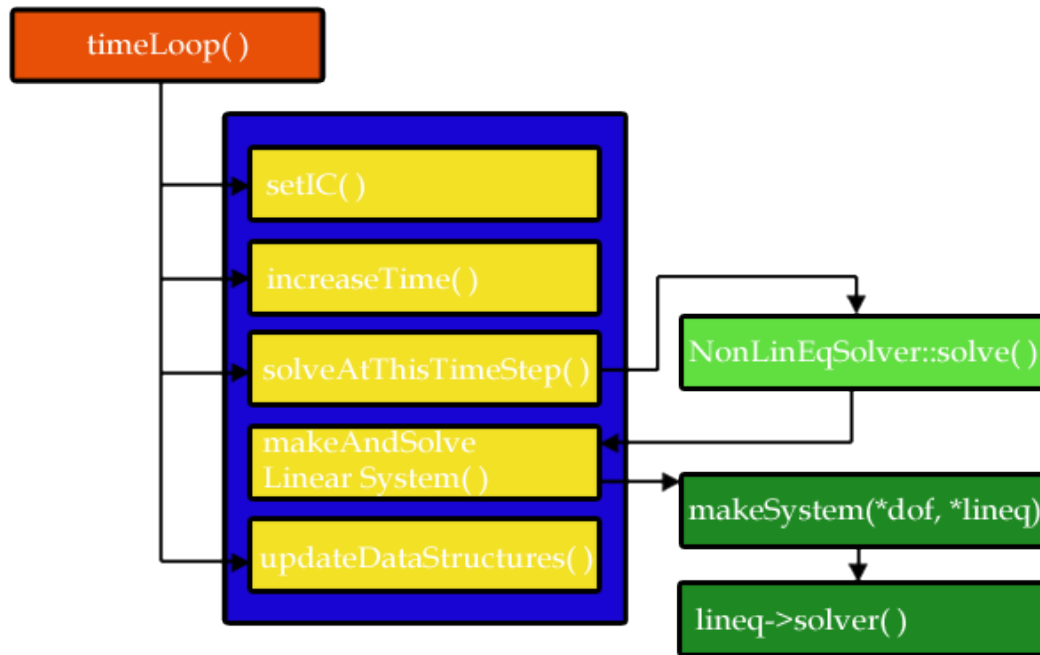


Abbildung 6.4: Zeitschleife zum Lösen einer zeitabhängigen nichtlinearen PDE

6.2.2 Konkrete Umsetzung des Lölers zur Optimalsteuerung in DIFFPACK

Um verschiedene Arten von Optimalsteuerungsproblemen mit parabolischen Differentialgleichungsnebenbedingungen mittels DIFFPACK lösen zu können, wurden für den Löser mehrere Klassen implementiert, bei denen der jeweilige Name darauf schließen lässt, welche Probleme sich mit ihr lösen lassen. Die Grundlage der Programmierung ist durch die Basisklasse `DOCPSolver` geschaffen. Sie enthält die Datenelemente und Methoden, die für alle Klassen gleich sind sowie einige als virtuell deklarierte Methoden, die in den abgeleiteten Klassen implementiert sind. Dieses Konzept ermöglicht es, ausgehend von der Basisklasse, relativ schnell weitere Löser zu implementieren, die neue Probleme, beispielsweise in drei oder mehr Raumdimensionen, lösen können. Folgendes Klassendiagramm zeigt die Hierarchie der implementierten Klassen:

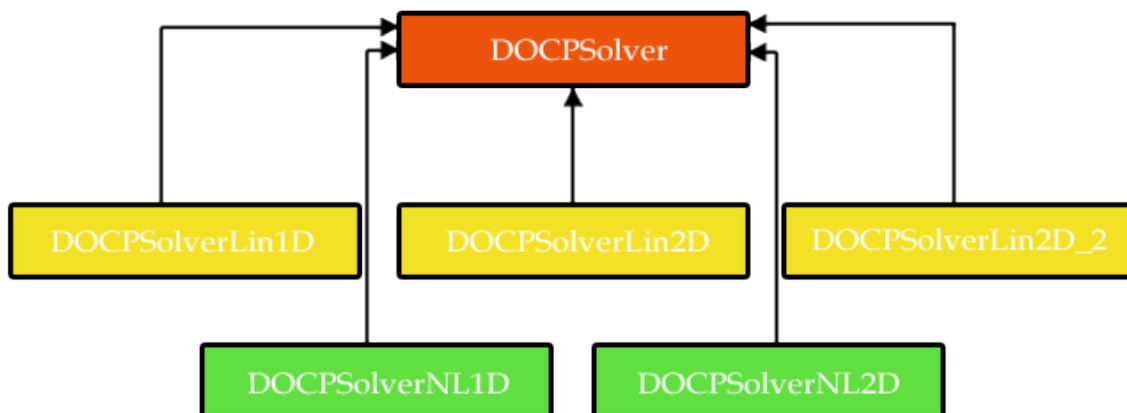


Abbildung 6.5: Klassendiagramm der implementierten Löser

Wie in obiger Grafik zu sehen ist, sind die Klassen `DOCPSolverLin1D`, `DOCPSolverNL1D`, `DOCPSolverLin2D`, `DOCPSolverLin2D_2` und `DOCPSolverNL2D` von der Basisklasse `DOCPSolver` abgeleitet. Es wird kurz auf die Bedeutung der Namen eingegangen:

- `DOCPSolver`:
Die Abkürzung OCP steht für optimal control problem, der Buchstabe D deutet an, dass dazu ein **direktes** Verfahren verwendet wird. Der Name `DOCPSolver` gibt also wieder, dass es sich um einen Löser eines Optimalsteuerungsproblems mittels einer direkten Methode handelt.
- `DOCPSolverLin1D`:
Mit Hilfe dieser Klasse lassen sich lineare (**Lin** = linear) 1D-Probleme lösen.
- `DOCPSolverNL1D`:
Diese Klasse ermöglicht die Lösung nichtlinearer (**NL** = nonlinear) 1D-Probleme.
- `DOCPSolverLin2D` und `DOCPSolverLin2D_2`:
Zur Lösung linearer 2D-Probleme wurden diese beiden Klassen implementiert. Mit ihnen lassen sich unterschiedliche Typen von Beispielen lösen, was in den Testbeispielen veranschaulicht wird.
- `DOCPSolverNL2D`:
Diese Klasse wird eingesetzt, um nichtlineare 2D-Probleme zu lösen.

Die Basisklasse `DOCPSolver`

Die wichtigsten Datenelemente der Basisklasse `DOCPSolver` werden aufgezählt und beschrieben:

- Das `GridFE`-Handle `grid` gibt das verwendete Finite-Element-Gitter wieder.
- Die beiden `FieldFE`-Handles `y_curr` und `y_prev` enthalten den Zustand des Systems zum aktuellen Zeitpunkt l bzw. zum vorherigen Zeitpunkt $l - 1$. Es handelt sich um Skalarfelder über dem Finite-Element-Gitter, bei denen an jedem Knoten ein entsprechender Wert gegeben ist.
- Im `FieldsFE`-Handle `y_all_times` sind die Felder aller diskreten Zeitpunkte gespeichert. Der l -te Eintrag beinhaltet den Zustand des Systems zum l -ten Zeitpunkt.
- Das `LinEqAdmFE`-Handle `lineq` und der `Vec(real)`-Vektor `linsol` werden zur Speicherung und Lösung des linearen Systems benötigt.
- Zur Verwaltung der Freiheitsgrade der FEM-Approximation wird das `DegFreeFE`-Handle `dof` eingesetzt. Es ermöglicht, dass die Werte des Skalarfelds `y_curr` in einen Vektor von Unbekannten des linearen Gleichungssystems und zurück umgewandelt werden³.
- Das `TimePrm`-Handle `tip` enthält Zeitangaben wie Zeitschrittweite und aktuellen Zeitschritt.
- Weitere Datenelemente der Basisklasse stellen die `real`-Werte `u_const`, `deltaT`, `lambda`, `upperBndCntrl`, `lowerBndCntrl` und `numTol` dar:
 - `u_const`: Wert der Anfangssteuerung,
 - `deltaT`: Größe des Zeitschritts (Δt),

³Falls nicht nur eine PDE, sondern ein System von PDEs gelöst werden muss, kann diese Transformation äußerst kompliziert sein.

- `lambda`: Wert des Regularisierungsfaktors λ ,
- `upperBndCntrl` und `lowerBndCntrl`: Obere und untere Steuerbeschränkungen,
- `numTol`: Numerische Toleranz als Abbruchkriterium der Optimierer.
- Folgende `int`-Variablen sind Datenattribute:
 - `ttlNmbrOfTSteps`: Gesamte Anzahl an Zeitschritten,
 - `timeStepNumber`: Aktuelle Zeitschrittnummer,
 - `numOptVar`: Anzahl der Optimierungsvariablen,
 - `maxNumIter`: Maximale Anzahl an Iterationen des NLP-Optimierers,
 - `example`: Nummer des Testbeispiels.
- In den `String`-Variablen `nlp_solver` und `methodCalcDeriv` ist gespeichert, welcher nichtlineare Optimierer (IPOPT, MMA oder SCP) bzw. welche Methode zur Berechnung des Gradienten der Zielfunktion (Vorwärts-, Rückwärts- oder zentrale Differenzen) eingesetzt werden soll.

Die wichtigsten Methoden der Klasse `DOCSolver` sind:

- `virtual void adm(MenuSystem & menu)`:
Wie im vorherigen Abschnitt beschrieben, ruft diese Funktion die Methoden `virtual void define(MenuSystem & menu)` und `virtual void scan()` auf, um die Menüeinträge zu definieren, einzulesen und um die Datenelemente zu initialisieren.
- `virtual void calcElmMatVec(int e, ElmMatVec& elmat, FiniteElement& fe)`:
Zur Berechnung der rechten Seite und der Matrix des linearen Systems wird diese Funktion aufgerufen. Sie untersucht Knoten des Netzes, ob bei ihnen Randindikatoren aktiv sind und stellt sicher, dass Randintegrale an den richtigen Stellen berechnet werden.
- `virtual void solveProblem()`:
Sie ist bekanntlich für die Steuerung der Funktionsaufrufe zur Lösung des Problems verantwortlich. Durch einen ersten Aufruf von `timeLoop()` wird die PDE für die Ausgangssteuerung gelöst, damit den Optimierern ein Startvektor übergeben werden kann. Zu den Anfangssteuerungswerten werden also die Zustandswerte auf dem gesamten Feld berechnet. Anschließend wird je nach Wahl des Optimierers IPOPT oder SCIP entprechend der Beschreibung in (6.3.1) und in (6.3.2) aufgerufen.
- `virtual void timeLoop()`:
Sie ruft, wie ebenfalls im vorherigen Abschnitt dargestellt, die Funktionen `setIC()`, `increaseTime()`, `solveAtThisTimeStep()` und `updateDataStructures()` auf. Nach Aufruf von `increaseTime()` wird zusätzlich die Zeitschrittnummer (`timeStepNumber`) inkrementiert und vor dem Aktualisieren der Datenstrukturen wird zur Speicherung der Daten des aktuellen Zeitpunkts `saveResults()` aufgerufen. Ihre Aufgabe ist es somit alle diskreten Zeitpunkte zu durchlaufen und in jedem Zeitpunkt weitere Funktionen aufzurufen, die das System zu gegebener Steuerung lösen.
- `real time(const int l)`:
Diese einfache Funktion gibt zur Zeitschrittnummer l den tatsächlichen Zeitpunkt zurück.
- `virtual void saveResults()`:
An der Stelle `timeStepNumber` des `FielsFE`-Handles `y_all_times` wird das aktuelle Skalarfeld `y_curr` gespeichert.

- `real* calcGradient(const int n, const real*x):`

Der `real`-Vektor `x` enthält die aktuellen Steuerungswerte, die die Optimierungsvariablen des Problems darstellen. Mit Hilfe dieser Methode wird der Gradient des Zielfunktional in Abhängigkeit von den Steuervariablen berechnet. Entsprechend des Inhalts der Variable `methodCalcDeriv` wird die Funktion `calcGradientFDfw()`, `calcGradientFDbw()` oder `calcGradientFDCen()` aufgerufen.

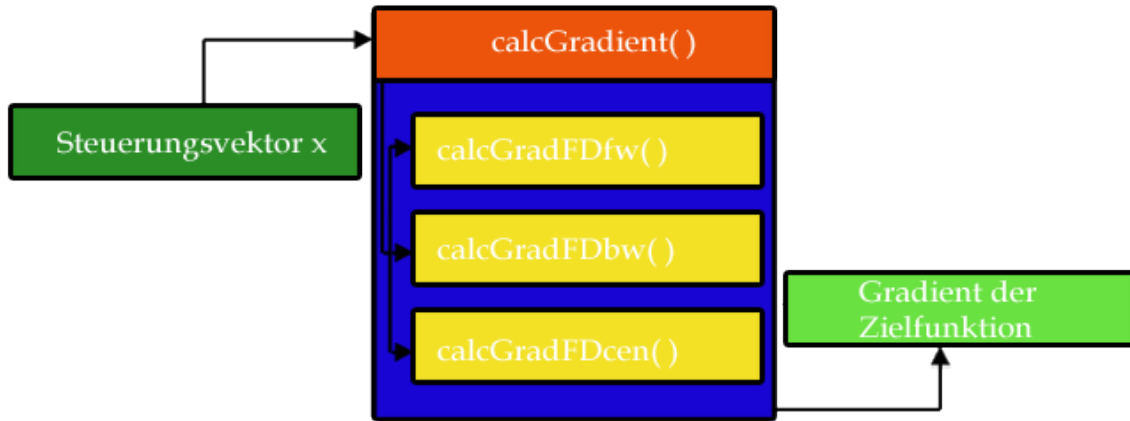


Abbildung 6.6: Gradientenberechnung des Zielfunktional

- `real* calcGradientFDfw(const int n, const real* x, real* y, real* grad_f, const real epsilon):`

Falls `methodCalcDeriv` mit "FDfw" belegt ist, wird diese Funktion zur Berechnung des Gradienten des Zielfunktional mittels Vorwärtsdifferenzen aufgerufen.

- `real* calcGradientFDbw(const int n, const real* x, real* y, real* grad_f, const real epsilon):`

Beinhaltet `methodCalcDeriv` den String "FDbw", werden durch diese Funktion Rückwärtsdifferenzen zur Ableitungsberechnung verwendet.

- `real* calcGradientFDCen(const int n, const real* x, real* y, real* z, real* grad_f, const real epsilon):`

Diese Funktion berechnet den Gradienten des Zielfunktional mittels zentraler Differenzen.

Die bislang dargestellten Methoden sind komplett in der Basisklasse implementiert und können so vollständig in den abgeleiteten Klassen verwendet werden. Bei den nachfolgenden Funktionen ist das anders. Sie sind zwar in der Basisklasse deklariert und teilweise implementiert, jedoch werden sie in den abgeleiteten Klassen (nochmal) problemspezifisch implementiert. Funktionen, die in den bisherigen Ausführungen noch nicht aufgetaucht sind, werden kurz beschrieben.

- `virtual void define(MenuSystem & menu, int level=MAIN);`

- `virtual void scan();`

- `virtual void integrands(ElmMatVec& elmat, const FiniteElement& fe);`

- `virtual void integrands4side(int side, int boind, ElmMatVec& elmat, const FiniteElement& fe);`

- `virtual void SolveAtThisTimeStep();`
- `virtual void updateDataStructures();`
- `virtual void setIC();`
- `virtual void printAsciiYandU(void):`
Diese Methode wird verwendet, um die aktuelle Steuerung und die zugehörigen Zustände an den Knotenpunkten des FE-Netzes in ein File zu schreiben.
- `virtual int getNumOfVar():`
Sie gibt die Anzahl der Optimierungsvariablen zurück.
- `virtual void setOptVar(real* optVar):`
Der Vektor `real* optVar` wird übergeben und mit den aktuellen Steuerungen, die die Optimierungswerte darstellen, besetzt.
- `virtual real evaluateF(const real* x):`
Der Vektor `x`, der der Funktion übergeben wird, beinhaltet den Optimierungsvektor, also die aktuelle Steuerung. Durch Aufruf von `timeLoop()` wird der zugehörige Zustand des Systems für die aktuelle Steuerung berechnet. Danach wird das Zielfunktional ausgewertet und der aktuelle Zielfunktionswert zurückgegeben. In nachfolgender Grafik ist die Methode zur Berechnung des Zielfunktionswertes durch `calcObjFunction()` dargestellt.

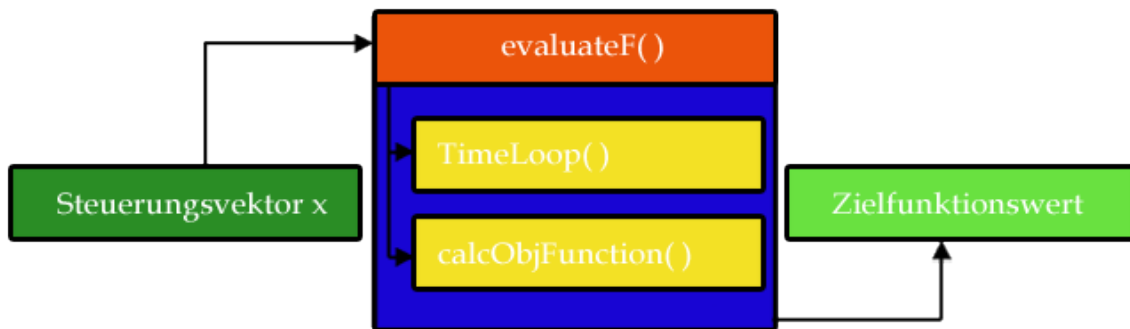


Abbildung 6.7: Berechnung des Zielfunktionswerts durch die Funktion `evaluateF()`

- `virtual real I(const Ptv(real)& p):`
Der Zustand des Systems wird initialisiert, indem durch diese Funktion die Anfangsbedingungen gesetzt werden.
- `virtual real y_hat(const Ptv(real)& p):`
Diese Methode legt den gewünschten vorgegebenen Endzustand fest, der als Trackingfunktion bezeichnet wird.

Klassen, die von der D0CPSolver-Klasse abgeleitet sind

Die beiden Klassen `D0CPSolverLin1D` und `D0CPSolverNL1D` zur Lösung von 1D-Problemen sind zusätzlich mit folgenden Datenelementen ausgestattet:

- `real u_curr`: Steuerung zum aktuellen Zeitpunkt⁴.

- `real u_prev`: Steuerung zum vorherigen Zeitpunkt⁴.
- `Vec(real) u`: Vektor mit den Steuerungen zu allen Zeitpunkten.
- `Vec(real) y_trac`: Vektor mit den Endtracking-Werten.

Neben den Datenelementen, die von `DOCPSolver` vererbt werden, besitzen die Klassen `DOCPSolverLin2D`, `DOCPSolverLin2D_2` und `DOCPSolverNL2D` folgende Attribute:

- `int noBoNodes`: Anzahl der Knotenpunkte, an denen die Steuerung wirkt.
- `Ptv(int) cntrlPts`:
Array mit den Nummern der Knoten im FE-Gitter, an denen die Steuerung eingeht.
- `Handle(FieldsFE) uFE`:
Skalarfeld der Steuerungen auf dem FE-Gitter für alle Zeitpunkte. Allerdings nehmen nur die Knotenpunkte Werte ungleich null an, bei denen die Steuerung wirkt.
- `Mat(real) u`: Matrix mit Steuerungswerten zum Schreiben in ein Ausgabefile.
- `Vec(real) y_trac`: Vektor mit den Endtracking-Werten.

Hinzu kommen bei den beiden Klassen `DOCPSolverNL1D` und `DOCPSolverNL2D`, die von der Klasse `NonLinEqSolverUDC` abgeleitet sind, zur Lösung nichtlinearer Probleme zwei Elemente und eine Methode:

- `Vec(real) nonlin_solution`:
Dieser Vektor beinhaltet die Lösung des nichtlinearen Systems. `linear_solution` hingegen speichert die Lösung des linearen approximierenden Systems, das in jeder Iteration gelöst werden muss.
- `Handle(NonLinEqSolver) nlsolver`:
Ein Objekt, das das Lösen des nichtlinearen Systems steuert, indem es unter anderem die `solve()`-Funktion aufruft.
- `virtual void makeAndSolveLinearSystem(void)`:
Diese Funktion erstellt das lineare System bei den Iterationen des Newton-Raphson-Verfahrens.

Die Methoden, die in den abgeleiteten Klassen reimplementiert sind, werden nicht noch einmal explizit genannt, da sie bereits im vorhergehenden Abschnitt aufgezählt und alle bereits mindestens einmal erklärt wurden. Lediglich auf die aufgrund von Besonderheiten erwähnenswerten Methoden wird eingegangen:

- `virtual void integrands(ElmMatVec& elmat, const FiniteElement& fe)` und
`virtual void integrands4side(int side, int boind, ElmMatVec& elmat, const FiniteElement& fe)`:

Auf die Bedeutung dieser beiden Funktionen wurde bereits hingewiesen. Exemplarisch wird anhand des Modellproblems die Implementierung dieser beiden Methoden zum l -ten Zeitpunkt auszugsweise gezeigt:

- `integrands()` im linearen Fall:

⁴Da das Problem eindimensional ist und nur Randsteuerungen behandelt werden, reicht ein einziger Wert für die Steuerung zu einem festen Zeitpunkt. In den gewählten Beispielen ist die Steuerung am linken und rechten Rand zu einem festen Zeitpunkt jeweils gleich oder sie geht nur an einem Randpunkt ein.

```

for(i = 1; i <= nbf; i++)
{
    for(j = 1; j <= nbf; j++)
    {
        elmat.A(i,j) += (fe.N(i)*fe.N(j) + theta*dt*gradNi_gradNj)*detJxW;
    }
    elmat.b(i) += (theta*dt*f(l) + yp_pt + (1-theta)*dt*f(l-1))*fe.N(i)*detJxW;
    elmat.b(i) -= (1-theta)*dt*gradNi_gradyp*detJxW;
}
- integrands4side() im linearen Fall:
for(i = 1; i <= nbf; i++)
{
    for(j = 1; j <= nbf; j++)
    {
        elmat.A(i,j) -= theta*dt*vtheta(fe.N(j))*fe.N(i)*detSideJxW;
    }
    elmat.b(i) += theta*dt*(u_curr_val - g(l))*fe.N(i)*detSideJxW;
    elmat.b(i) -= (1-theta)*dt*(vtheta(yp_pt) - u_prev_val + g(l-1))
        *fe.N(i)*detSideJxW;
}
- integrands() im nichtlinearen Fall:
for(i = 1; i <= nbf; i++)
{
    for(j = 1; j <= nbf; j++)
    {
        elmat.A(i,j) += (fe.N(i)*fe.N(j) + theta*dt*gradNi_gradNj)*detJxW;
    }
    elmat.b(i) += (y_pt-yp_pt-theta*dt*f(l)-(1-theta)*dt*f(l-1))
        *fe.N(i)*detJxW;
    elmat.b(i) += (theta*dt*gradNi_grady + (1-theta)*dt*gradNi_gradyp)
        *detJxW;
}
- integrands4side() im nichtlinearen Fall:
for(i = 1; i <= nbf; i++)
{
    for(j = 1; j <= nbf; j++)
    {
        elmat.A(i,j) += theta*dt*dvtheta(l,fe,j)*fe.N(i)
            *fe.N(j)*detSideJxW;
    }
    elmat.b(i) += theta*dt*(vtheta(y_pt) - u_curr_val + g(l))
        *fe.N(i)*detSideJxW;
    elmat.b(i) -= (1-theta)*dt*(vtheta(yp_pt) - u_prev_val + g(l-1))
        *fe.N(i)*detSideJxW;
}

```

Es ist ersichtlich, dass der Inhalt der Funktionen jeweils sehr nahe an der schwachen Formulierung ist.

- evaluateF(const real* x):

Die Zielfunktion setzt sich immer aus zwei oder mehreren Komponenten zusammen. Für

jede Komponente ist eine eigene Unterfunktion geschrieben, die eine reelle Zahl, den Beitrag der Komponente zur Zielfunktion, zurückgibt. Zu ihrer Berechnung werden in DIFFPACK vorhandene Tools zur numerischen Integration verwendet.

Die `main()`-Funktion

In der `main()`-Funktion wird nach dem Befehl `initDiffpack()` und der Initialisierung des globalen Menüs durch `global_menu.init()` ein Zeiger `p` auf ein `DOCPSolver`-Objekt angelegt. Das wird gemacht, um das Konzept der Polymorphie aus der Objektorientierung zu nutzen:

```
initDiffpack(argc, argv);
global_menu.init("Solve OCP with PDE-constraints by a direct method",
                "DOCPSolver");
DOCPSolver * p;
```

Durch Übergabe des Beispiels in Form von `exampleI`, $I \in \{1, \dots, 5\}$, wird bestimmt, welches Testbeispiel ausgeführt werden soll und dementsprechend ein Objekt des Lösesers angelegt. Ist kein Beispiel angegeben, so wird standardmäßig Testbeispiel 1 verwendet:

```
if (strcmp(argv[1], "--example") == 0)
{
    if (strcmp(argv[2], "example1") == 0)
    {
        p = new DOCPSolverLin1D();
    }
    else if (strcmp(argv[2], "example2") == 0)
    {
        p = new DOCPSolverNL1D();
    }
    :
}
else
{
    p = new DOCPSolverLin1D();
}
```

Anschließend wird nicht das übliche `global_menu.multipleLoop()` aufgerufen, sondern, da es sich bei `p` um einen Zeiger handelt, die Steuerung vom Programm selbst verwaltet. Das geschieht durch folgenden Code:

```
p->adm(global_menu);
p->solveProblem();
p->printAsciiYandU();

delete p;
```

Der Endzustand und die finale Steuerung werden durch die Funktion `printAsciiYandU()` in ein File ausgegeben. Zum Schluss wird `p` gelöscht, um Speicherplatz zu deallokieren.

Ablauf der Optimierungsroutine

Nachdem nun die hauptsächlichen Datenelemente und Methoden vorgestellt wurden, wird der Ablauf des Programms erklärt. Er ist in der Grafik 6.8 dargestellt.

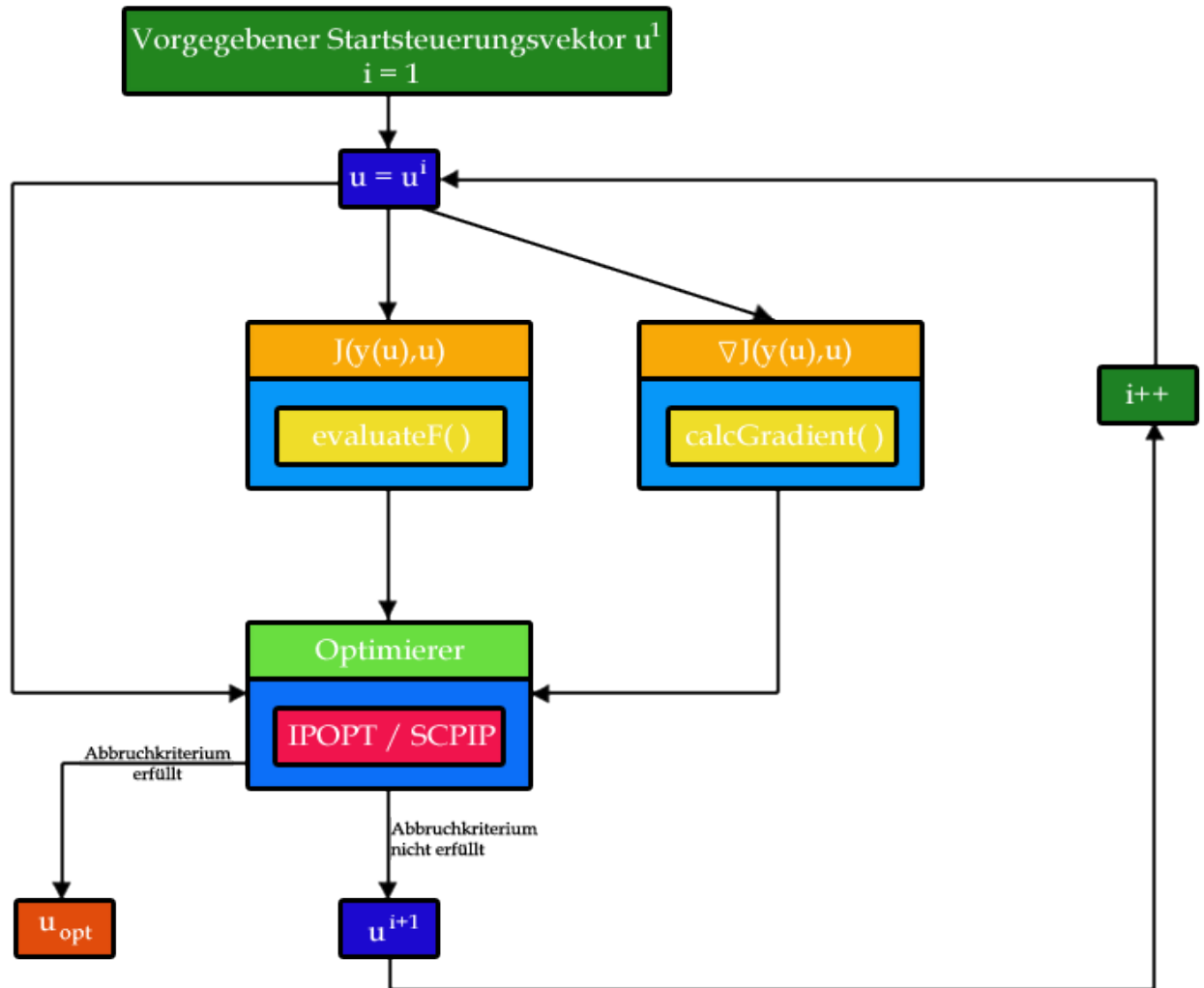


Abbildung 6.8: Ablauf der Optimierungsroutine

Ausgangspunkt der Optimierung ist ein Vektor u^1 , der die Steuerungen am Rand für alle Zeitpunkte enthält. Die Initialisierung dieses Vektors erfolgt über den Menüeintrag **set initial control**. Der Iterationszähler i hat zunächst den Wert 1.

Der Startsteuerungsvektor u wird an DIFFPACK übergeben, um die Zielfunktion $J(y(u), u)$ und den Gradienten der Zielfunktion $\nabla_u J(y(u), u)$ zu bestimmen. Das geschieht durch die Funktion `evaluateF()` bzw. `calcGradient()`. Somit hat man die Informationen, die dem Optimierer übergeben werden müssen, gesammelt (Gleichungs- und Ungleichungsnebenbedingungen sind nicht vorhanden, da alle Informationen im Zielfunktional enthalten sind). Steuerbeschränkungen stellen für den Optimierer sogenannte Box-Constraints dar. Der Optimierer findet nun anhand dieser Informationen eine neue, bessere Steuerung für das System. Ist das Abbruchkriterium erfüllt, so wird die Iteration gestoppt und der optimale Steuerungsvektor u_{opt} ausgegeben. Falls es jedoch nicht erfüllt ist, wird der Iterationszähler inkrementiert und die neue, bessere Steuerung ist Ausgangspunkt einer weiteren Iteration.

6.3 Aufruf der Optimierer aus der DIFFPACK-Umgebung

Im Eingabefile wird festgelegt, welcher Optimierer in Zusammenarbeit mit DIFFPACK das angegebene Problem lösen soll. Der Menüeintrag `set NLP-Solver = IPOPT` sorgt dafür, dass IPOPT als NLP-Löser verwendet wird, `set NLP-Solver = SCIP` bestimmt entsprechend SCIP als Löser des NLPs. Ist im Eingabefile kein NLP-Solver spezifiziert, wird standardmäßig IPOPT verwendet.

6.3.1 Aufruf von IPOPT aus DIFFPACK

Um aus der `DOCPSolver`-Klasse IPOPT aufzurufen, sind folgende Schritte notwendig: Zunächst wird ein Objekt des nichtlinearen Problems⁵ `IpOptProblem` angelegt und eine Instanz `app` der `IpoptApplication`-Klasse zum Festsetzen verschiedener Optionen und zum Aufruf des IPOPT-Solvers erzeugt. Dabei kommen sogenannte `SmartPtr`⁶ zum Einsatz:

```
SmartPtr<TNLP> mynlp = new IpOptProblem(this);
SmartPtr<IpoptApplication> app = new IpoptApplication();
```

Für die Anwendung `app` können optional verschiedene Einstellungen festgelegt werden:

- `app->Options()->SetNumericValue("tol", numTol):`
Legt die gewünschte Konvergenztoleranz fest. Der Algorithmus terminiert, sobald der (skalierte) NLP-Fehler kleiner als dieser Wert wird. In unserem Fall entspricht die Toleranz dem Wert `numTol`, der im `DOCPSolver`-Objekt bestimmt wurde.
- `app->Options()->SetStringValue("mu_strategy", "adaptive"):`
Gibt an, welche Update-Strategie für den Barriereparameter verwendet wird. Alternativ zu "adaptive" kann auch die Option "monotone" gewählt werden.
- `app->Options()->SetStringValue("output_file", "ipopt.out"):`
Bestimmt den Namen des Output-Files ("ipout.out"), in das die von IPOPT erzielten Ergebnisse geschrieben werden sollen.
- `app->Options()->SetStringValue("hessian_approximation", "limited-memory"):`
Um die zweiten Ableitungen nicht explizit angeben zu müssen, setzt man die Option "hessian_approximation" auf "limited-memory". IPOPT verwendet dann intern ein Quasi-Newton-Verfahren, um die Hessematrix der Lagrange-Funktion zu approximieren.
- `app->Options()->SetIntegerValue("max_iter", maxNumIter):`
Die maximale Anzahl an Iterationen kann mit dieser Option angepasst werden. Sie wurde im `DOCPSolver`-Objekt mit dem Integerwert `maxNumIter` festgelegt.
- `app->Options()->SetStringValue("derivative_test", "first-order"):`
Mit Hilfe dieser Einstellungsmöglichkeit kann überprüft werden, ob die übergebenen ersten Ableitungen am Startpunkt korrekt sind. IPOPT verwendet zur Kontrolle der Ergebnisse eine Finite Differenzen-Methode. Statt "first-order" können die Optionen "none" und "second-order" gesetzt werden, um keine oder Ableitungen zweiter Ordnung am Startpunkt zu verifizieren.

Hier sind nur diejenigen Optionen aufgeführt, die auch wirklich in dieser Arbeit verwendet wurden. Weitere sind in der Dokumentation zu IPOPT [3] zu finden.

⁵Unter IPOPT existiert eine eigene Klasse für nichtlineare Probleme, `TNLP` genannt.

⁶Bei `SmartPtr` handelt es sich um eine Template-Klasse, die zur geschickten Verwaltung von Zeigern benutzt wird. Statt reiner C++-Zeiger werden unter IPOPT stets `SmartPtr` verwendet.

Nun wird das `app`-Objekt entsprechend zuvor festgelegter Einstellung initialisiert, IPOPT aufgerufen und versucht, das gestellte Problem zu lösen:

```
app->Initialize();  
app->OptimizeTNLP(mynlp);
```

Von IPOPT benötigte Informationen zur Problemlösung

Damit IPOPT das NLP lösen kann, müssen ihm einige Informationen zur Verfügung gestellt werden:

1. Problemdimensionen
 - Anzahl der Variablen `n`
 - Anzahl der Nebenbedingungen `m`
2. Problemschranken
 - Schranken für die Variablen (obere Schranken: `x_u`, untere Schranken: `x_l`)
 - Schranken für die Nebenbedingungen (obere Schranken: `g_u`, untere Schranken: `g_l`)
3. Initialisierungswerte für den Startpunkt
 - Initialisierungswerte für die primalen Variablen `x`
4. Problemstruktur
 - Anzahl der NNE⁷ in der Jacobimatrix der Nebenbedingungen `nnz_jac_g`
5. Ausgewertete Problemfunktionen
Informationen, die man durch Benutzung eines gegebenen Punkts (x, λ, σ_f) kommt von IPOPT) erhält
 - Zielfunktion $f(x)$ (`obj_value`)
 - Gradient der Zielfunktion $\nabla f(x)$ (`grad_f`)
 - Hessematrix der Lagrange-Funktion $\sigma_f \nabla^2 f(x) + \sum_{i=1}^m \lambda_i \nabla^2 g_i(x)$
(Diese Angabe wird aufgrund des Setzens der Option "`hessian_approximation`" auf "`limited-memory`" nicht benötigt.)

Daneben benötigt IPOPT in vielen Fällen noch weitere Informationen, für dieses Verfahren sind sie jedoch irrelevant. Einen Gesamtüberblick über alle Informationen, die für IPOPT erforderlich sind, findet man in der IPOPT-Dokumentation [3].

Die Klasse `IpOptProblem`

In der Klasse `IpOptProblem`, die von `TNLP` abgeleitet ist, wird das nichtlineare Optimierungsproblem festgelegt. Die Bedeutung der einzelnen Attribute und Methoden wird nun erläutert:

- `DOCPSolver * m_sp;`
Das einzige Datenmember ist ein Pointer der `DOCPSolver`-Klasse `m_sp`.
- `IpOptProblem(DOCPSolver * sp):`
Neben dem Standardkonstruktor, der genauso wie der Destruktor hier nicht aufgeführt wird, ein weiterer Konstruktor, der lediglich die Funktion `InitializeProblem(sp)` mit dem Pointer `sp` als Übergabeparameter aufruft.

⁷Nicht-Null-Elemente

- `InitializeProblem(DOCPSolver * sp):`

Legt `m.sp` fest. Dabei wird der Pointer der `DOCPSolver`-Klasse `sp` übergeben und mit `m.sp` gleichgesetzt. Falls die Anzahl der Optimierungsvariablen kleiner als eins ist, gibt die Methode "false", ansonsten "true" zurück.

- `virtual bool get_nlp_info(Index & n, Index & m, Index & nnz_jac_g,
Index & nnz_h_lag, IndexStyleEnum & index_style):`

In dieser Funktion werden IPOPT die Problemdimensionen `m` und `n` übergeben sowie die Anzahl der NNE der Jacobimatrix der Nebenbedingungen und der sogenannte `index_style` festgelegt. `n` als die Anzahl der Variablen entspricht dem Attribut `numOptVar` des Members `m.sp`. Die Anzahl der Nebenbedingungen und somit auch die Anzahl der NNE der Jacobimatrix der Nebenbedingungen wird auf null gesetzt.

Beim `index_style` kann zwischen `TNLP::C_STYLE` und `TNLP::FORTRAN_STYLE` gewählt werden, wobei bei `TNLP::C_STYLE` die Indexzählung mit null, bei `TNLP::FORTRAN_STYLE` mit eins beginnt. In dieser Arbeit wird stets `TNLP::C_STYLE` verwendet.

- `virtual bool get_bounds_info(Index n, Number * x_l, Number * x_u,
Index m, Number * g_l, Number * g_u):`

Legt obere und untere Schranken der Variablen `x` und der Nebenbedingungen fest. Obere Schranken der Variablen werden mit `upperBndCntrl` belegt, untere mit `lowerBndCntrl`. Die Schranken der Nebenbedingungen sind überflüssig, da keine Nebenbedingungen vorhanden sind.

- `virtual bool get_starting_point(Index n, bool init_x, Number * x,
bool init_z, Number * z_L, Number * z_U,
Index m, bool init_lambda,
Number * lambda):`

Der Startpunkt der Iteration wird gesetzt. Die Optimierungsvariablen werden mit den `initial control`-Werten an den entsprechenden Randpunkten belegt. Das geschieht, indem einmal die Funktion `evaluateF(x)` aufgerufen wird, in der alle Anfangssteuerungen festgesetzt werden und die PDE mit diesen Anfangssteuerungen zur Berechnung der zugehörigen Zustände gelöst wird.

- `virtual bool eval_f(Index n, const Number * x, bool new_x,
Number & obj_value):`

Die vom Optimierer neu berechnete Steuerung wird in Form der Optimierungsvariablen `x` übergeben und die Funktion `evaluateF(x)` zur Lösung der PDE aufgerufen. Der Rückgabewert von `evaluateF(x)` ist der Wert der Zielfunktion zur aktuellen Steuerung.

- `virtual bool eval_grad_f(Index n, const Number * x, bool new_x,
Number * grad_f):`

Die Auswertung des Gradienten der Zielfunktion erfolgt in dieser Funktion. Es wird `calcGradient(n,x)` aufgerufen und intern mit finiten Differenzen der Gradient bestimmt. Die Größe dieses Vektors ist durch `n` festgelegt.

- `virtual bool eval_g(Index n, const Number * x, bool new_x, Index m,
Number * g):`

Da keine Nebenbedingungen für das NLP vorhanden sind, wird hier nichts gemacht.

- `virtual bool eval_jac_g(Index n, const Number * x, bool new_x,
Index m, Index nele_jac, Index * iRow,`

```
Index * jCol, Number* values):
```

Hier gilt das Gleiche wie für die vorherige Methode.

```
- virtual void finalize_solution
(SolverReturn status, Index n, const Number * x, const Number * z_L,
const Number * z_U, Index m, const Number * g, const Number * lambda,
Number obj_value, const IpoptData * ip_data, IpoptCalculatedQuantities
* ip_cq):
```

Der finale Zielfunktionswert `obj_value` und der Wert der Variablen `x` am Ende des Algorithmus werden ausgegeben bzw. in ein File geschrieben.

6.3.2 Aufruf von SCIP aus DIFFPACK

Der Aufruf von SCIP mit Hilfe eines Wrappers geschieht, indem man ein Objekt der Klasse `ScipProblem` anlegt. Dieses Objekt bzw. genauer gesagt der Zeiger auf ein `ScipProblem`-Objekt heißt wieder `mynlp`. Anschließend wird direkt die Methode `Optimize()` verwendet, in der das Problem initialisiert, SCIP aufgerufen und das NLP gelöst wird:

```
ScipProblem * mynlp = new ScipProblem(this);
mynlp->Optimize();
```

Von SCIP benötigte Informationen zur Problemlösung

SCIP benötigt eine Vielzahl von Daten zur Lösung des NLPs, die im Großen und Ganzen denen von IPOPT entsprechen. Sie werden hier nicht explizit genannt, es wird stattdessen auf die sehr ausführliche Beschreibung [30] verwiesen.

Die Klasse `ScipProblem`

Die Klasse `ScipProblem` verfügt über eine ganze Reihe von Attributen und Methoden (mehr als 50 Attribute und über 60 Methoden), von denen die für die Arbeit relevanten aufgezählt werden:

- `DOCPSolver * m_sp;`
Wie bei der Klasse `IpOptProblem` ist das einzige Datenmember ein Pointer auf ein `DOCPSolver`-Objekt.
- `ScipProblem(DOCPSolver * sp);`
Zweiter Konstruktor (neben dem Standardkonstruktor) zur Initialisierung aller nicht aufgeführten Attribute. Diese werden mit Default-Werten belegt. Außerdem wird die Funktion `InitializeProblem(DOCPSolver * sp)` aufgerufen.
- `virtual bool InitializeProblem(DOCPSolver * sp);`
Das Problem wird initialisiert, indem `m_sp` mit dem Übergabeparameter `sp` gleichgesetzt wird. Falls die Anzahl der Optimierungsvariablen kleiner als eins ist, wird `"false"`, ansonsten `"true"` zurückgegeben.

Bei den nachfolgenden Methoden gilt stets, dass bei einem Fehler der Wert 1 zurückgegeben wird. Sind alle Daten korrekt, lautet der Rückgabewert 0.

- `int PutNumDV(int iNumDV);`
Die Anzahl der Optimierungsvariablen (Design-Variablen) wird mit `iNumDV` festgelegt.

- `int PutNumEqConstr(int iNumEqConstr);`
Diese Methode setzt die Anzahl der Gleichungsnebenbedingungen auf `iNumEqConstr`. In unserem Fall ist `iNumEqConstr = 0`.
- `int PutNumIneqConstr(int iNumIneqConstr);`
Entspricht vorheriger Funktion für Ungleichungsnebenbedingungen mit `iNumIneqConstr = 0`.
- `int PutInitialGuess(double * dPtrInitialGuess);`
Belegt die Optimierungsvariablen, also die Steuerung an den jeweiligen Randpunkten, mit den entsprechenden Werten der Anfangssteuerung.
- `int PutMaxNumIter(int iMaxNumIter);`
`iMaxNumIter` wird als maximale Iterationsanzahl für den Algorithmus festgelegt. Sie entspricht der Zahl, die im Eingabefile angegeben ist und wurde im Attribut `maxNumIter` von `m_sp` gespeichert.
- `int PutTermAcc(double dAcc);`
Legt die Genauigkeit des Optimierers fest und liefert somit ein Abbruchkriterium. Der Optimierer terminiert, wenn die Genauigkeit `dAcc` erreicht ist.
- `int PutOptMethod(int iOptMethod);`
Entscheidet, welche Optimierungsmethode zum Einsatz kommt. Zur Auswahl stehen `MMA` für die Methode der beweglichen Asymptoten und `SCPIP` für den SCP-Algorithmus. Welches Optimierungsverfahren verwendet werden soll, wird über das Eingabefile bestimmt.
- `int PutAsympStrat(int iAsympStrat);`
Legt die Methode zur Asymptotenwahl fest:
 - 1: Standard-Asymptotenwahl (5.2.2)
 - 2: Asymptotenwahl nach Schenk (5.2.4)
 - 3: Asymptotenwahl zur CONLIN-Approximation (5.2.5)
 - 4: Asymptotenwahl nach Svanberg (5.2.3)
- `int PutUpperBoundDV(double * dPtrUpperBound);`
Setzt obere Schranken für die Optimierungsvariablen fest, die im Eingabefile bestimmt werden und in `m_sp` als Attribut `upperBndCntrl` gespeichert sind.
- `int PutLowerBoundDV(double * dPtrLowerBound);`
Gleiches wie die vorherige Funktion für die unteren Schranken. Die Speicherung des Wertes im Eingabefile erfolgt in `m_sp` unter `lowerBndCntrl`.
- `int PutObjVal(double dObjVal)`
Mit dieser Methode wird der aktuelle Zielfunktionswert `dObjVal` übergeben.
- `int StartScp(void);`
Diese Funktion wird am Anfang jeder Iteration verwendet. Dort wird unter anderem zu Beginn Speicherplatz für die Größe der Members allokiert und die Funktion `CallFortranSCP()` aufgerufen. Ihr Rückgabewert entscheidet, ob mit der Methode `scpfct()` oder `scpgrd()` fortgefahren wird oder ob der optimale Wert bereits gefunden wurde.
- `int scpfct(void);`
Hier erfolgt der Aufruf der Funktion `evaluateF(m_X0)`. `m_X0` ist dabei der aktuelle Iterationspunkt (der Steuerungsvektor für alle Zeitpunkte), an dem das System ausgewertet wird.

- `int scpgrd(void);`

Der Gradient des Zielfunktional im aktuellen Iterationspunkt wird durch Aufruf von `calcGradient(m_N, m_X0)` bei `m_sp` bestimmt. `m_N` steht dabei für die Anzahl der Optimierungsvariablen, `m_X0` für den aktuellen Iterationspunkt. Intern werden zur Berechnung der ersten Ableitungen wieder Finite Differenzen angewandt.

- `int Optimize(void);`

In ihr werden nacheinander sämtliche vorgestellte Funktionen benutzt, um das Problem zu initialisieren und die Informationen für die Übergabe an SCPIP festzusetzen. Innerhalb dieser Funktion wird auch verwaltet, welche der Funktionen `StartScp()`, `scpfct()` oder `scpgrd()` aufgerufen werden muss.

- `CallFortranSCP();`

Diese Routine ruft schlussendlich das in Fortran programmierte SCPIP auf und stellt somit das Bindeglied zwischen der C++- und Fortranimplementierung dar.

7 Numerische Untersuchungen und Resultate

Das vorgestellte Verfahren wird nun an fünf Testbeispielen untersucht. Dabei ist bei drei Beispielen eine analytische Lösung, die zum Vergleich mit den Ergebnissen der Berechnungen dient, bekannt. Die anderen beiden Testbeispiele sind aus Veröffentlichungen entnommen, so dass auch hier Referenzwerte für die erhaltenen Ergebnisse vorliegen.

Gemeinsam ist allen fünf Testbeispielen, dass die Probleme nicht nur orts-, sondern auch zeitabhängig sind und jeweils der Anfangszustand zum Zeitpunkt $t = 0$ vorgegeben ist. Des Weiteren handelt es sich bei der Steuerung stets um eine Randsteuerung, so dass die Probleme der Klasse der *Anfangs-Randwert-Probleme* angehören.

Die Form des Zielfunktionalen variiert mitunter zu der des Modellproblems, was aber bei der Implementierung keine Schwierigkeit darstellt. Da in den Problemen das Ziel verfolgt wird, eine gewünschte Funktion y_Ω anzusteuern, bezeichnet man diese Zielfunktionale auch als *Tracking-funktionale*. Die Art der Randbedingungen (ob Neumann-, Robin- oder Stefan-Boltzmann-Randbedingung) ist im jeweiligen Fall angegeben.

Der Ort der Testbeispiele 1 und 2 ist eindimensional, die Testbeispiele 3, 4 und 5 sind zweidimensional. Testbeispiel 2 unterscheidet sich von Beispiel 1 genauso wie Testbeispiel 4 von Beispiel 3 und 5 durch das Vorhandensein einer Nichtlinearität in einer Randbedingung.

Die Anfangssteuerung zu Beginn ist für jeden Steuerungspunkt auf einen konstanten Wert festgesetzt. Eine bessere Startsteuerung beeinflusst das Verfahren insofern, dass sie die Konvergenzgeschwindigkeit erhöht.

Die Berechnungen wurden auf einem Intel Pentium M (1.73 GHz) mit 1,5 GB Arbeitsspeicher unter Windows durchgeführt.

7.1 Testbeispiel 1

Als erstes Testbeispiel dient ein Optimalsteuerungsproblem mit eindimensionaler, homogener Wärmeleitungsgleichung und gegebenem Anfangszustand sowie einer Neumann-Randbedingung.

Minimiere

$$J(y, u) := \frac{1}{2} \|y(x, T) - y_\Omega(x, T)\|_{L^2(\Omega)}^2 + \frac{\lambda}{2} \|u(t)\|_{L^2(\Sigma)}^2$$

unter

$$\begin{aligned} y_t(x, t) - y_{xx}(x, t) &= 0 & \text{in } Q = \Omega \times (0, T), \\ y_x(x, t) &= u(t) & \text{auf } \Sigma = \Gamma \times (0, T), \\ y(x, 0) &= \cos(x) & \text{in } \Omega = (0, \pi), \end{aligned}$$

mit

$$\lambda = 0.001, \quad T = 1, \quad \Gamma = \{0, \pi\}.$$

Für das Endtracking $y_\Omega(x, T) = \exp(-T) \cos(x)$ lautet die optimale Steuerung $\bar{u}(t) \equiv 0$. Der Zielfunktionswert analytisch berechnet bei optimaler Steuerung beträgt 0.0, die vorgegebene Anfangssteuerung hat konstant den Wert 2.0.

Nachfolgende Tabelle gibt die Parametereinstellungen zur numerischen Lösung des Problems mittels der implementierten Löserklasse `D0CPSolverLin1D` an:

Zeitdiskretisierung	θ	0.5
	Δt	0.1
Ortsdiskretisierung	Elementtyp	ElmB2n1D
	Anzahl der Elemente	10
	Anzahl der Knoten	11
Angaben für die Optimierer	Maximale Iterationsanzahl	100
	tol	1e-07
	Anzahl der Optimierungsvariablen	11
	Anzahl der Zustandsvariablen	121
	Art der Gradientenberechnung	FDfw

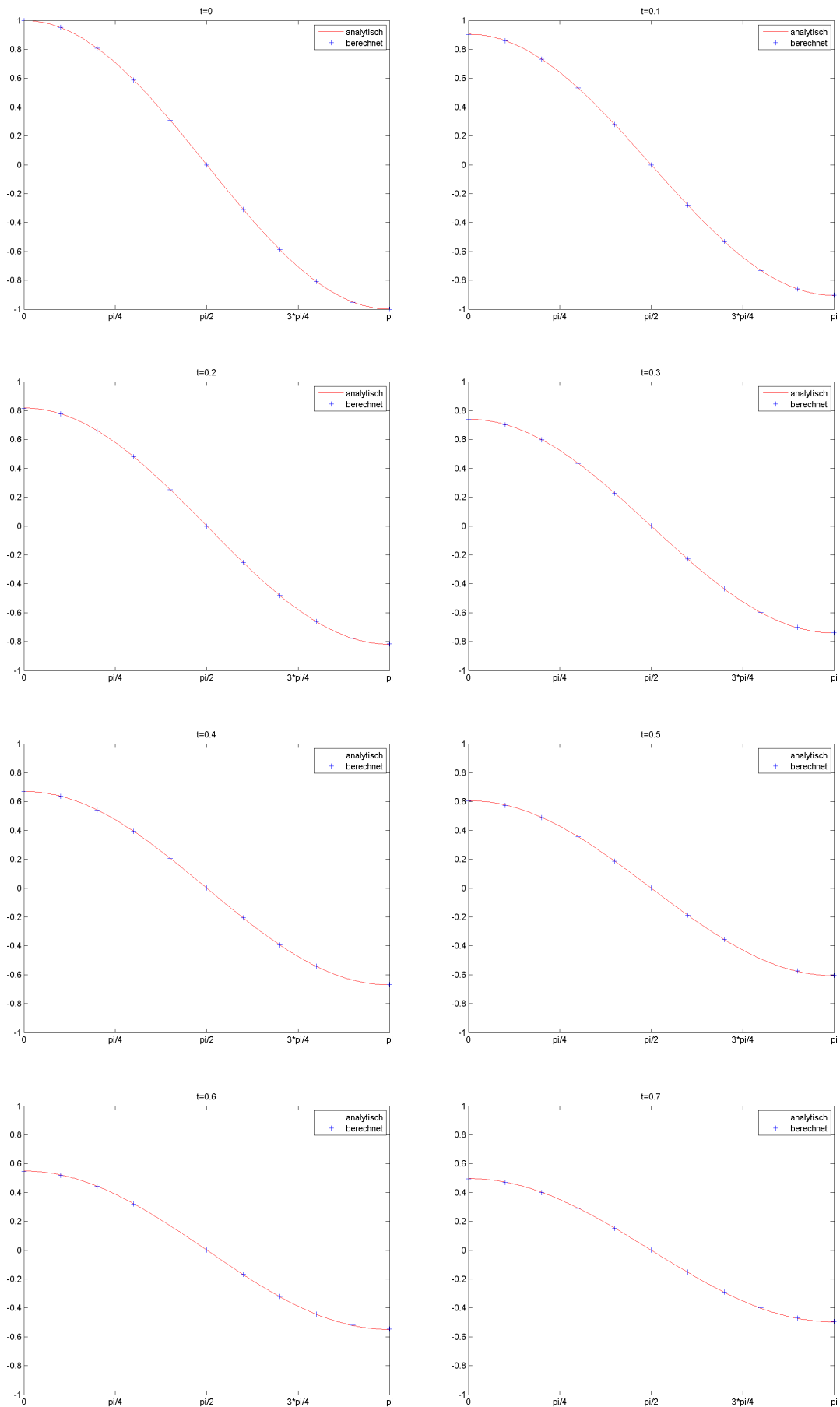
Tabelle 7.1: Problemdata für die Auswertung von Beispiel 1 mit grober Diskretisierung

Aus Tabelle 7.2 lässt sich erkennen, dass sowohl mit dem NLP-Optimierer IPOPT, als auch mit SCIP, in relativ kurzer Zeit ein Zielfunktionswert nahe 0 erreicht wird, obwohl die Diskretisierung nicht sehr fein ist. Es fällt auf, dass IPOPT schneller eine bessere Lösung erzielt als SCIP, was auf die weitaus geringere Anzahl an Iterationsschritten zurückzuführen ist. Auf eine grafische Darstellung der Steuerungen wird verzichtet, da sie betragsmäßig alle kleiner als 0.00012 sind.

Kategorie \ Optimierer	IPOPT	SCIP
Zielfunktionswert	$4.80649 \cdot 10^{-5}$	$5.09826 \cdot 10^{-5}$
Gesamte Rechenzeit in CPU-Sekunden	26 Sekunden	84 Sekunden
Anzahl an Iterationen	24	100
Grund des Iterationsabbruchs	OSF	MNI

Tabelle 7.2: Zusammenfassung der Ergebnisse für Beispiel 1 bei grober Diskretisierung

OSF steht für "optimal solution found", MNI für "Maximum Number of Iterations performed".



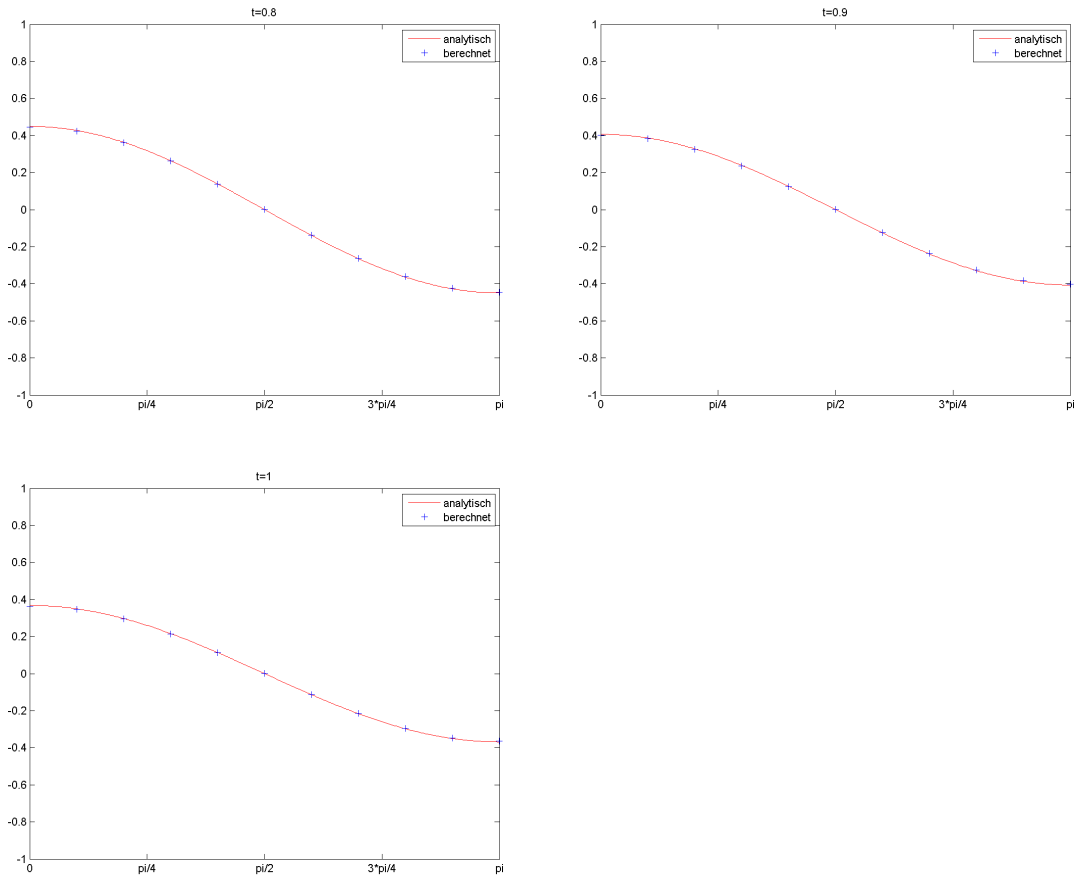


Tabelle 7.3: Vergleich der numerisch und analytisch berechneten Zustandswerte bei grober Diskretisierung für Beispiel 1

Die Abbildungen in Tabelle 7.3 zeigen, dass das Verfahren für dieses Problem bereits bei grober Diskretisierung sehr gute Ergebnisse liefert. Die numerisch berechneten Werte für den Zustand an diskreten Punkten weichen nur sehr wenig von der analytischen Lösung ab. Dennoch wurden für feinere Diskretisierungen weitere Tests durchgeführt, bei denen folgende Parametereinstellungen verwendet wurden:

Zeitdiskretisierung	θ	0.5
	Δt	0.01
Ortsdiskretisierung	Elementtyp	ElmB2n1D
	Anzahl der Elemente	100
	Anzahl der Knoten	101
Angaben für die Optimierer	Maximale Iterationsanzahl	50
	tol	1e-07
	Anzahl der Optimierungsvariablen	101
	Anzahl der Zustandsvariablen	10201
	Art der Gradientenberechnung	FDbw

Tabelle 7.4: Problemdaten für die Auswertung von Beispiel 1 mit feiner Diskretisierung

Es lässt sich auch hier erkennen, dass sowohl die mit SCIP, als auch die mit IPOPT erzielten

Kategorie \ Optimierer	IPOPT	SCIP
Zielfunktionswert	$4.90574 \cdot 10^{-9}$	$1.99067 \cdot 10^{-7}$
Gesamte Rechenzeit in CPU-Sekunden	1835 Sekunden	4323 Sekunden
Anzahl an Iterationen	21	50
Grund des Iterationsabbruchs	OSF	MNI

Tabelle 7.5: Zusammenfassung der Ergebnisse für Beispiel 1 bei feiner Diskretisierung

Ergebnisse sehr gut sind. Anzumerken bleibt erneut, dass IPOPT weit weniger Zeit zur Berechnung benötigt. Exemplarisch für die Güte der Berechnung sind die Zustandsapproximationen und die analytische Zustandskurve für die Zeitpunkte $t = 0.5$ und $t = 1$ dargestellt. Die rote Kurve mit den analytischen Zustandswerten ist nur durch genaues Hinsehen erkennbar, da sie von den numerisch berechneten Zustandswerten (blau) überdeckt ist.

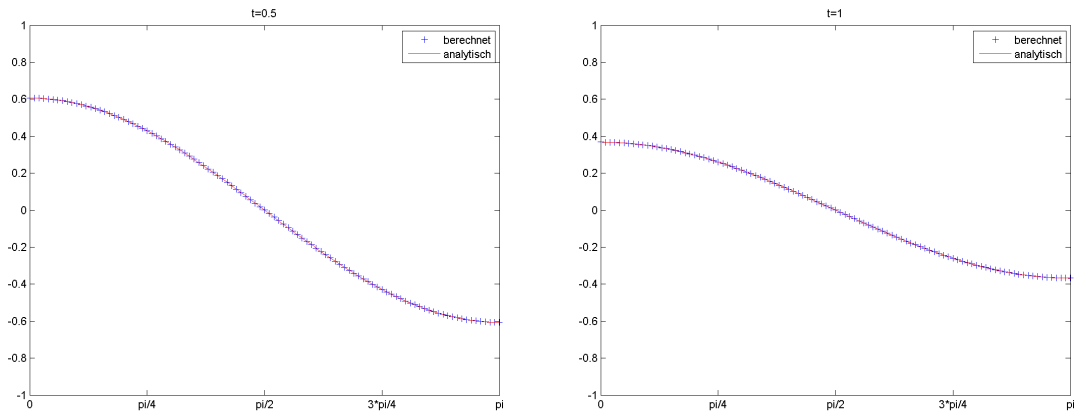


Tabelle 7.6: Vergleich der numerisch und analytisch berechneten Zustandswerte bei feiner Diskretisierung für Beispiel 1

Auf eine bei den Diskretisierungsmethoden bereits erwähnte Problematik wird hier kurz eingegangen: Wählt man den Parameter θ kleiner als 0.5, so sollte für Δt die Formel $\Delta t \leq h^2/2$ erfüllt sein. Insbesondere darf die Störung zur Bestimmung der Gradienten mittels Finiter Differenzen nicht zu klein sein, da es sonst zu numerischen Schwierigkeiten und Instabilitäten kommen kann. Für $\theta = 0.0, h = \frac{\pi}{100}$ und $\Delta t = 0.1$ wurde beispielsweise unter IPOPT die Optimierung nach 22 Iterationen mit einem Ergebnis von $1.1326 \cdot 10^{+30}$ abgebrochen, was ein Beleg für Instabilität des Verfahrens bei falschen Parametern darstellt. Deshalb gilt bei den nachfolgenden Testbeispielen stets, dass $\theta \geq 0.5$ ist.

7.2 Testbeispiel 2

Aus der Veröffentlichung „Sufficient Optimality for Discretized Control Problems“ von Hans Mittelmann [18] beziehungsweise „On an Augmented Lagrangian SQP Method for a Class of Optimal Control Problems in Banach Spaces“ von Tröltzsch [5] stammt das folgende eindimensionale parabolische Randwertproblem mit Steuerbeschränkungen. Der linke Rand des Gebiets ist durch eine Neumann-Bedingung, der rechte durch eine Stefan-Boltzmann-Bedingung festgelegt:

Minimiere

$$J(y, u) := \frac{1}{2} \|y(x, T) - y_\Omega(x)\|_{L^2(\Omega)}^2 + \frac{\lambda}{2} \|u(t)\|_{L^2(\Sigma)}^2 + \int_0^T (a_y(t)y(l, t) + a_u(t)u(t))dt$$

unter

$$\begin{aligned} y_t(x, t) - y_{xx}(x, t) &= 0 & \text{in } Q = \Omega \times (0, T), \\ y(x, 0) &= y_0(x) & \text{in } \Omega = (0, l), \\ y_x(0, t) &= 0 & \text{mit } t \in (0, T), \\ y_x(l, t) + \beta y(l, t) &= b(t) + u(t) - \varphi(y(l, t)) & \text{mit } t \in (0, T), \\ u_a &\leq u(t) \leq u_b & \text{mit } t \in (0, T), \end{aligned}$$

mit

$$\begin{aligned} l &= \pi/4, \quad T = 1, \quad \lambda = \frac{\sqrt{2}}{2}(e^{2/3} - e^{1/3}), \\ \beta &= 1, \quad u_a = 0, \quad u_b = 1, \quad y_\Omega(x) = (e + e^{-1})\cos(x), \\ a_y(t) &= -e^{-2t}, \quad a_u(t) = \frac{\sqrt{2}}{2}e^{1/3}, \quad y_0(x) = \cos(x), \\ b(t) &= \frac{1}{4}e^{-4t} - \min\left(u_b, \max\left(u_a, \frac{e^t - e^{1/3}}{e^{2/3} - e^{1/3}}\right)\right), \\ \varphi(y) &= y|y|^3. \end{aligned}$$

Die analytische Lösung dazu lautet

$$\bar{u}(t) = \min\left(u_b, \max\left(u_a, \frac{e^t - e^{1/3}}{e^{2/3} - e^{1/3}}\right)\right), \quad \bar{y}(x, t) = e^{-t}\cos(x).$$

Zeitdiskretisierung	θ	0.5
	Δt	0.1
Ortsdiskretisierung	Elementtyp	ElmB2n1D
	Anzahl der Elemente	10
	Anzahl der Knoten	11
Angaben für die Optimierer	Maximale Iterationsanzahl	100
	tol	1e-05
	Anzahl der Optimierungsvariablen	11
	Anzahl der Zustandsvariablen	121
	Art der Gradientenberechnung	FDcen

Tabelle 7.7: Problemdata für die Auswertung von Beispiel 2 bei grober Diskretisierung

Kategorie \ Optimierer	IPOPT	SCPIP
Zielfunktionswert	2.719147	2.719145
Gesamte Rechenzeit in CPU-Sekunden	67 Sekunden	177 Sekunden
Anzahl an Iterationen	9	28
Grund des Iterationsabbruchs	OSF	OSF

Tabelle 7.8: Zusammenfassung der Ergebnisse für Beispiel 2 bei grober Diskretisierung

Das obige Paar $(\bar{y}, \bar{u})$ ist gemäß [5, S.389-391] und [23, S. 227f] global optimal für das Testbeispiel, wobei beim Beweis der adjungierte Zustand p eingeht. Der Zielfunktionswert bei Einsetzen des Paares in die Zielfunktion wurde mittels einer Matlab-Routine, die auf der Daten-CD zu finden ist, berechnet und beträgt gerundet 2.719799. Die konstante Anfangssteuerung ist bei den aufgeführten Tests auf 1.0 festgesetzt. Die Klasse zur numerischen Lösung von Problemen dieser Art trägt den Namen `DOCPSolverNL1D`.

Tabelle 7.7 zeigt die Parametereinstellungen, die zur Auswertung verwendet wurden.

Wie in Tabelle 7.8 zu sehen ist, erreichte SCPIP mit den gewählten Parametern einen etwas besseren Zielfunktionswert als IPOPT, allerdings in weit mehr Zeit. Den Zielfunktionswert, den IPOPT am Ende der Optimierung ausgab, berechnete SCPIP nach 20 Iterationen und einer Laufzeit von 143 Sekunden. Insgesamt ist das erhaltene Ergebnis für beide Optimierer bei diesem Beispiel schon bei grober Diskretisierung zufriedenstellend. Bessere Ergebnisse konnten erneut durch feinere Diskretisierungen, sowohl in der Zeit, als auch im Ort, erzielt werden.

Zeitdiskretisierung	θ	0.5
	Δt	0.01
Ortsdiskretisierung	Elementtyp	ElmB2n1D
	Anzahl der Elemente	50
	Anzahl der Knoten	51
Angaben für die Optimierer	Maximale Iterationsanzahl	50
	tol	1e-05
	Anzahl der Optimierungsvariablen	101
	Anzahl der Zustandsvariablen	5151
	Art der Gradientenberechnung	FDbw

Tabelle 7.9: Problemdaten für die Auswertung von Beispiel 2 bei feiner Diskretisierung

Die Zielfunktionswerte des Löser sind bei beiden eingesetzten NLP-Optimierern der analytischen Lösung schon sehr nahe:

Kategorie \ Optimierer	IPOPT	SCPIP
Zielfunktionswert	2.719920	2.719920
Gesamte Rechenzeit in CPU-Sekunden	1507 Sekunden	6045 Sekunden
Anzahl an Iterationen	9	45
Grund des Iterationsabbruchs	OSF	OSF

Tabelle 7.10: Zusammenfassung der Ergebnisse für Beispiel 2 bei feiner Diskretisierung

Ursächlich für die längeren Laufzeiten im Vergleich zu Testbeispiel 1 (unter SCPIP 6045 Sekunden statt zuvor 4323 Sekunden) trotz groberer Diskretisierung ist das Vorhandensein der Nichtlinearitäten und dem damit verbundenen Aufruf des Newton-Raphson-Verfahrens. Es sei

daran erinnert, dass für jeden Zeitschritt ein nichtlineares Problem zu lösen ist, was durch eine Folge von linearen Problemen geschieht.

Um die Qualität der Lösungen zu veranschaulichen, wird die numerisch berechnete Steuerung, ausgewertet zu diskreten Zeitpunkten, mit der analytischen Lösung visualisiert. Die linke Grafik beinhaltet die Steuerung für $\theta = 1.0$, $\Delta t = 0.01$ und $h = \frac{\pi}{200}$, das rechte Bild stellt die Steuerungen für $\theta = 0.5$ bei sonst gleichen Einstellungen dar:

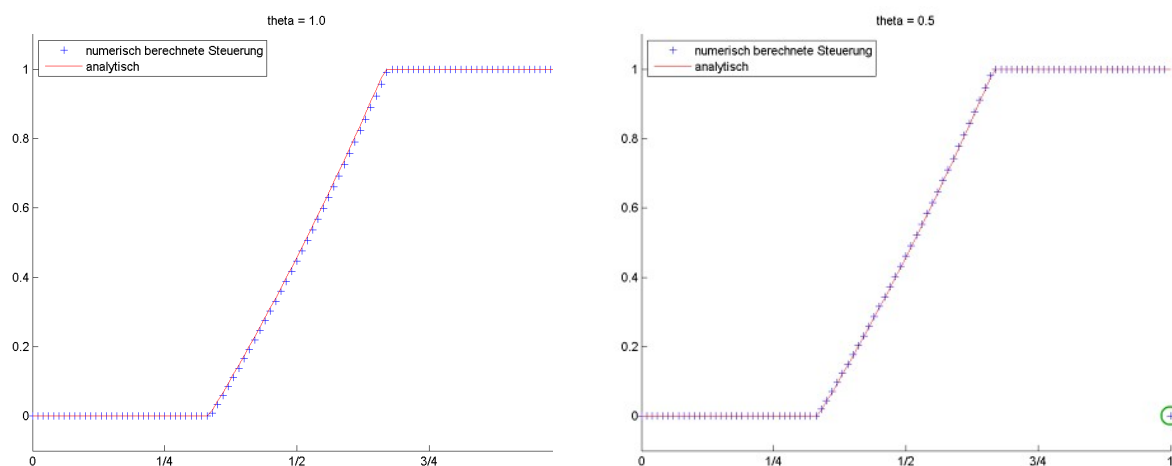
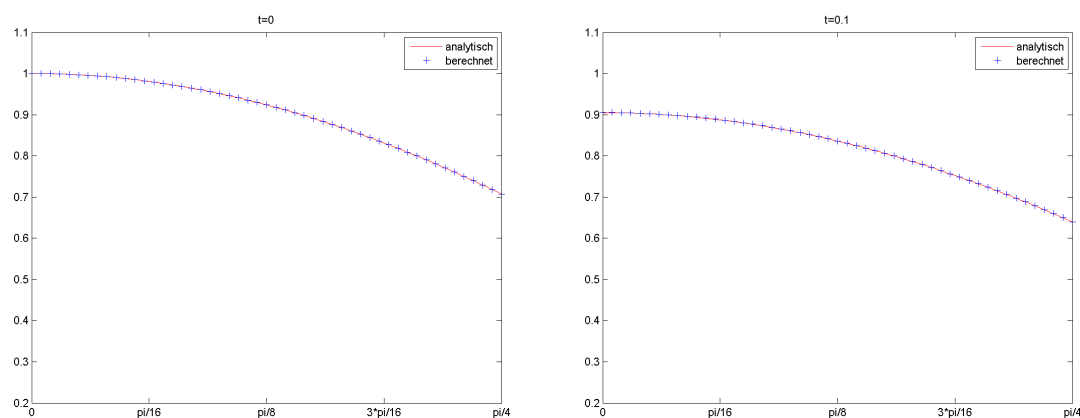
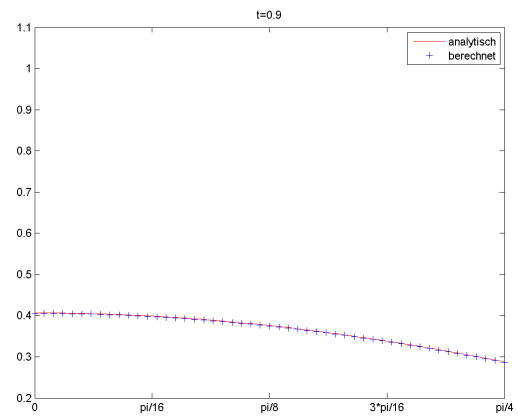
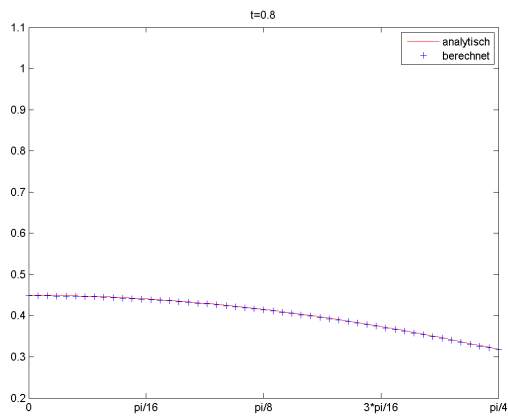
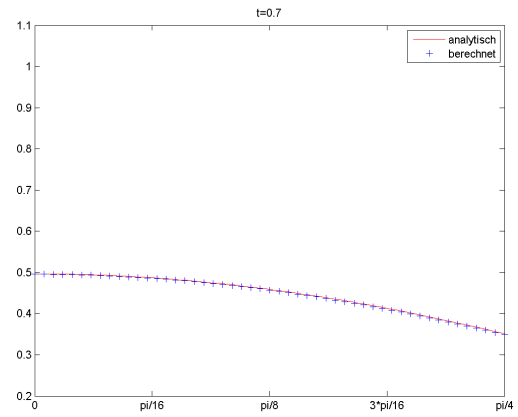
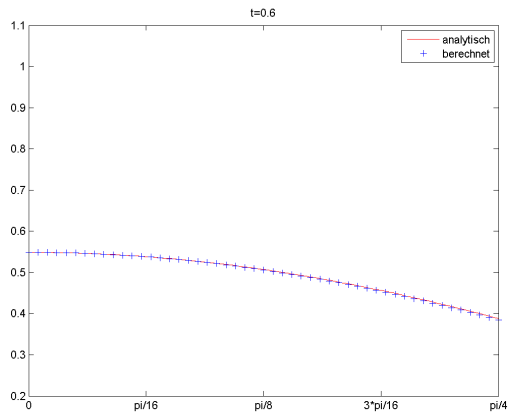
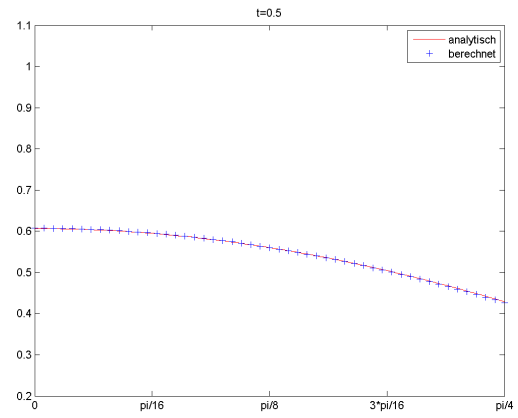
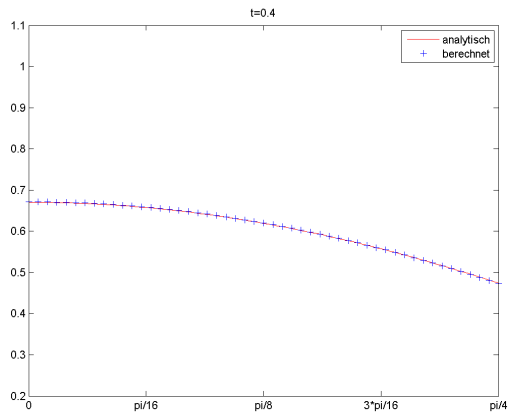
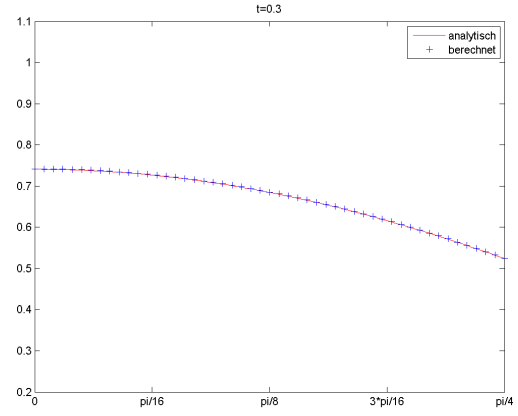
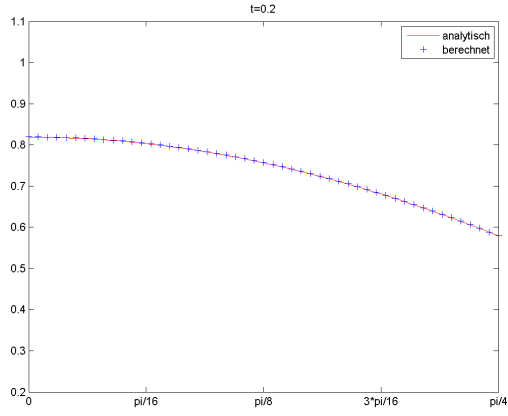


Tabelle 7.11: Numerisch und analytisch berechnete Steuerungen für Testbeispiel 2 bei feiner Diskretisierung mit $\theta = 1.0$ bzw. $\theta = 0.5$

Die numerisch berechneten Steuerungen für $\theta = 1.0$ und $\theta = 0.5$ approximieren die analytische Lösung im Rahmen der numerischen Genauigkeit gut. Besonders bei $\theta = 0.5$ ist der Zielfunktionswert (2.719888) dem der analytischen Lösung (2.719799) sehr nahe. Eine Besonderheit ist jedoch bei Anwendung des Crank-Nicolson-Verfahrens festzustellen, nämlich, dass die Steuerung zum letzten Zeitpunkt gegen Null konvergiert (in der Grafik mit einem grünen Punkt markiert). Es handelt sich dabei um sogenannte Randeﬀekte, die im Zusammenhang mit der numerischen Lösung von Optimalsteuerungsproblemen häufiger auftreten.

Die nachfolgenden Abbildungen demonstrieren, dass die zu den berechneten Steuerungen für $\theta = 1.0$ zugehörigen Zustände kaum von denen der analytischen Lösung abweichen:





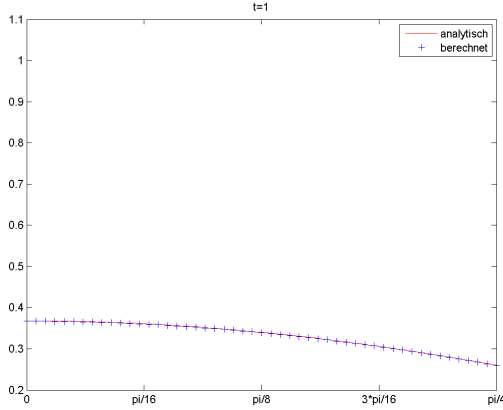


Tabelle 7.12: Vergleich der numerisch und analytisch berechneten Zustandswerte bei feiner Diskretisierung für Beispiel 2

Es ist ersichtlich, dass die berechneten Zustände des Systems zu den verschiedenen Zeitpunkten gut mit der analytischen Lösung übereinstimmen.

Wählt man den Regularisierungsfaktor $\lambda = 0$, lässt sich feststellen, dass es sich bei der optimalen Steuerung dieses Problems um eine so genannte *Bang-Bang-Steuerung* handelt. Damit wird eine Steuerungsfunktion bezeichnet, die fast überall nur die Werte der Schranken u_a und u_b annimmt und zwischen diesen umschaltet.

Mathematisch formuliert spricht man bei $u(x)$ von einer **Bang-Bang-Steuerung**, wenn für fast alle $x \in \Gamma$

$$u(x) = u_a(x) \text{ oder } u(x) = u_b(x)$$

erfüllt ist.

Im Gegensatz dazu bezeichnet man eine Steuerung, die linear in die PDE und ins Zielfunktional eingeht und bei der für fast alle $x \in \Gamma$

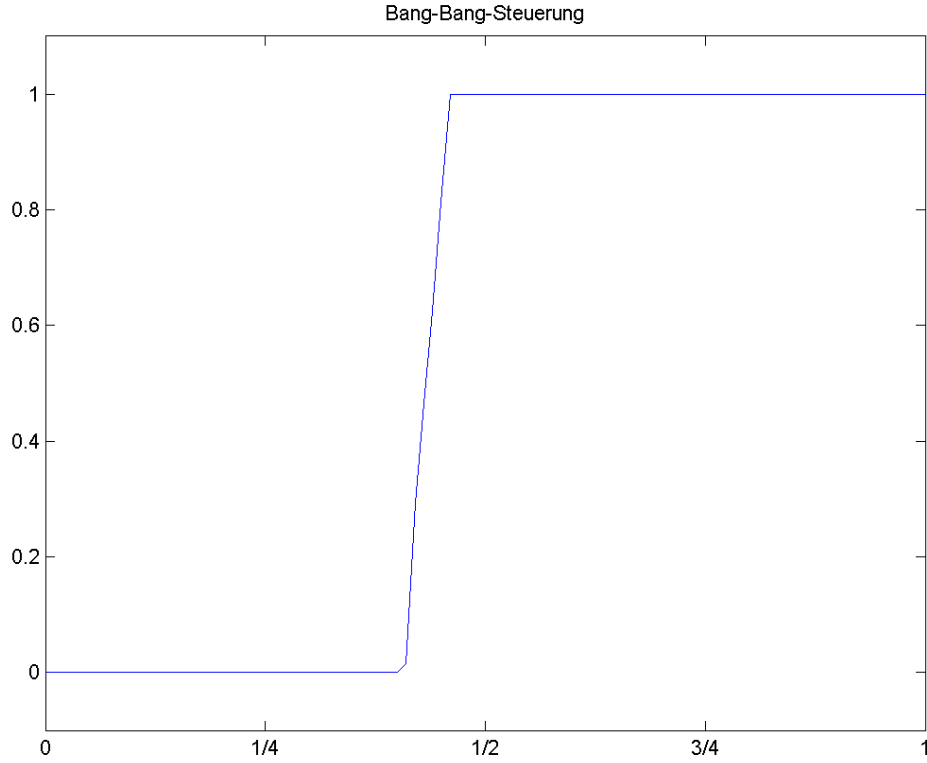
$$u_a(x) < u(x) < u_b(x)$$

gilt, als **singuläre Steuerung**.

Grafik 7.1 zeigt die durch das Verfahren errechnete optimale Steuerung (eine Bang-Bang-Steuerung) für das Problembeispiel mit gleichen Parametereinstellungen wie die linke Abbildung in 7.11, allerdings mit Regularisierungsfaktor $\lambda = 0.0$. Je feiner die Diskretisierung ist, desto stärker ist die Sprungstelle von u_a zu u_b zum Zeitpunkt $t = 0.4$ ausgeprägt.

Das Setzen des Regularisierungsfaktors λ auf einen Wert ungleich null führt also dazu, dass die Lösungskurve der Steuerungen glatter wird. Das unregulierte Problem weist eine starke Sprungstelle auf, die die mathematische Behandlung erschwert.

Führt man bei Testbeispiel 1 obere und untere Steuerbeschränkungen ein, beispielsweise $u_a = -1$ und $u_b = 1$, so wird die optimale Steuerung auch als $\bar{u} \equiv 0$ berechnet. Das ist ein Beispiel einer (total) singulären Steuerung.


 Abbildung 7.1: Bang-Bang-Steuerung für Regularisierungsfaktor $\lambda = 0$

7.3 Testbeispiel 3

Die Testbeispiele 3 und 4 sind den Veröffentlichung [10] und [26] entnommen und haben folgende Form:

Minimiere

$$J(y, u) := \frac{1}{2} \|y(x, T) - y_\Omega(x)\|_{L^2(\Omega)}^2 + \frac{\lambda}{2} \|u(x, t)\|_{L^2(\Sigma_1)}^2$$

unter

$$\begin{aligned} y_t(x, t) - \Delta_x y(x, t) &= 0 & \text{in } Q = \Omega \times (0, T), \\ y(x, 0) &= y_0(x) & \text{in } \Omega = (0, 1) \times (0, 1) \subset \mathbb{R}^2, \\ \partial_\nu y(x, t) &= b(y(x, t)) + u(x, t) & \text{auf } \Sigma_1 = \Gamma_1 \times (0, T), \\ \partial_\nu y(x, t) &= -y(x, t) & \text{auf } \Sigma_2 = \Gamma_2 \times (0, T), \\ u_a &\leq u(t) \leq u_b & \text{mit } t \in (0, T), \end{aligned}$$

mit

$$\begin{aligned} \Gamma_1 &= \{(x_1, x_2) \in \bar{\Omega} : x_2 = 1\}, \quad \Gamma_2 = \partial\Omega \setminus \Gamma_1, \\ T &= 1, \quad \lambda = 0.001, \quad u_a = 0, \quad u_b = 1, \\ y_I &\equiv 0, \quad y_T(x) = 0.5x_1x_2 + 0.25, \quad b(y) = 0. \end{aligned}$$

Die Klasse, in der dieser Problemtyp für Funktionen, die linear in $b(y)$ sind, implementiert

ist, heißt `DOCPSolverLin2D`. Als Anfangssteuerung wurde am Rand der konstante Wert 0.0 festgelegt und nachfolgende Parametereinstellungen zur Untersuchung verwendet:

Zeitdiskretisierung	θ	1.0
	Δt	0.1
Ortsdiskretisierung	Elementtyp	ElmB4n2D
	Anzahl der Elemente	10×10
	Anzahl der Knoten	121
Angaben für die Optimierer	Maximale Iterationsanzahl	20
	tol	1e-07
	Anzahl der Optimierungsvariablen	121
	Art der Gradientenberechnung	FDbw

Tabelle 7.13: Problemdata für die Auswertung von Beispiel 3 bei grober Diskretisierung

Kategorie \ Optimierer	IPOPT	SCIP
Zielfunktionswert	0.0022063	0.0022209
Gesamte Rechenzeit in CPU-Sekunden	238 Sekunden	310 Sekunden
Anzahl an Iterationen	17	20
Grund des Iterationsabbruchs	OSF	MNI

Tabelle 7.14: Zusammenfassung der Ergebnisse für Beispiel 3 bei grober Diskretisierung

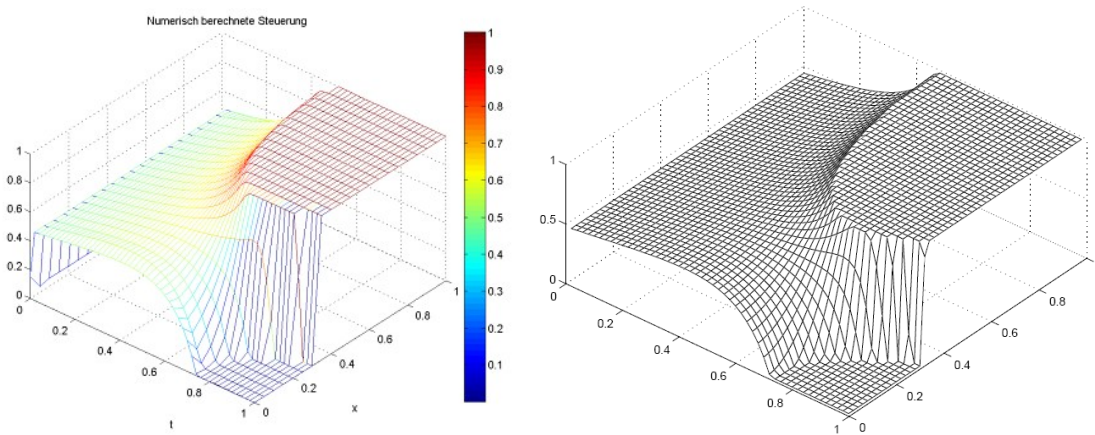


Tabelle 7.15: Vergleich der numerisch berechneten Steuerungswerte und der Ergebnisse aus [10] für Beispiel 3 bei feiner Diskretisierung

Bei grober Diskretisierung wurden mit SCIP und IPOPT fast die gleichen Steuerungswerte berechnet, was sich in einer geringen Abweichung der beiden Zielfunktionen widerspiegelt. Dieser Zielfunktionswert entspricht auch in befriedigendem Maße dem der Vorlage von Wachsmuth, der dort mit 0.00207 angegeben ist.

Im Fall der feinen Diskretisierung stimmt die von IPOPT erzielte Optimalsteuerung sehr gut mit der Referenzlösung überein, was sich auch an der grafischen Darstellung zeigt. Unter SCIP wurde die Optimierung bereits nach zwei Iterationen mit einem Funktionswert abgebrochen, der ziemlich weit von der Vorgabe abweicht. Der Grund dafür ist, dass sich die Zielfunktionswerte von erster und zweiter Iteration nicht unterscheiden und der Gradient der

Zeitdiskretisierung	θ	1.0
	Δt	0.02
Ortsdiskretisierung	Elementtyp	ElmB4n2D
	Anzahl der Elemente	20×20
	Anzahl der Knoten	441
Angaben für die Optimierer	Maximale Iterationsanzahl	20
	tol	1e-07
	Anzahl der Optimierungsvariablen	1071
	Art der Gradientenberechnung	FDbw

Tabelle 7.16: Problemdata für die Auswertung von Beispiel 3 bei feiner Diskretisierung

Kategorie \ Optimierer	IPOPT	SCPIP
Zielfunktionswert	0.0022076	0.0763861
Gesamte Rechenzeit in CPU-Sekunden	20622 Sekunden	14717 Sekunden
Anzahl an Iterationen	20	2
Grund des Iterationsabbruchs	MNI	OSF

Tabelle 7.17: Zusammenfassung der Ergebnisse für Beispiel 3 bei feiner Diskretisierung

Lagrange-Funktion bereits nach der zweiten Iteration unterhalb der vorgegebenen Toleranz lag.

7.4 Testbeispiel 4

Testfall 4 behandelt das gleiche Optimalsteuerungsproblem wie Beispiel 3, allerdings wird zusätzlich eine Nichtlinearität in die Nebenbedingungen eingebaut. Beim Aufgabentyp von Testbeispiel 3 werden nun folgende Parameter bzw. Funktionen verwendet:

$$\begin{aligned}\Gamma_1 &= \{(x_1, x_2) \in \overline{\Omega} : x_2 = 1\}, \quad \Gamma_2 = \partial\Omega \setminus \Gamma_1, \\ T &= 1, \quad \lambda = 0.001, \quad y_0 \equiv 0, \quad y_\Omega = 0.5x_1x_2 + 0.25, \\ b(y) &= -y^2, \quad u_a = 0, \quad u_b = 0.2.\end{aligned}$$

Die Anfangssteuerung ist auf den konstanten Wert 0.0 gesetzt. Die Klasse zur Implementierung dieses Problemtyps mit $b(y)$, das nichtlinear in y ist, heißt `DOCPSolverNL2D`.

Zeitdiskretisierung	θ	1.0
	Δt	0.05
Ortsdiskretisierung	Elementtyp	ElmB4n2D
	Anzahl der Elemente	15×15
	Anzahl der Knoten	256
Angaben für die Optimierer	Maximale Iterationsanzahl	10
	tol	1e-06
	Anzahl der Optimierungsvariablen	176
	Art der Gradientenberechnung	FDbw

Tabelle 7.18: Problemdaten für die Auswertung von Beispiel 4

Leider konnte nach vielfacher Überprüfung nicht das gleiche Ergebnis wie in [10] erzielt werden. Die bildliche Darstellung der Steuerungen mit linker Achse als Zeitachse und rechter Achse als Ortsachse, bei der die Randsteuerungen wirken, stimmt nicht mit den Ergebnissen von [10] überein. Eine analytische Lösung ist für dieses Beispiel nicht vorhanden, was die Suche nach möglichen Fehlern erschwert.

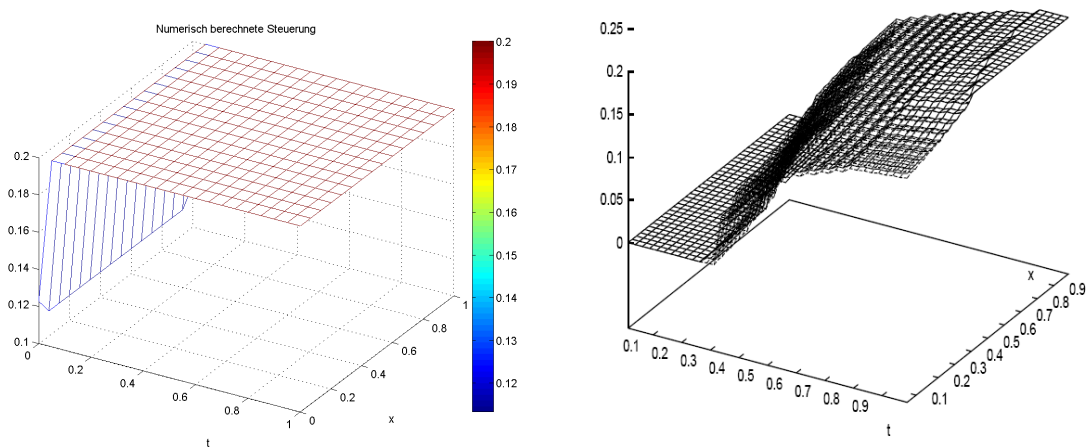


Tabelle 7.19: Vergleich der numerisch berechneten Steuerungswerte (links) und der Ergebnisse aus [10] (rechts) für Beispiel 4

Da in diesem Fall nicht mit Bestimmtheit davon ausgegangen werden kann, dass die Ergebnisse korrekt sind, wird nicht auf den Vergleich der beiden NLP-Optimierer eingegangen.

Eine mögliche Fehlerquelle ist die unbewusste Festlegung eines Parameters, der diese Ergebnisse nach sich zieht. Ein Fehler in der Programmierung oder der Herleitung kann trotz mehrmaligen Nachrechnens (auch durch weitere Personen) nicht mit Sicherheit ausgeschlossen werden. Dass numerische Fehler für die Abweichung der Resultate verantwortlich sind, wird für eher unwahrscheinlich gehalten. Weitere intensive Beschäftigungen mit den Ursachen der Diskrepanz zwischen Ist- und Soll-Lösung wären interessant und notwendig.

Obwohl sich nachweisen lässt, dass das kontinuierliche Problem eine eindeutige Lösung besitzt (siehe dazu [23, Kapitel 5]), ist es nicht ausgeschlossen, dass das diskretisierte Problem mehrere Lösungen haben kann. Insbesondere wenn der Endzustand für die optimale Steuerung des Problems weit vom Endtracking abweicht, kann es solche Fälle geben.

7.5 Testbeispiel 5

Das letzte Testbeispiel ist ein lineares Randsteuerungsproblem im Zweidimensionalen, entnommen aus der Veröffentlichung von Armin Rund „Konstruktion analytischer Testbeispiele für Randsteuerungsprobleme parabolischer Differentialgleichungen“ ([20]). Bei der Randbedingung handelt es sich um eine Robin-Randbedingung:

Minimiere

$$J(y, u) := \frac{1}{2} \|y(x, T) - y_\Omega(x)\|_{L^2(\Omega)}^2 + \frac{\lambda}{2} \|u(t)\|_{L^2(\Sigma)}^2 + \int_Q f_p y \, dxdt + \int_\Sigma g_p y \, dsdt$$

unter

$$\begin{aligned} y_t - \Delta y &= f & \text{in } Q = \Omega \times (0, T), \\ y(x, 0) &= 0 & \text{in } \Omega = [0, l]^2, \\ \partial_\nu y(x, t) + y(x, t) &= g(x, t) + u(x, t) & \text{in } \Sigma = \Gamma \times (0, T), \\ u_a &\leq u(t) \leq u_b & \text{mit } t \in (0, T), \end{aligned}$$

mit

$$\begin{aligned} l &= 1, \quad T = 1, \quad \lambda = 4, \quad u_a = 0, \quad u_b = 1, \quad y_\Omega(x) = 2\lambda, \\ f &= (x_1 - 1)^2 + (x_2 - 1)^2 - 4t, \\ g &= t((x_1 - 1)^2 + (x_2 - 1)^2 + k(x)) - \bar{u}, \\ g_p &= (x_1 - 1)^2 + (x_2 - 1)^2 - 2\lambda t + k(x), \\ f_p &= 2\lambda - 4, \\ k(x) &= \begin{cases} 0 & \text{für } x_1 + x_2 \geq 1 \\ 2 & \text{für } x_1 + x_2 < 1 \end{cases}. \end{aligned}$$

Die optimale Lösung lautet

$$\bar{y}(x, t) = t((x_1 - 1)^2 + (x_2 - 1)^2), \quad \bar{u}(t) = P_{[0,1]} \left\{ -\frac{1}{\lambda}((x_1 - 1)^2 + (x_2 - 1)^2) + 2t \right\}.$$

Der Name der Klasse, die zur Berechnung dieses Problems dient, lautet `DOCPSolverLin2D_2`. Der analytisch berechnete Zielfunktionswert bei optimaler Steuerung liegt gerundet etwa bei 28.66.

Unter beiden Optimierern lieferte das Verfahren trotz recht grober Diskretisierung bereits gute Ergebnisse.

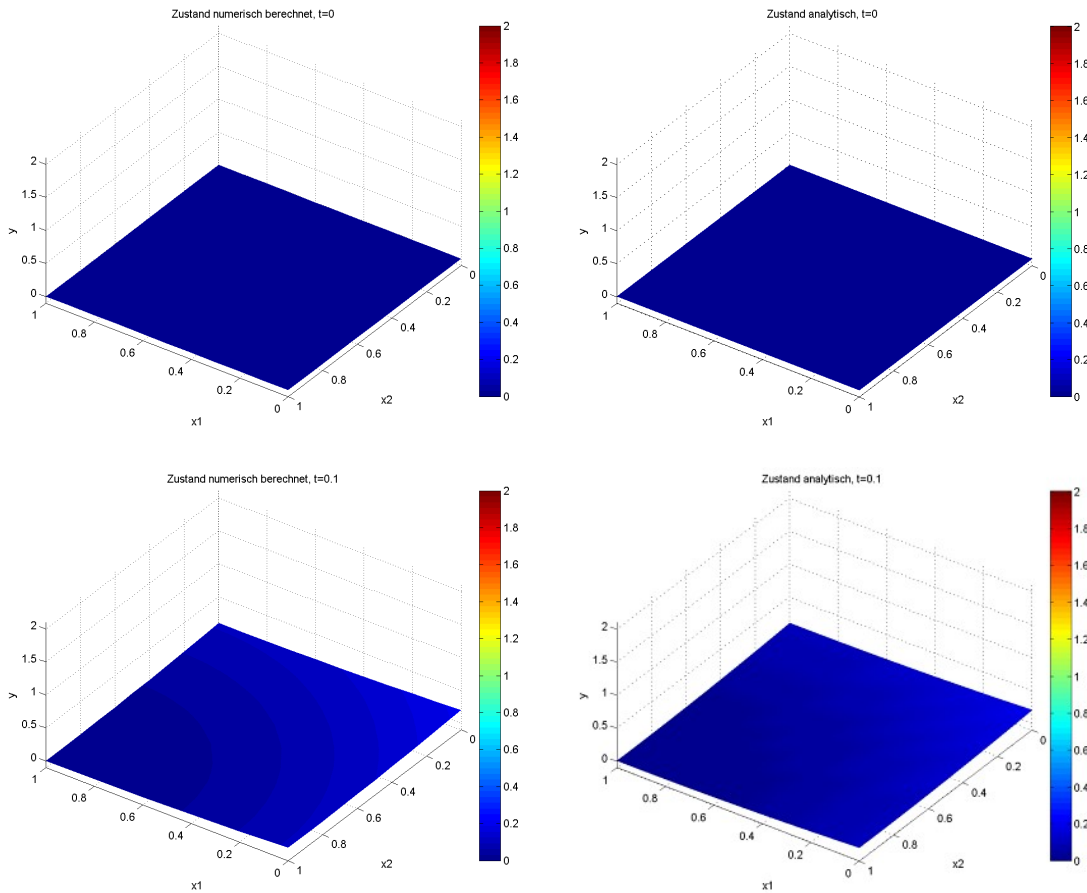
Obwohl die Rechenzeit unter IPOPT etwas geringer war, wurde ein marginal besseres Ergebnis als unter SCIP erzielt. Nachfolgende Tabelle zeigt einen grafischen Vergleich der numerisch berechneten Zustände mit den durch das Programm erhaltenen Steuerungen, links die numerische Lösung zu den verschiedenen Zeitpunkten, rechts die analytische.

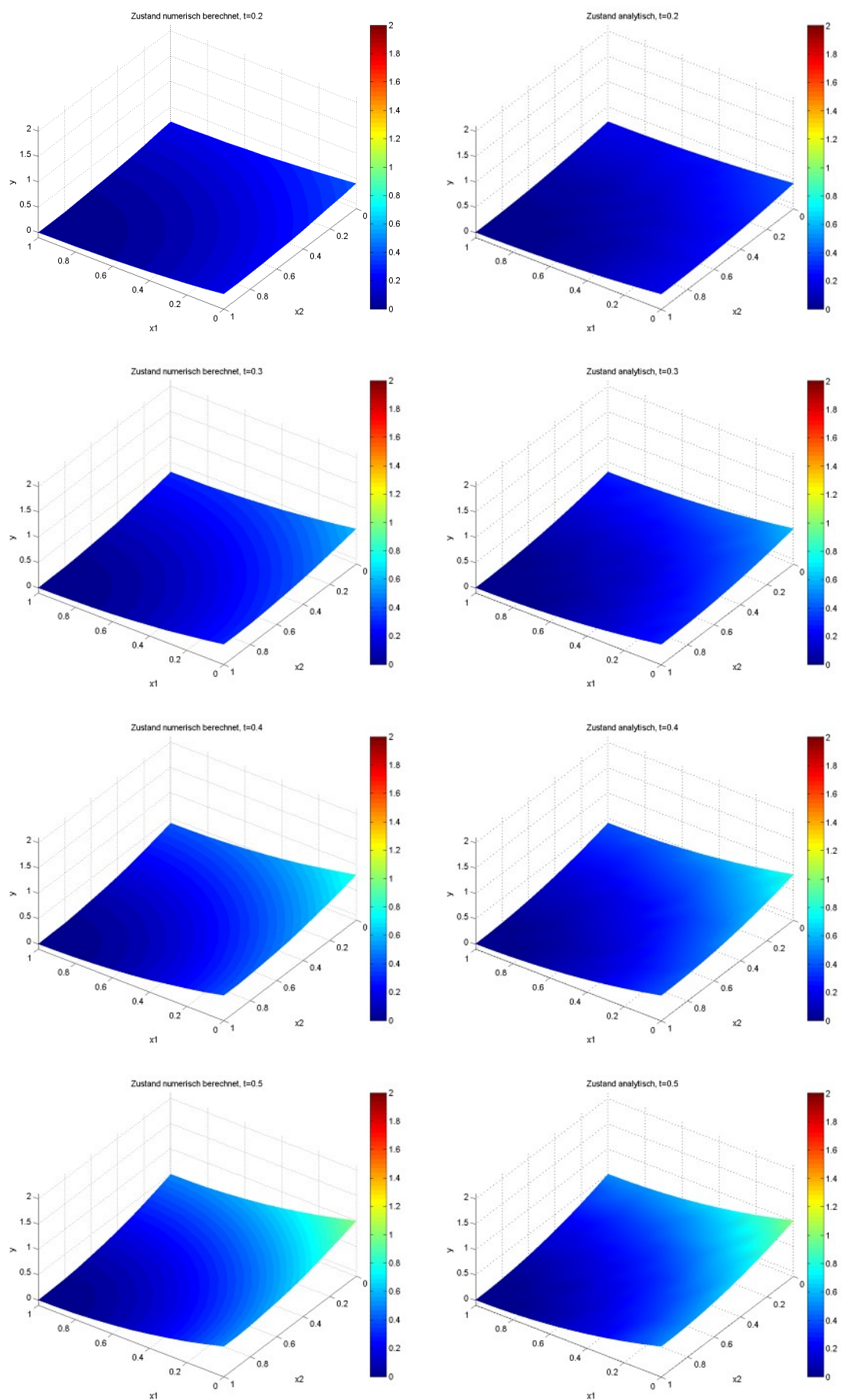
Zeitdiskretisierung	θ	1.0
	Δt	0.1
	Zeitschritte	11
Ortsdiskretisierung	Elementtyp	ElmB4n2D
	Anzahl der Elemente	10×10
	Anzahl der Knoten	121
Angaben für die Optimierer	Maximale Iterationsanzahl	25
	tol	1e-07
	Anzahl der Optimierungsvariablen	440
	Anzahl der Zustandsvariablen	1331
	Art der Gradientenberechnung	FDbw

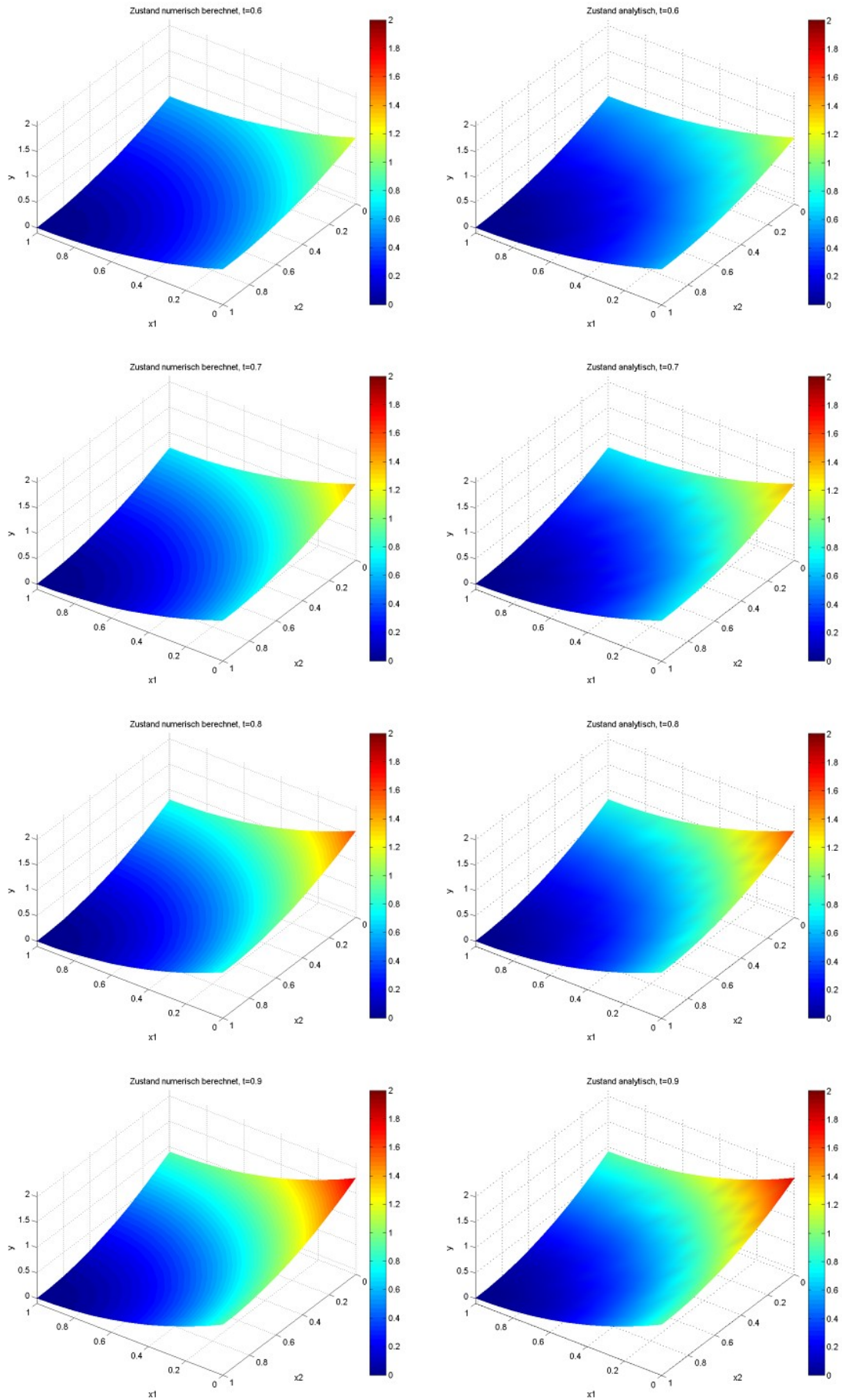
Tabelle 7.20: Problemdata für die Auswertung von Beispiel 5 bei grober Diskretisierung

Kategorie \ Optimierer	IPOPT	SCIP
Zielfunktionswert	28.16218	28.16233
Gesamte Rechenzeit in CPU-Sekunden	1306 Sekunden	1711 Sekunden
Anzahl an Iterationen	19	25
Grund des Iterationsabbruchs	OSF	MNI

Tabelle 7.21: Zusammenfassung der Ergebnisse für Testbeispiel 5 bei grober Diskretisierung







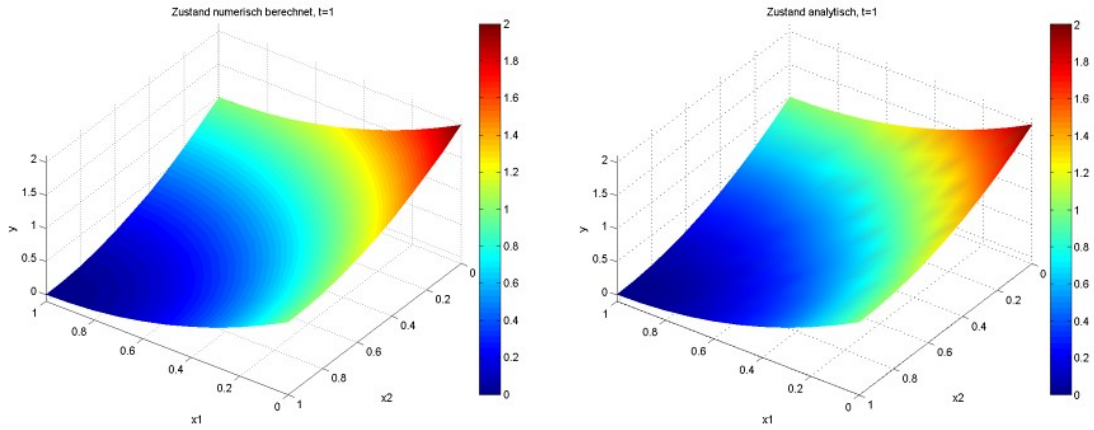


Tabelle 7.22: Vergleich der numerisch und analytisch berechneten Zustandswerte bei grober Diskretisierung für Beispiel 5

Die Abbildungen der numerisch berechneten Zustände unterscheiden sich also kaum von den analytisch berechneten.

Jedoch lässt sich auch bei diesem Testbeispiel das gleiche Phänomen wie in Beispiel 2 bemerken: Wendet man das Crank-Nicolson-Verfahren, also $\theta = 0.5$, an, so weichen die Steuerungen zum letzten diskreten Zeitpunkt an einzelnen Punkten des Randes (im höchsten Fall betragsmäßig um 0.08) von der analytischen Lösung ab, was beim impliziten Eulerverfahren nicht der Fall ist. Für $\theta = 1.0$ ist die betragsmäßig größte Abweichung der numerisch berechneten Lösung von der analytischen stets kleiner als 0.00002. Dieses Phänomen ist unabhängig vom eingesetzten NLP-Optimierer bei beiden anhand der erhaltenen Werte erkennbar. Es handelt sich erneut um die bereits erwähnten Randeffekte, die bei numerischen Berechnungen häufiger zu beobachten sind.

Auch für dieses Testbeispiel wurden weitere Tests mit feinerer Diskretisierung durchgeführt:

Zeitdiskretisierung	θ	1.0
	Δt	0.02
	Zeitschritte	51
Ortsdiskretisierung	Elementtyp	ElmB4n2D
	Anzahl der Elemente	15×15
	Anzahl der Knoten	256
Angaben für die Optimierer	Maximale Iterationsanzahl	15
	tol	1e-08
	Anzahl der Optimierungsvariablen	3060
	Anzahl der Zustandsvariablen	13056
	Art der Gradientenberechnung	FDbw

Tabelle 7.23: Problemdata für die Auswertung von Beispiel 5 bei feiner Diskretisierung

Tabelle 7.24 zeigt die erhaltenen Ergebnisse.

Wie zu erwarten, sind die Zielfunktionswerte erneut sehr viel dichter an dem der analytischen Lösung als bei der groben Diskretisierung. Durch die feinere Diskretisierung konnten die ohnehin schon guten Ergebnisse weiter verbessert werden. Allerdings geht diese Verbesserung mit erheblich längeren Laufzeiten einher. Die numerische Genauigkeit wurde bewusst auf einen

Kategorie \ Optimierer	IPOPT	SCPIP
Zielfunktionswert	28.55489	28.55485
Gesamte Rechenzeit in CPU-Sekunden	47353 Sekunden	43382 Sekunden
Anzahl an Iterationen	15	15
Grund des Iterationsabbruchs	MNI	MNI

Tabelle 7.24: Zusammenfassung der Ergebnisse für Testbeispiel 5 bei feiner Diskretisierung

sehr kleinen Wert gesetzt, damit beide Optimierer die maximale Anzahl an Iterationen erreichen. IPOPT und SCPIP erzielten ähnlich gute Zielfunktionswerte, bei der zeitlichen Dauer ist SCPIP in diesem Fall interessanterweise besser. Bei beiden ist die Laufzeit mit über 12 Stunden beträchtlich. Der Grund liegt, wie bereits erwähnt, in der Berechnung der Gradienten mittels Finiter Differenzen. Hierfür wird die meiste Zeit aufgewendet, die Berechnungen in den Optimierern benötigen lediglich einen kleinen Teil der gesamten Laufzeit.

8 Zusammenfassung und Ausblick

Im Rahmen dieser Arbeit wurde ein direktes Verfahren zur Optimalsteuerung mit parabolischen Differentialgleichungen implementiert und auf Stärken und Schwächen untersucht. Dabei wurde nicht der übliche Weg direkter Verfahren gewählt, bei dem nach der Diskretisierung Zustands- und Steuerungsvariablen als gleichwertige Variablen betrachtet werden. Um die C++-Klassenbibliothek DIFFPACK sinnvoll einzusetzen und die Dimension der entstehenden nichtlinearen Optimierungsprobleme klein zu halten, wurden lediglich die Steuerungsvariablen als Optimierungsvariablen verwendet und durch Aufruf von DIFFPACK der jeweils zugehörige Zustand berechnet.

Ein interessanter Aspekt dabei ist, dass das Verfahren drei mathematische Gebiete verbindet. Neben Ansätzen aus der Optimalsteuerungstheorie, die bei indirekten Verfahren natürlich weitaus detaillierter beschrieben sind, wurden Methoden aus der Numerik durch Finite Differenzen, FEM und das Newton-Raphson-Verfahren eingesetzt. Die nichtlineare Optimierung leistet durch Optimalitätsbedingungen, Innere-Punkte-Verfahren und weitere Methoden, die in den Optimierern implementiert sind und beleuchtet wurden, ihren Beitrag.

Insgesamt ist es gelungen, ein Verfahren anzuwenden, mit dem sich auf unkomplizierte Art und Weise Optimalsteuerungsprobleme mit parabolischen PDEs lösen lassen. Diese Methode ist allgemein genug gehalten, dass sie auch auf weitere, komplexere Probleme übertragen werden kann. Testbeispiel 2 zeigt, dass die Behandlung von Nichtlinearitäten problemlos möglich ist. Interessant wäre es, das Verfahren bei der Lösung weiterer Optimalsteuerungsprobleme wie den Optimalsteuerungsproblemen chemotaktischer Strukturbildungsprozesse aus der Arbeit von Hoffmann [14] anzuwenden.

Im Gegensatz zu den in dieser Arbeit behandelten Testbeispielen befinden sich die Nichtlinearitäten bei den Optimalsteuerungsproblemen chemotaktischer Strukturbildungsprozesse jedoch nicht in den Randbedingungen, sondern in den Zustandsgleichungen. Die Vorgehensweise in DIFFPACK mit Herleitung der schwachen Formulierung, Verwendung des Newton-Raphson-Verfahrens und Implementierung der notwendigen Klassen ist in diesen Fällen analog, lediglich gehen die Nichtlinearitäten dann nicht in die Methode `integrands4side()`, sondern in die Funktion `integrands()` ein.

Über die beiden Optimierer IPOPT und SCIP lässt sich abschließend Folgendes sagen:

Wie die Testergebnisse zeigen, lassen sich mit Hilfe der beiden Optimierer zufriedenstellende Ergebnisse erzielen, die in beiden Fällen im Rahmen der numerischen Genauigkeit übereinstimmen. Die guten Dokumentationen zum Einsatz der NLP-Optimierer erleichtern den Umgang und die Anbindung in ein DIFFPACK-Projekt deutlich.

Es bleibt festzuhalten, dass der Optimierer IPOPT besser für die Anwendung dieses Verfahrens geeignet ist als SCIP. In fast allen Fällen wird die Optimallösung durch IPOPT schneller approximiert als mit SCIP. Zu betonen ist jedoch, dass das keine allgemeine Aussage über die Qualität der Optimierer darstellt. IPOPT und SCIP sind Optimierer, die sich speziell zur Lösung von Optimierungsproblemen mit einer großen Anzahl an Variablen **und** Nebenbedingungen eignen. Da bei dieser speziellen Methode alle Informationen in das Zielfunktional eingehen und keinerlei Nebenbedingungen angegeben sind, kommen die Stärken von SCIP möglicherweise nicht zum Tragen. IPOPT hingegen kann besser mit dieser Art von Problem-

stellung umgehen.

Um die Laufzeiten der Algorithmen auch bei sehr feinen Diskretisierungen zu beschleunigen, wäre es wichtig, eine alternative Methode zur Bestimmung der Gradienten mittels Finiter Differenzen zu benutzen. Denkbar wären neben der analytischen Angabe des Gradienten der Zielfunktion, die sich jedoch in diesem Fall als relativ kompliziert erweist, Methoden aus der Sensitivitätsanalyse, beispielsweise mittels semianalytischer Ansätze.

Die Vorschläge stellen interessante Erweiterungen des angegebenen Verfahrens und des implementierten Löfers dar. Eine Weiterentwicklung der Methode wäre also durchaus sinnvoll, da somit ein flexibles Werkzeug zur Lösung einer Vielzahl verschiedener Formen von Optimalsteuerungsproblemen zur Verfügung steht.

A Anhang

A.1 Abkürzungen und Notationen

A.1.1 Abkürzungsverzeichnis

CONLIN	Convex Linearisation
CQ	Constraint Qualification
FDM	Finite-Differenzen-Methode
FEM	Finite-Elemente-Methode
FJ	Fritz John
IPOPT	Interior Point Optimizer
KKT	Karush-Kuhn-Tucker
LICQ	Linear Independance Constraint Qualification
MMA	Method of Moving Asymptotes
NLP	Nichlineares Optimierungsproblem
NNE	Nicht-Null-Elemente
PDE	Partial Differential Equation
SCP	Sequential Convex Programming
SCPIP	SCP mit Innerer-Punkte-Methode
SOC	Second Order Correction
SOP	Standard-Optimierungsproblem
SQP	Sequentielle Quadratische Optimierung

A.1.2 Notationen

Mathematische Notationen

∇	Nabla-Operator
∇^2	Hessematrix
Δ	Laplace-Operator
$ \cdot $	Absolutbetrag
$\ \cdot\ $	Euklidische Norm
$\ \cdot\ _1$	Betragssummennorm
$\ \cdot\ _\infty$	Maximumsnorm
$\frac{\partial y}{\partial x}$	Partielle Ableitung der Funktion y nach x
$\partial_\nu y$	Ableitung von y in Richtung der äußeren Normalen ν
$\overline{M}$	Abschluss der Menge M
V^*	Dualraum von V
α	Multiindex
δ_{ij}	Kronecker-Delta
d	Diskriminante einer PDE
e	$(1, \dots, 1)^T$
e_K	Konditionierungsfehler
e_R	Rundungsfehler
ess sup	Wesentliches Supremum oder Maximum
L	Differentialoperator

sgn	Signum
$(\cdot, \cdot)_H$	Skalarprodukt zum Hilbertraum H
$\text{supp } v$	Träger von v
τ	Spuroperator
θ_{lok}	Lokale Fehlerordnung
$C(\Omega)$	Raum aller auf Ω stetigen reellwertigen Funktionen
$C^k(\Omega)$	Raum aller auf Ω k -mal stetig partiell differenzierbaren reellwertigen Funktionen
$C_0^k(\Omega)$	Menge aller k -mal stetig differenzierbaren Funktionen mit kompaktem Träger in Ω
$C([a, b]; X)$	Raum aller stetig abstrakten Funktionen von $[a, b]$ nach X .
$\mathbb{H}^k(\Omega)$	Sobolevraum vom Grad k zum Index 2 ($\mathbb{H}^k(\Omega) = \mathbb{W}^{k,2}(\Omega)$)
$\mathbb{H}_0^k(\Omega)$	Sobolev-Raum vom Grad k zum Index 2 mit Nullrandwerten
$\mathbb{L}^p(\Omega)$	Raum aller auf Ω p -Lebesgue integrierbaren und messbaren Funktionen
$\mathbb{L}^2(Q)$	Raum der messbaren und quadratisch Lebesgue-integrierbaren Funktionen
$\mathbb{L}_{loc}^1(\Omega)$	Menge aller in Ω lokal integrierbarer Funktionen
$\mathbb{L}^p([a, b]; X)$	Raum der messbaren abstrakten Funktionen von $[a, b]$ nach X , die auf $[a, b]$ p -Lebesgue-integrierbar sind.
$\mathbb{W}^{k,p}(\Omega)$	Sobolev-Raum vom Grad k zum Index p
$\mathbb{W}_0^{k,p}(\Omega)$	Sobolev-Raum vom Grad k zum Index p mit Nullrandwerten
$\mathbb{W}_2^{1,0}(Q)$	Raum der auf Q quadratisch Lebesgue-integrierbaren Funktionen, die alle schwachen Ableitungen erster Ordnung in $\mathbb{L}^2(Q)$ besitzen.

Notationen der Optimalsteuerungsprobleme

$\Gamma, \partial\Omega$	Rand von Ω
$J(u, y)$	Zielfunktional, das von der Steuerung u und dem Zustand y abhängt
λ	Regularisierungsfaktor
Ω	Ortsgebiet der Zustandvariablen
Q	Raum-Zeit-Zylinder ($\Omega \times (0, T)$)
Σ	Rand bei zeitabhängigen Problemen ($\Gamma \times (0, T)$)
T	Endzeitpunkt
$u(x, t)$	Orts- und zeitabhängige Steuerung
u_a	Obere Steuerbeschränkungen
u_b	Untere Steuerbeschränkungen
$y(x, t)$	Orts- und zeitabhängiger Zustand
y_0	Anfangszustand
y_Ω	Gewünschter Endzustand, Endtracking

Notationen der Diskretisierung

Δt	Zeitschrittweite
e	Elementzähler
h	Ortsschrittweite
K	Menge der Freiheitsgrade
N	Anzahl der Basisfunktionen
N_i	Basis-, Ansatz- oder Formfunktion
$q(e, i)$	Abbildung des lokal i -ten Knotens des Elements e auf die globale Knotennummer
T_h	Triangulierung
θ	Parameter zur Zeitdiskretisierung ($0 \leq \theta \leq 1$)
$\hat{y}$	Approximation des Zustands y
y^l	Zustand y zum l -ten diskreten Zeitpunkt
ξ, η	Lokale Koordinate auf dem Referenzelement

Notationen nichtlinearer Optimierungsprobleme

α_k^x	Schrittweite in x -Richtung bei k -ter Iteration
$diag(x)$	Diagonalmatrix mit Vektor x auf der Hauptdiagonalen
d_k^x	Suche in x -Richtung bei k -ter Iteration
E_μ	Optimalitätsfehler des Barriere-Problems mit Barriereparameter μ
ϵ_{tol}	Vorgegebene Toleranzgröße
$\mathcal{F}_k$	Filter im k -ten Iterationsschritt
g	Ungleichheitsrestriktionen ($g = (g_1, \dots, g_{m_{ieq}})^T$)
h	Gleichheitsrestriktionen ($h = (h_1, \dots, h_{m_{eq}})^T$)
$I(x)$	Indexmenge der in x aktiven Ungleichungsrestriktionen
$I_0(x)$	Indexmenge der in x aktiven Ungleichungsrestriktionen mit $\lambda_i = 0$
$I_{>}(x)$	Indexmenge der in x aktiven Ungleichungsrestriktionen mit $\lambda_i > 0$
$I_{j,k}^+$	Menge der Indizes mit nicht-negativen ersten Ableitungen im k -ten Iterationsschritt
$I_{j,k}^-$	Menge der Indizes mit negativen ersten Ableitungen im k -ten Iterationsschritt
$\mathcal{L}(\cdot, \cdot, \cdot)$	Lagrange-Funktion
$L_{i,k}, U_{i,k}$	Asymptoten des Teilproblems bei MMA im k -ten Iterationsschritt
λ_i, μ_i	Lagrange-Multiplikator
m	Anzahl der Nebenbedingungen
m_{eq}	Anzahl der Gleichungsnebenbedingungen
m_{ieq}	Anzahl der Ungleichungsnebenbedingungen
μ	Barriereparameter
n	Anzahl der Variablen
φ_μ	Zielfunktion des Barriere-Problems zum Barriereparameter μ
S	Zulässiger Bereich
$\theta(x)$	Maß der Nebenbedingungsverletzung
w	Schlupfvariablen
W^{-1}	Inverse der Matrix W
x_L	Untere Schranken der Optimierungsvariablen
x_U	Obere Schranken der Optimierungsvariablen

A.2 DIFFPACK-spezifische Datentypen, Klassen und Funktionen

Die verschiedenen Datentypen, Klassen und Funktionen sind hier nur sehr kurz beschrieben. Ausführliche Informationen zu diesen und weiteren Datentypen, Klassen, etc. sind in der DIFFPACK-Dokumentation auf der Homepage [1] zu finden.

Datentypen, Zeiger und Felderverwaltung

- **Handle:**
Vergleichbar mit Zeigern in C/C++ werden Handles zur geschickten dynamischen Speicherverwaltung verwendet.
- **Mat:**
Eingesetzt zur effizienten Verwaltung von Matrizen mit Matrixeinträgen eines bestimmten Datentyps (z.B. `Mat(real)`). Die Nummerierung der Indizes beginnt, im Gegensatz zur üblichen C++-Zählweise, mit 1. Außerdem steht der Index in runden statt eckigen Klammern.
- **Ptv:**
Zeiger auf ein Datenelement, dessen Typ in Klammern spezifiziert wird (beispielsweise `Ptv(real)`). `Ptv` trägt ebenfalls zur effizienten Speicherverwaltung unter DIFFPACK bei.

- **real:**
Ersetzt den Datentyp `double` bei DIFFPACK-Anwendungen.
- **NUMT:**
Steht für NUMericalType und wird für reelle oder komplexe Zahlen verwendet.
- **Vec:**
Eingesetzt zur effizienten Verwaltung von Feldern mit Vektoreinträgen eines bestimmten Datentyps (z.B. `Vec(real)`). Die Nummerierung der Indizes beginnt, im Gegensatz zur üblichen C++-Zählweise, mit 1. Außerdem steht der Index in runden statt eckigen Klammern.

Klassen

- **DegFreeFE:**
Klasse zur Verwaltung der Freiheitsgrade Finiten Elemente bei linearen Systemen.
- **ElmItgRules:**
Klasse, die die numerische Integration über ein Finites Element ausführt, sei es im Inneren oder über die Seiten des Elements.
- **ElmMatVec:**
Klasse, die unter anderem die Elementmatrix und den -vektor bei der Programmierung Finiten-Element-Methoden zur Verfügung stellt.
- **FEM:**
Basisklasse zur Berechnung mit Finiten Elementen. Sie beinhaltet die Standard-FEM-Algorithmen mit Assemblierungsschleife über die Elemente und numerischer Integration über die Elemente.
- **FieldFE:**
Klasse, mit der ein skalares Feld bezüglich eines Finiten-Elemente-Gitters berechnet und gespeichert wird.
- **FieldsFE:**
Klasse, mit der ein Vektorfeld von `FieldFE`-Elementen implementiert wird. Im Zusammenhang mit zeitabhängigen Problemen kann beispielsweise jeder Vektoreintrag ein `FieldFE`-Element zum zugehörigen Zeitpunkt beinhalten.
- **FiniteElement:**
Klasse, die über alle Informationen zu einem Finiten Element verfügt, als da wären die Definition des Elements in seinem lokalen Koordinatensystem, die Basisfunktionen und die Geometrieabbildung an einem Auswertungspunkt, die Beziehung zwischen lokalen und globalen Knotennummern und die Knotenkoordinaten des Elements.
- **GridFE:**
Klasse, mit der ein Finites-Elemente-Gitter wiedergegeben wird, insbesondere Knotenkoordinaten, Elementtopologie und Knoten, die durch Randindikatoren gekennzeichnet sind.
- **IntegrateOverGridFE:**
Klasse, die zur Integration einer frei wählbaren, vom Benutzer vorgegebenen Größe über einem Finite-Elemente-Feld verwendet wird. Kommt insbesondere bei der Berechnung von Randintegralen zum Einsatz.

- **IntegrandCalc:**
Funktork¹, der die virtuelle Funktion `integrands()` definiert, die zur Berechnung des Integranden in Elementmatrizen und -vektoren bei Finiten-Element-Problemen eingesetzt werden kann.
- **LinEqAdmFE:**
Klasse, die ein benutzerfreundliches User-Interface für lineare Systeme und Löser bereitstellt. Unter anderem wird es zur Assemblierung der Koeffizientenmatrix und der rechten Seite verwendet.
- **MenuSystem:**
Diese Klasse ermöglicht es dem Anwender, ein relativ einfaches User-Interface zu erstellen in Form von `menu`- und `submenu`-Punkten (siehe auch (A.4)). Durch sie kann auch der Ablauf eines Programms gesteuert werden, wie es in (6.1) und (6.3) dargestellt ist.
- **NonLinEqSolver:**
Basisklasse zur Lösung von Systemen nichtlinearer algebraischer Gleichungen.
- **NonLinEqSolverUDC:**
"UDC" steht für user dependent code. Diese Klasse stattet Löser der "NonLinEqSolver"-Hierarchie mit benutzerabhängigem Code, also problemabhängigen Informationen zu nichtlinearen algebraischen Gleichungen, die gelöst werden sollen, aus. Sie ist Basisklasse für die nichtlinearen Löser in dieser Arbeit und in ihr ist unter anderem die wichtige Funktion `void makeAndSolveLinearSystem()` deklariert.
- **SaveSimRes:**
Klasse, die ein vereinfachtes und flexibles Interface zu den Tools, die zur Speicherung von Feldern und Kurven für spätere Visualisierung eingesetzt werden, bietet.
- **SimCase:**
Klasse, die den problemabhängigen Code beim Einsatz des Menuwerkzeugs `MenuSystem` wiedergibt. Sie ist unter anderem Basisklasse der FEM-Klasse.
- **TimePrm:**
Klasse, die bei zeitabhängigen Problemen zur „Zeitverwaltung“ eingesetzt wird. Sie beinhaltet Informationen zum aktuellen Zeitpunkt, zur Zeitschrittweite und verschiedene Methoden zur Beeinflussung des Systems in einem zeitlichen Bezug.

Funktionen

- **s_i:**
Standardeingabe in DIFFPACK mit `s_i >> input`.
- **s_o:**
Standardausgabe in DIFFPACK mit
`s_o << "Wert von Eingabe:" << input << "\n"`.

¹Objekte, die genauso aufgerufen werden können wie Funktionen

A.3 Inhalt der Daten-CD-ROM

Auf der beiliegenden Daten-CD-ROM sind neben einem Exemplar dieser Diplomarbeit im pdf-Format (*Ausarbeitung.pdf*) die Ordner *Daten* und *Programm* zu finden. Der Ordner *Daten* mit den Unterordnern *Testbeispiel_1* bis *Testbeispiel_5* enthält alle im Kapitel 7 aufgezeigten Ergebnisse in Dateiformat sowie die zugehörigen Eingabefiles.

Im Einzelnen sind das:

- das Eingabefile, in dem die Menüeinträge festgelegt sind (**example.i**),
- das Ausgabefile des jeweiligen Optimierers (**ipout.dat** oder **SCPERG.TXT**),
- die Datei **Info.dat** mit der Laufzeit bei der durchgeführten Optimierung,
- zwei Dateien **U_VEC_.dat**, die die Anfangssteuerung vor der Optimierung und die Endsteuerung nach der Optimierung enthalten,
- zwei Dateien **Y_MAT_.dat** mit den zugehörigen Anfangs- bzw. Endzuständen,
- MATLAB-Routinen zur Erzeugung der Abbildungen, um die Ergebnisse zu visualisieren und
- die grafischen Darstellungen der Ergebnisse.

Im Ordner *Programm* sind die Unterordner *Programmfiles* und *Examples* enthalten. Eine Projektmappe *DP.sln*, das Projekt *DOCPSolver.vcproj* und alle für diese Arbeit implementierten Files sind in *Programmfiles* zu finden; in *Examples* wurden exemplarische Dateien zur Menüeingabe zur Verfügung gestellt.

A.4 Erklärung der Menüeinträge eines Datenfiles

Die einzelnen Menüpunkte und die Untermenüs werden wieder nur sehr kurz beschrieben. Für ausführliche Informationen sei auf [17] verwiesen.

- **set example:**
Festsetzen des ausgewählten Beispiels 1-5, (default: 1).
 - **set time parameters:**
Bestimmung der Zeitparameter wie Zeitschrittweite Δt und Endzeitpunkt T , (default: **dt =0.05 t in [0,1]**).
 - **set gridfile:**
Auswahl der Zerlegung des Gitters:
 - Auswahl des Gitter-Präprozessors, z.B. **PreproBox**, **PreproStdGeom**;
 - Auswahl der Geometrie:
 - * ggf. Geometrie: **BOX**, **BOXT**, **DISK**;
 - * Dimensionen, z.B. **d=2**;
 - * Größe der Geometrie, z.B. **[0,1]x[0,1]**;
 - Bestimmung der Elemente:
 - * Elementtyp, z.B. **e=ElmB4n2D** für quadratische 2D-Elemente;
 - * Anzahl der Elemente, z.B. **[4,4]**;
 - * Abstufung der Elemente, z.B. **[1,1]**;
- (default: **P=PreproBox | d=2 [0,1]x[0,1] | d=2 e=ElmB4n2D div=[4,4] grading=[1,1]**).

- **set NLP-Solver:**
IPOPT, MMA oder SCP stehen als mögliche Optimierer zur Verfügung, (default: IPOPT).
- **set strategy for choice of asymptotes:**
 - 1: Standard-Asymptotenwahl (5.2.2),
 - 2: Asymptotenwahl nach Schenk (5.2.4),
 - 3: Asymptotenwahl zur CONLIN-Approximation (5.2.5),
 - 4: Asymptotenwahl nach Svanberg (5.2.3),
 (default: 1).
- **set calculate derivatives:**
Festlegen der Methode zur Berechnung der Gradienten. Zur Auswahl stehen FDfw für Vorwärtsdifferenzen, FDbw für Rückwärtsdifferenzen und FDcen für zentrale Differenzen, (default: FDfw).
- **set theta:**
Festlegen des Parameters θ für die Zeitdiskretisierung, $0 \leq \theta \leq 1$, (default: 0.0).
- **set redefine boundary indicators:**
Definition der Randbedingungen, (default: NONE).
- **set add boundary nodes:**
Setzen von Randbedingungen auf Teilen des Gitters, (default: NONE).
- **set remove boundary nodes:**
Entfernen von Randbedingungen auf Teilen des Gitters, (default: NONE).
- **set lambda:**
Größe des Regularisierungsfaktors $\lambda \geq 0$, (default: 0.0).
- **set initial control:**
Ausgangssteuerung an allen Punkten zu Beginn der Optimierung, (default: 2.0).
- **set upper bound control:**
Setzen oberer Steuerbeschränkungen, (default: 1.0).
- **set lower bound control:**
Festlegen unterer Steuerbeschränkungen, (default: 0.0).
- **set maximum number of iterations:**
Angabe der maximalen Anzahl an Iterationen bei Aufruf des ausgewählten Optimierers, (default: 100).
- **set numerical tolerance:**
Bestimmung der numerischen Genauigkeit des Verfahrens, die für den ausgewählten Optimierer als Abbruchkriterium verwendet wird, (default: $1e - 07$).
- **set epsilon for calculation of derivatives:**
Perturbationsgröße zur Berechnung der Ableitung mit der FDM (default: $1e - 08$).
- **set directory:**
Bestimmung des Dateipfad, in dem die Ergebnisse gespeichert werden sollen (Pfadangabe ausgehend von lokalem Speicherort des Projekts), (default: " ").

- **sub LinEqAdmFE:**

Untermenü zur Festlegung der Optionen zur Lösung des linearen Gleichungssystems. Dabei stehen eine Reihe weiterer Subuntermenüpunkte wie **sub Precond_prm** zur Einstellung eines Vorkonditionierers oder **sub Matrix_prm** zur Bestimmung des Matrixtyps (beispielsweise **sparse** bei dünnbesetzten Matrizen) zur Verfügung.

- **sub NonLinEqSolver_prm:**

Untermenü mit dem die Optionen des nichtlinearen Löfers festgelegt werden.

- **sub SaveSimRes:**

Untermenü zum Speichern der Daten.

Literaturverzeichnis

- [1] <http://www.diffpack.com>. Internetseite.
- [2] http://de.wikipedia.org/wiki/Finite_Elemente. Internetseite.
- [3] <http://www.coin-or.org/Ipopt/documentation>. Internetseite.
- [4] ALT, WALTER: *Nichtlineare Optimierung*. Vieweg-Verlag, Wiesbaden, 2002.
- [5] ARADA, NADIR, RAYMOND, JEAN-PIERRE und TRÖLTZSCH, FREDI: *On an Augmented Lagrangian SQP Method for a Class of Optimal Control Problems in Banach Spaces*. Computational Optimization and Applications, 22, 369-398, Kluwer Academic Publishers, 2002.
- [6] BENSON, HANDE, SHANNO, DAVID und VANDERBEI, ROBERT: *Interior-Point Methods for Nonconvex Nonlinear Programming: Jamming and comparative numerical Testing*. Operations Research and Financial Engineering, Princeton University, 2000.
- [7] BRAESS, DIETRICH: *Finite Elemente*. Springer-Verlag, Berlin, 1992.
- [8] BÜSKENS, CHRISTOF: *Linienmethode*. Vorlesungsskript, Universität Bayreuth, Wintersemester 2002/03.
- [9] GEIGER, KARL und KANZOW, CHRISTIAN: *Theorie und Numerik restringierter Optimierungsaufgaben*. Springer-Verlag, Berlin, 2002.
- [10] GOLDBERG, H. und TRÖLTZSCH, FREDI: *On a SQP-multigrid technique for nonlinear parabolic boundary control problems*. Veröffentlichung, Technische Universität Chemnitz-Zwickau, Fakultät für Mathematik, 1997.
- [11] GÖTSCHEL, SEBASTIAN: *Parameteridentifizierung bei parabolischen partiellen Differentialgleichungen mittels Optimaler Steuerung*. Diplomarbeit, Universität Bayreuth, 2008.
- [12] GROSSMANN, CHRISTIAN und ROOS, HANS-GÖRG: *Numerik partieller Differentialgleichungen*. Teubner-Verlag, Stuttgart, 1994.
- [13] HAFTKA, RAPHAEL und GÜRDAL, ZAFER: *Elements of Structural Optimization*. Kluwer Academic Publishers, 3. erweiterte Auflage, 1992.
- [14] HOFFMANN, KATHRIN: *Simulation und Optimale Steuerung von chemotaktischen Strukturbildungsprozessen*. Diplomarbeit, Universität Bayreuth, 2006.
- [15] JAHNY, MELANIE: *Numerische Lösung nichtlinearer Balkengleichungen mit dem Finite Elemente Software-Programm Diffpack*. Diplomarbeit, Universität Augsburg, 2008.
- [16] KNABNER, PETER und ANGERMANN, LUTZ: *Numerik partieller Differentialgleichungen*. Springer Verlag, Berlin, 2000.
- [17] LANGTANGEN, HANS PETTER: *Computational Partial Differential Equations - Numerical Methods and Diffpack Programming*. Springer-Verlag, Berlin, 2. Auflage, 2003.

- [18] MITTELMANN, HANS: *Sufficient Optimality for Discretized Parabolic and Elliptic Control Problems*, in Fast solution of discretized optimization problems. K.-H. Hoffmann, R.H.W. Hoppe, and V. Schulz (eds.), ISNM 138, Birkhäuser, Basel, 2001.
- [19] PESCH, HANS JOSEPH: *Optimal Control of Partial Differential Equations*. Vorlesungsskript, Universität Bayreuth, Sommersemester 2008.
- [20] RUND, ARMIN: *Konstruktion analytischer Testbeispiele für Randsteuerungsprobleme parabolischer Differentialgleichungen*. Private Mitteilung, 2008.
- [21] SEIDEL, CATHLEEN: *Einsatz eines Verfahrens der nichtlinearen Optimierung in der technischen Simulation*. Diplomarbeit, Universität Bayreuth, 2006.
- [22] TOMETY, EKUE-SSE SITU.: *Konjugierte Gradientenverfahren zur Lösung von Optimalsteuerungsproblemen mit semi-linearen elliptischen Differentialgleichungen*. Diplomarbeit, Universität Bayreuth, 2007.
- [23] TRÖLTZSCH, FREDI: *Optimale Steuerung partieller Differentialgleichungen - Theorie, Verfahren und Anwendungen*. Vieweg-Verlag, Wiesbaden, 2005.
- [24] THEISSEN, KARSTEN: *Optimale Steuerprozesse unter partiellen Differentialgleichungs-Restriktionen mit linear eingehender Steuerfunktion*. Dissertation, Universität Münster, 2006.
- [25] URBAN, KARSTEN: *Partielle Differentialgleichungen*. Vorlesungsskript, Universität Ulm, Sommersemester 2004.
- [26] WACHMUTH, DANIEL und TRÖLTZSCH, FREDI: *On convergence of a receding horizon method for parabolic boundary control*. Veröffentlichung, 2003.
- [27] WÄCHTER, ANDREAS und BIEGLER, LORENZ: *On the Implementation of an Interior-Point Filter Line-Search Algorithm for Large-Scale Nonlinear Programming*. Veröffentlichung, 2004.
- [28] WÄCHTER, ANDREAS und BIEGLER, LORENZ: *Line Search Filter Methods for Nonlinear Programming: Motivation and Global Convergence*. Veröffentlichung, Department of Chemical Engineering, Carnegie Mellon University, 2003.
- [29] WLOKA, JOSEPH: *Partielle Differentialgleichungen*. Teubner-Verlag, Leipzig, 1982.
- [30] ZILLOBER, CHRISTIAN: *SCIP 3.0 - Optimization technology for very large scale nonlinear programming*. Marktoberdorf, 2005.

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen.

Bayreuth, den 23. Dezember 2008

.....
Florian Loos